



Ministry of Higher Education and Scientific Research
General Directorate of Technological Studies
Higher Institute of Technological Studies of Tozeur

DEPARTMENT OF COMPUTER TECHNOLOGY

END OF STUDIES PROJECT

In order to obtain

Bachelor's Degree in Information Technologies

Speciality: Information Systems Development

SCHOOL MANAGEMENT SYSTEM

Realized by : Youcef DEGACHI
Supervised by : Mrs Haifa DEGACHI - Academic Supervisor
: Mr Mohamed FOURATI - Professional Supervisor
Host Organism : CODECOOPERATION GMBH



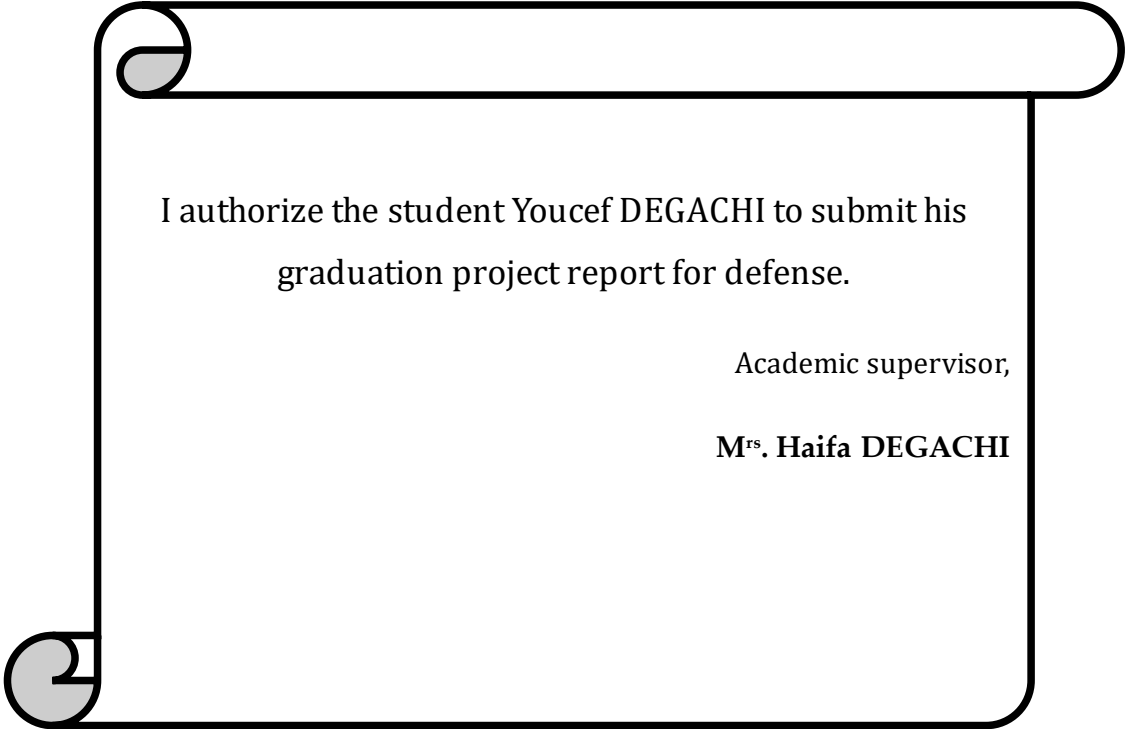
Internship period : From February 3, 2025 to May 30, 2025

Code: PFE/DSI3/2025/03



Academic Year: **2024/2025**





I authorize the student Youcef DEGACHI to submit his
graduation project report for defense.

Academic supervisor,

M^{rs}. Haifa DEGACHI

DEDICATION

To my mother «Hayet», *the guardian angel who has never ceased to watch over me. It is thanks to your prayers that I have become who I am.*

To my father «Ahmed», *for the sacrifices you have made without regret and with love, for your respect for my ideas and my choices. You have always been the friend and a shore where I have entrusted all my worries and my weaknesses.*

*This work is only the culmination of your prayers and encouragement.
May God protect you and grant you good health and long life.*

To my sisters and brothers, *with all my wishes for great success and happiness in their lives*

To my professors, *whom followed me during my studies...*

To my friends, *for the friendship that has always united us and for our good moments together...*

To all those who are dear to me

I dedicate this final graduation project...

Youcef 

ACKNOWLEDGMENTS

I would like to take a moment to thank all of those involved in finalizing this work, whether directly or indirectly.

My profound gratitude goes to the entire CodeCooperation GmbH team for their warm welcome, invaluable support, and for accompanying me throughout this wonderful journey.

Special thanks to my company supervisor and product owner, Mr. Mohamed Fourati, who has been very involved with me almost daily throughout the long period of the internship. His trust in me has truly allowed me to come very far during this project.

My deepest gratefulness also goes to Mr. Wissem Elayari and Mr. Taieb Ben Dai for their huge efforts in contributing to our weekly mentoring sessions, among other things. Their input has truly been a huge source of knowledge for me and will be forevermore. I also extend my thanks to the rest of the CodeCooperation team: Mr. Hassib Hammami, Mrs. Rayen Kacem, and Mr. Mutas Bazari.

I also wish to acknowledge the great support of my university supervisor, Mrs. Haifa Degachi, for her assistance, availability, and valuable spot-on advice. Her faith in me has truly allowed me to keep working on this project stress-free while confident this report will turn out better than initially intended.

My heartfelt appreciation goes to the jury members who agreed to evaluate my work, and I hope they find it up to standard. I equally thank all of the teachers of the Higher Institute of Technological Studies of Tozeur (ISETT) who contributed to my education during these wonderful two and a half years.

TABLE OF CONTENTS

General Introduction	1
 Chapter 1: Project Overview.....	3
Introduction.....	3
1. Host Organization: <i>Code Cooperation GmbH</i>	3
1.1. Presentation.....	3
1.2. Areas of Expertise	4
1.3. Team and Culture.....	5
1.4. Clients and Collaborations.....	5
2. General context.....	5
2.1. Problem Statement.....	6
2.2. Project's purpose.....	6
2.3. Project's challenges	7
3. Study of the existing.....	8
3.1. Analysis of Existing Systems	8
3.1.1. Traditional (Classical) Method	8
3.1.2. Popular School Management Systems	8
3.1.3. Existing Timetable Management Tools	10
3.2. Critique of Existing Systems.....	11
4. Problematic.....	12
5. Our solution.....	13
5.1. Key Features.....	13
5.2. Project Specifications Document.....	14
5.3. Project Phases.....	15
5.4. GANTT Diagram	16
Conclusion	16

Chapter 2: Functional and Conceptual Specification.....17

Introduction.....	17
1. Analysis of needs.....	17
1.1. Functional requirements.....	17
1.2. Non-functional Requirements.....	18
2. Specification.....	20
2.1. Actors identification.....	20
2.2. Use Cases Identification.....	21
3. Sequences diagrams.....	29
3.1. Sequence diagram of the use case "Authenticate"	29
3.2. Sequence diagram of the use case "display list"	30
3.3. Sequence diagram of the use case "add subject".....	31
3.4. Sequence diagram of the use case "edit teacher information".....	32
3.5. Sequence diagram of the use case "delete a class"	34
3.6. Sequence diagram of the use case "generate Timetable"	34
4. Activity diagram.....	36
5. State diagram.....	37
6. Design Class Diagram	38
Conclusion	39

Chapter 3: Engineering Process & Technologies.....40

Introduction.....	40
1. Adopted development methodology.....	40
1.1. Work Methodology.....	40
1.2. Kanban Implementation	41
1.3. Comparison with Other Methodologies.....	42
1.4. Sprint planning	43
2. Key concepts and Engineering Foundations.....	45
2.1. DevOps.....	45
2.2. Containerization and Environment Management.....	46
2.3. CI/CD	46

2.4.	Devops and deployment tools.....	47
2.4.1.	Docker.....	47
2.4.2.	Web Server.....	48
2.4.3.	Cloud Infrastructure.....	48
3.	Technological choices	50
3.1.	Operating system: Arch Linux.....	51
3.2.	Development Tools.....	51
3.2.1.	Modeling tool.....	51
3.2.2.	Visual Studio Code	51
3.2.3.	Version control system.....	52
3.2.1.	Code repository and Version Control and Collaboration	52
3.2.2.	GitHub Actions.....	53
3.2.3.	UI/UX Design.....	53
3.2.4.	Test tools	53
3.2.5.	Documentation tool.....	54
3.3.	Frontend technologies.....	54
3.3.1.	React & React native	54
3.3.2.	TanStack.....	55
3.3.3.	Interactive 3D Components Three.js	55
3.4.	Backend technologies.....	56
3.4.1.	Node.js.....	56
3.4.2.	NestJS.....	56
3.4.3.	Fastapi.....	56
3.4.4.	Python.....	57
3.5.	Database Management.....	58
4.	Algorithmic Strategies.....	58
4.1.	Timetable Generation Problem.....	59
4.2.	Hard and Soft Rules in Scheduling.....	60
4.3.	Algorithm Development and Evolution.....	60
4.3.1.	Brute Force Approach.....	60
4.3.2.	Greedy Algorithm.....	61

4.3.3.	Hybrid Greedy + Genetic Algorithm.....	61
5.	AI-Powered Features & Integrations.....	62
5.1.	Intelligent Document Processing with AI.....	62
5.2.	Speech-to-Text Integration with OpenAI Whisper.....	63
6.	Authentication & Security	64
7.	Real-time Communication.....	64
7.1.1.	WebSockets.....	64
7.1.2.	Socket.IO.....	65
8.	Architecture Concept.....	65
	Conclusion	67

Chapter 4: Implementation and validation tests.....68

	Introduction.....	68
1.	Application Screenshots & Key Features	68
1.1.	Navigation Flow & Pages management	69
1.2.	Graphic Charter (UI Guidelines)	69
1.2.1.	Logo and Branding.....	70
1.2.2.	Color Palette.....	70
1.2.3.	Typography	70
1.2.4.	Iconography.....	70
1.2.5.	Layout and Spacing.....	71
1.2.6.	Responsive Design.....	71
1.2.7.	UI Consistency Across Roles	71
1.3.	Common interfaces.....	71
1.3.1.	Homepage.....	71
1.3.1.	Navigation Bar & Side Menu.....	73
1.3.2.	The footer.....	73
1.3.3.	Authentification Pages (Login / Registration)	74
1.3.4.	Settings and Profile Management.....	77
1.4.	Super admin profile	77
1.4.1.	Database Overview.....	77

1.4.2.	Admin management.....	78
1.4.3.	Schools management.....	81
1.4.4.	Profile management.....	85
1.5.	Headmaster/admin profile.....	86
1.5.1.	Database Overview.....	87
1.5.2.	Timetable module.....	92
1.6.	Teacher profile.....	100
1.7.	Student profile.....	104
1.8.	Mobile Experience and Responsiveness	109
2.	Tests and user Insights	111
2.1.	Tests and evaluation.....	111
2.2.	Customer Feedback and User Acceptance.....	112
3.	Challenges and encountered difficulties.....	112
4.	Lessons Learned	113
5.	Impact and Real-World Deployment.....	114
6.	Future Perspective.....	115
7.	Final Thoughts.....	116
	Conclusion	116
	General Conclusion	117
	Bibliography	118

LIST OF FIGURES

Figure 1. 1: Code Cooperation logo.....	4
Figure 2. 1: Uses case diagram "superadmin"	22
Figure 2. 2: Use Cases diagram "headmaster"	23
Figure 2. 3: Use Cases diagram "teacher"	25
Figure 2. 4: Use Cases diagram "student"	27
Figure 2. 5: Use Cases diagram "parent"	28
Figure 2. 6: Use Cases diagram "guest"	29
Figure 2. 7: Sequence diagram of the use case "authenticate"	30
Figure 2. 8: Sequence diagram of the use case "display list"	31
Figure 2. 9: Sequence diagram of the use case "add subject"	32
Figure 2. 10: Sequence diagram of the use case "edit teacher"	33
Figure 2. 11: Sequence diagram of the use case "delete class"	34
Figure 2. 12: Sequence diagram of the use case "generate Timetable"	35
Figure 2. 13: Activity diagram	36
Figure 2. 14: Statechart diagram "create student"	37
Figure 2. 15: Statechart diagram "Timetable generation"	37
Figure 2. 16: Class Diagram.....	39
Figure 3. 1: Agile methodology	41
Figure 3. 2: Kanban Board Used for Project Management.....	42
Figure 3. 3: Visual representation of DevOps Life Cycle	45
Figure 3. 4: CI/CD process.....	46
Figure 3. 5: docker logo	47
Figure 3. 6: Nginx logo	48
Figure 3. 7: DigitalOcean logo	48
Figure 3. 8: DigitalOcean Droplets	49
Figure 3. 9: DegitalOcean Monitor Tools	49

Figure 3. 10: DigitalOcean Machine Prices.....	50
Figure 3. 11: Arch Linux logo	51
Figure 3. 12: StarUML logo.....	51
Figure 3. 13: Visual Studio Code logo	52
Figure 3. 14: git logo	52
Figure 3. 15: GitHub logo	52
Figure 3. 16: Github action logo	53
Figure 3. 17: Figma logo	53
Figure 3. 18: Postman logo.....	54
Figure 3. 19: Overleaf logo.....	54
Figure 3. 20: React logo.....	55
Figure 3. 21: TanStack logo	55
Figure 3. 22: three js logo.....	55
Figure 3. 23: Node js logo	56
Figure 3. 24: Nest Js logo	56
Figure 3. 25: Fastapi logo.....	57
Figure 3. 26: Python logo.....	57
Figure 3. 27: PostgreSQL and Neon logos.....	58
Figure 3. 28: Docling logo.....	62
Figure 3. 29: AI-powered document processing pipeline	63
Figure 3. 30: Whisper AI logo.....	64
Figure 3. 31: Token based authentication	64
Figure 3. 32: General solution architecture.....	66
Figure 3. 33: Multi-tenant model.....	67
Figure 4. 1: Deployment diagram.....	69
Figure 4. 2: Solution logo.....	70
Figure 4. 3: Home Page.....	72
Figure 4. 4: Contact Interface.....	73
Figure 4. 5: Footer.....	74
Figure 4. 6: Authentication page	74
Figure 4. 7: Sign up Interface.....	75

Figure 4. 8: Email format validation.....	76
Figure 4. 9: Credential Verification.....	76
Figure 4. 10: Super admin Dashboard.....	77
Figure 4. 11: The "Database Overview" interface.....	78
Figure 4. 12: User management.....	78
Figure 4. 13: "Add New Admin" button.....	79
Figure 4. 14: "Add new Admin" form.....	79
Figure 4. 15: Notification message.....	79
Figure 4. 16: Admin list updated.....	80
Figure 4. 17: Activate/desactivate button.....	80
Figure 4. 18: delete button	81
Figure 4. 19: Confirmation dialog for deleting a user	81
Figure 4. 20: Notification message.....	81
Figure 4. 21: School management interface.....	82
Figure 4. 22: School editing interface.....	83
Figure 4. 23: School website created automatically	84
Figure 4. 24: Profile dropdown menu.....	85
Figure 4. 25: Profile management interface.....	85
Figure 4. 26: change Password interface.....	86
Figure 4. 27: Headmaster Dashboard.....	86
Figure 4.28: Database Overview interface.....	87
Figure 4. 29: Database Overview Menu	87
Figure 4. 30: User management submenu.....	88
Figure 4. 31: Students management interface	88
Figure 4. 32: Teachers management interface.....	89
Figure 4. 33: Establishment management submenu.....	90
Figure 4. 34: Grade management interface.....	90
Figure 4. 35: Classes management interface.....	91
Figure 4. 36: Classrooms management interface.....	91
Figure 4. 37: Subjects management interface.....	91
Figure 4. 38: Timetable menu	92

Figure 4. 39: Timetable General Settings.....	93
Figure 4. 40: Period Timing configuration.....	94
Figure 4. 41: Subjects / courses configuration.....	94
Figure 4. 42: Teacher configuration	95
Figure 4. 43: Classes Configuration.....	95
Figure 4. 44: Classrooms/Salles Configuration.....	96
Figure 4. 45: Lessons configuration.....	96
Figure 4. 46: Alternating subject example	97
Figure 4. 47: Configure Lessons example.....	98
Figure 4. 48: "Generate Timetable" interface.....	98
Figure 4. 49: "Generate Timetable" button.....	98
Figure 4. 50: Generating in progress.....	99
Figure 4. 51: Timetable generated	99
Figure 4. 52: Timetable updated.....	100
Figure 4. 53: Teacher Dashboard	100
Figure 4. 54: Teacher Timetable interface.....	101
Figure 4. 55: Quizz generator interface	102
Figure 4. 56: Whiteboard interface	102
Figure 4. 57: My Notes interface.....	102
Figure 4. 58: Focus Room interface.....	103
Figure 4. 59: Chat button	103
Figure 4. 60: Chat interface	104
Figure 4. 61: Student Dashboard	105
Figure 4. 62: All in One AI Learning interface.....	106
Figure 4. 63: AI chatbot.....	106
Figure 4. 64: Attendance interface	107
Figure 4. 65: Flashcards interface	107
Figure 4. 66: Chat interface.....	107
Figure 4. 67: Academic performance interface « Subject scores »	108
Figure 4. 68: Performance analysis	108
Figure 4. 69: Academic performance interface « Semester averages and comparison ».....	109

Figure 4. 70: Mobile Experience and Responsiveness.....	110
Figure 4. 71: Chat Mobile Interface.....	110
Figure 4. 72: Flashcard Mobile Interface.....	110
Figure 4. 73: Azure found.....	115

LIST OF TABLES

Table 1. 1: Comparative Table of Existing School Management Systems	9
Table 1. 2: Comparative Table of Timetable Management Tools	10
Table 4. 1: Test cases and their outcomes	111

ACRONYMS AND ABBREVIATION

Abbreviation	Designation
AI	Artificial Intelligence
API	Application Programming Interface
CDN	Content Delivery Network
CRUD	Create, Read, Update, Delete
CI/CD setup	Continuous Integration / Continuous Deployment setup
CDN server	Content Delivery Network server
CORS	Cross-Origin Resource Sharing
CTO	Chief Technology Officer
DB	Database
GIZ	Deutsche Gesellschaft für Internationale Zusammenarbeit (German Agency for International Cooperation)
GmbH	Gesellschaft mit beschränkter Haftung (société à responsabilité limitée en allemand)
IT	Information Technology
ISETT	Institut Supérieur des Études Technologiques de Tozeur (Higher Institute of Technological Studies of Tozeur)
JWT	JSON Web Token
MCP	Model Context Protocol
QA testing	Quality Assurance testing
SAAS	Software As A Service
SME	Small and Medium-sized Enterprise
SMS	School Management System
UI	User Interface
UML	Unified Modeling Language
WIP	Work-in-Progress

General Introduction

In the evolving landscape of digital education, schools are increasingly seeking intelligent, scalable, and user-friendly platforms to manage academic operations, enhance communication, and support personalized learning. This report presents the design, development, and deployment of **JeridSchool**, a comprehensive school management system powered by modern technologies including AI, semantic search, timetable optimization algorithms, and multi-tenancy architecture.

Developed as part of an academic final project in collaboration with *Code Cooperation GmbH*, this platform addresses real-world challenges faced by educational institutions: fragmented communication, inefficient scheduling, and lack of data-driven insights. JeridSchool provides tailored dashboards for each actor, headmasters, teachers, students, parents, and administrators, offering tools such as attendance tracking, quiz generation, collaborative whiteboards, AI-based study assistance, and more.

By integrating cutting-edge features like AI-powered document processing and responsive web design in a multi-tenant system, JeridSchool aims to deliver a unified, smart, and adaptive digital school experience. This report outlines the context, design methodologies, implementation details, and validation of the solution, along with the challenges and learnings acquired throughout the project lifecycle.

Leveraging modern technologies such as React, Node.js, FastAPI, Docker, GitHub Actions, and AI services like OpenAI Whisper and Docling, JeridSchool provides role-based dashboards, automatic timetable generation, and a suite of tools tailored for teachers, students, headmasters, and parents. This document details the technical architecture, development methodology, sprint-based implementation, and validation of the platform, highlighting its potential as both a learning tool and a real-world software product.

This report documents the entire development cycle of our project, from the initial problem analysis to the implementation and validation phases. It outlines the context of the project, the

technological choices made, and the agile methodology followed throughout. The aim of this report is to provide a structured and comprehensive view of how the project evolved across its different stages, highlighting both the challenges encountered and the solutions provided.

This report describes the work carried out. For clarity, it is divided into four chapters organized as follows:

The first chapter, entitled "Project overview," is devoted, first, to an introduction to the host organization, the initial context and problems, as well as the project's objective.

The second chapter is devoted to analyzing the requested work and understanding the functional and non-functional requirements necessary to identify expectations and describe our system. The system architecture and detailed UML design, both at the data and interface levels, will be presented in this chapter.

The third chapter concerns the development methodology adopted and planning of the project. It focuses on the software environment and the technology selection.

The final chapter is devoted to the project's implementation. It also describes the implementation phase by detailing the application development process. The last part of this chapter is devoted to the validation tests performed. We close our report with a general conclusion, summarizing our main contributions and providing an overview of the perspectives to be considered.

Chapter 1: Project Overview

Introduction

This opening chapter introduces the context in which the project was conceived and developed. It begins with a presentation of the host organization, Code Cooperation GmbH, including its areas of expertise and collaborative environment. The chapter then explores the problem that the project aims to address, the main objectives, and the challenges faced. A study of existing solutions is conducted to better understand market needs and identify opportunities for improvement. Finally, we present our proposed solution and the planning of the project through its major phases.

1. Host Organization: *Code Cooperation GmbH*

In this section, we will talk about the hosting company where the project took place, we will start by describing the company and its fields of expertise.

1.1. Presentation

Code Cooperation GmbH is a software development company headquartered in Berlin, Germany, with additional operations in Tunis, Tunisia. Established in 2021, the company specializes in designing, building, and managing software solutions. Unlike traditional outsourcing agencies, Code Cooperation emphasizes long-term partnerships, aiming to act as a scalable CTO and development support for startups and companies at various stages.

Code Cooperation GmbH is a dynamic IT and software development company specializing in delivering innovative digital solutions. They focus on custom software development, web and mobile applications, and IT consulting, catering to a diverse range of industries. With a strong emphasis on agile methodologies, they ensure efficient project management and high-quality deliverables [1].



Figure 1. 1: Code Cooperation logo

The company is known for its expertise in modern technologies such as cloud computing, AI, and blockchain. Their team comprises experienced professionals with a proven track record in solving complex technical challenges. Code Cooperation GmbH also prioritizes client collaboration, tailoring solutions to meet specific business needs. Their portfolio includes successful projects for startups, SMEs, and large enterprises, showcasing their versatility and technical proficiency. With a commitment to innovation and customer satisfaction, they have established themselves as a reliable partner in the tech industry [1].

Website: www.codecooperation.com

Email: contact@codecooperation.com

1.2. Areas of Expertise

CodeCooperation offers a range of services tailored to meet the digital needs of businesses:

- **Digital Innovation Consulting:** Guiding companies through digital transformation processes.
- **Website and Mobile App Development:** Creating responsive and user-friendly web and mobile applications.
- **Software Product Development:** Building scalable and robust software products from conception to deployment.
- **Quality Assurance Services:** Ensuring the reliability and performance of software through rigorous testing.
- **Dedicated Development Teams:** Providing specialized teams to support clients' ongoing development needs.
- **General Tech Consulting:** Offering expert advice to optimize technological strategies.

1.3. Team and Culture

The team at CodeCooperation comprises entrepreneurial engineers who value transparency, collaboration, and continuous improvement. They embrace challenges and prioritize teamwork, believing that shared efforts lead to meaningful outcomes. The company's culture encourages open communication and a commitment to delivering tailored digital solutions that align with clients' strategic goals.

1.4. Clients and Collaborations

CodeCooperation serves a diverse clientele, including startups, scale-ups, and established companies. Notably, the company has partnered with GIZ Tunisia to strengthen the Tunisian IT ecosystem, providing sustainable and scalable development teams and CTO support at reasonable prices. This collaboration underscores CodeCooperation's commitment to fostering digital growth in emerging markets.

2. General context

The field of Computer Science has experienced a remarkable evolution, continually pushing the boundaries of what is possible in software development. At the same time, education remains a fundamental pillar of societal progress, and the efficient management of educational institutions plays a key role in delivering high-quality learning experiences. In recent years, the integration of technology into school administration has become increasingly essential, allowing institutions to streamline operations and enhance overall efficiency.

This graduation project explores the evolving landscape of school management system development and its impact on the education sector. It aims to identify innovative approaches for designing versatile and user-friendly platforms that go beyond traditional methods of school administration and learning management.

The project was undertaken as part of the requirements for obtaining a Bachelor's Degree in Computer Technologies at the Higher Institute of Technological Studies of Tozeur (ISET Tozeur). Titled "*School Management System*", the project was carried out at the company *Code Cooperation GmbH* between February 20th, 2023, and May 30th, 2023. It was supervised by Mr. Mohamed FOURATI, Software Engineer at Code Cooperation GmbH, and Mrs. Haifa DEGACHI, Computer Science Lecturer at ISET Tozeur.

2.1. Problem Statement

In many educational institutions, administrative tasks such as student registration, attendance tracking, grade management, timetable scheduling, and communication between staff, students, and parents are still performed manually or using outdated software systems. These traditional methods are time-consuming, error-prone, and inefficient, often leading to data inconsistency, communication gaps, and operational delays.

Timetable scheduling is a critical yet complex task for educational institutions. Schools must allocate teachers, students, and classrooms efficiently while adhering to multiple constraints such as subject availability, teacher workload, and room capacity. Manually creating a timetable can be extremely time-consuming and prone to human error, often resulting in scheduling conflicts and disruptions to the academic process.

As schools grow and the demand for digital solutions increases, there is a critical need for a centralized, automated, and user-friendly system that can manage both academic and administrative activities effectively. The lack of such an integrated platform hinders decision-making, reduces productivity, and negatively impacts the overall learning environment.

This project addresses the challenge of designing and implementing a School Management System that meets the specific needs of modern educational institutions. The goal is to provide a scalable and reliable solution that streamlines daily operations, optimizes timetable scheduling, improves communication, ensures data integrity, and enhances the overall educational experience for all stakeholders.

2.2. Project's purpose

As part of the end-of-studies project and in order to meet the needs of educational institutions in terms of administrative and academic efficiency, this project aims to provide a complete School Management System that helps schools manage their daily operations more effectively. The system is designed to reduce the workload of administrative staff, minimize human errors, and improve communication between all stakeholders including teachers, students, and parents.

Our project is composed of several key components: a student management module, an attendance tracking system, a timetable scheduler, and a communication interface. The main objective is to centralize and automate essential school functions such as student enrollment, attendance recording, grade reporting, and timetable generation.

More specifically, school administrators can use the platform to register students and staff, track their progress and presence, generate conflict-free schedules automatically, and facilitate direct communication through the system. This contributes to a more organized educational environment, where information is accessible, accurate, and easy to manage.

2.3. Project's challenges

Throughout the development of the School Management System, several challenges were encountered at both technical and functional levels. One of the most complex aspects was designing a flexible and conflict-free timetable scheduling system capable of accommodating a wide range of constraints, including teacher availability, classroom capacity, subject distribution, and school-specific policies. Timetable scheduling presented the following specific challenges:

- **Resource Allocation:** Schools must assign teachers to classes, students to subjects, and classrooms to time slots without overlaps or conflicts.
- **Multiple Constraints:** A teacher cannot be in more than one class at a time; students require a balanced schedule to avoid excessive workload on any given day; and classrooms must be used efficiently within their capacity.
- **Dynamic Changes:** Unforeseen events such as teacher absences or special events like exams require real-time schedule adjustments, which can lead to cascading issues if not managed properly.
- **Fair Workload Distribution:** Teachers should have a balanced number of classes throughout the week, and students should not have several intensive subjects scheduled consecutively.
- **Policy Compliance:** Some schools have specific rules, such as preventing teachers from teaching in different buildings back-to-back or requiring certain subjects to take place in specialized rooms like labs or computer centers.

Beyond scheduling, other major challenges included ensuring data consistency and real-time synchronization between different modules such as student records, attendance, and grading. Developing an intuitive and user-friendly interface was also critical, as the system is intended for use by a wide range of users with varying levels of technical expertise, including administrators, teachers, students...

Finally, technical challenges such as integrating diverse technologies, securing sensitive student data through proper authentication and access control, and conducting thorough testing to ensure system reliability and performance demanded careful planning, iteration, and time management throughout the project lifecycle.

3. Study of the existing

Before developing a new solution for school management and timetable scheduling, it is essential to analyze the existing systems currently available. This study provides an overview of the tools and software widely used by educational institutions to manage administrative tasks and organize schedules. By examining their features, strengths, and limitations, we aim to identify gaps and opportunities for improvement. Understanding the current landscape will help ensure that the proposed system effectively addresses the specific needs of schools, especially considering local contexts.

3.1. Analysis of Existing Systems

In these section, we examine the features, functionalities, and technologies behind popular school management solutions and timetable tools. This analysis helps identify how these systems address the needs of educational institutions.

3.1.1. *Traditional (Classical) Method*

Historically, the management of schools relied heavily on manual processes and paper-based methods. Administrative tasks such as student registration, attendance tracking, grade recording, and timetable scheduling were performed using physical files, notebooks, and spreadsheets. This classical approach was time-consuming, prone to human error, and inefficient, especially in large institutions with many students and staff.

In Tunisia, the majority of both public and private schools still use these classical manual methods for their daily management. The reliance on paperwork and manual scheduling limits efficiency, increases the risk of errors, and complicates communication among teachers, students, parents, and administration staff. Manual timetable creation requires extensive coordination to avoid conflicts, and last-minute changes can disrupt the entire schedule. Moreover, communication between school staff, students, and parents is often limited to face-to-face meetings, phone calls, or printed notices, leading to delays and miscommunication. This traditional method lacks efficiency and scalability, highlighting the need for automated, digital solutions.

3.1.2. *Popular School Management Systems*

In response to the limitations of manual methods, various digital School Management Systems (SMS) have emerged globally. These systems automate a wide range of administrative tasks including student data management, attendance tracking, grade recording, fee collection, and communication.

Several well-established school management software solutions have gained prominence worldwide. These platforms typically offer modules for student information management, attendance tracking, grade reporting, timetable scheduling, and communication tools. These popular international platforms such as **PowerSchool**, **Blackbaud**, **Fedena**, and **ClassDojo** offer comprehensive solutions, often cloud-based, providing real-time access and multi-user interfaces. These systems improve data accuracy, reduce paperwork, and enhance operational efficiency. However, many existing SMS solutions come with high costs and limited customization, making them less accessible or adaptable to the specific requirements of schools in developing regions. Language localization and adherence to local educational policies can also be lacking.

In this section, we will present a comparative Table existing school management systems.

Table 1. 1: Comparative Table of Existing School Management Systems

System Feature	PowerSchool 	Blackbaud 	Fedena 	ClassDojo 	Manual / Paper-Based
Deployment Type	Cloud-based	Cloud-based	Cloud-based & On-premise	Cloud-based	None
Student Information Management	Comprehensive (personal data, academic records, health info)	Comprehensive	Comprehensive	Basic (focus on behavior and communication)	None (manual records)
Attendance Tracking	Real-time, automated alerts	Real-time, integrated	Real-time	Basic manual input	Manual
Grade Reporting	Detailed, customizable reports	Detailed, customizable	Customizable reports	Limited	Manual
Fee Management	Automated billing and payment processing	Integrated billing and payments	Basic fee management	No	Manual
Communication Tools	Messaging, notifications, portals	Messaging, notifications	Messaging, SMS/email alerts	Parent-teacher messaging	Limited (phone, printed notes)
Customization Level	Moderate customization options	Highly customizable	Highly customizable	Low customization	None
Cost	High (licensing fees)	High	Moderate	Low (mostly free)	None
User-Friendliness	Moderate (requires training)	Moderate	Moderate	High (designed for non-tech users)	Low (time-consuming)

Local Language Support	Limited (mostly English)	Limited	Moderate (supports multiple languages)	Limited	Full (native language)
Integration with Timetable Systems	API available, some integration	Partial integration	Good integration	No integration	None
Scalability	Very high (district/state-wide use)	High	Moderate	Low	None

3.1.3. Existing Timetable Management Tools

Timetable scheduling is a specialized function within school management that presents its own set of challenges. It presents a complex task that involves balancing multiple constraints, such as teacher availability, classroom capacity, and subject requirements. Various timetable management tools exist, ranging from standalone software like **FET** and **ASC Timetables** to modules integrated within broader SMS platforms. These tools aim to automate the assignment of teachers, students, and classrooms to time slots while respecting constraints. However, many of these systems rely on simplistic algorithms or manual adjustments, limiting their effectiveness in handling complex, dynamic scenarios like last-minute teacher absences or exam rescheduling.

Recently, some solutions have begun to incorporate **Artificial Intelligence (AI)** and optimization algorithms to improve scheduling efficiency. AI-based systems can dynamically adjust timetables in response to unexpected changes like teacher absences or exam rescheduling, optimize resource allocation, and minimize conflicts.

Despite these advancements, AI-based timetable solutions are still not widely adopted in many regions due to high development costs, complexity, and lack of local adaptation. In many schools, particularly in Tunisia, timetable management remains largely manual or semi-automated using basic tools such as spreadsheets.

In this section, we will present Comparative Table of Timetable Management Tools.

Table 1. 2: Comparative Table of Timetable Management Tools

Tool Feature	FET (Free Timetabling)	Timefold	ASC Timetables	AI-Based Timetable Solutions
Automation Level	Full automatic timetable generation	Fully automated, AI-driven optimization	Semi-automatic (user input needed)	Fully automated with optimization and AI
Constraint Handling	Handles multiple constraints (teacher	Handles complex constraints	Handles basic constraints	Advanced multi-criteria optimization

	availability, room capacity, etc.)	including hard & soft constraints		
Handling Dynamic Changes	Limited, manual adjustments needed	Supports real-time adjustments and re-optimization	Limited, manual adjustments	Real-time dynamic rescheduling possible
User Interface	Basic, technical	Modern, intuitive web-based UI	User-friendly graphical UI	Varies; often requires technical expertise
Cost	Free	Enterprise pricing, scalable for institutions	Paid software	High (custom development/licensing)
Integration with School Management Systems	Limited (export/import formats)	API-based integration, flexible	Moderate (some integration features)	High integration with SMS and databases
Adaptability to Local Rules	Moderate (customizable constraints)	Highly customizable constraint modeling	Moderate	High (can learn and adapt from data)
Technical Skill Required	Moderate technical knowledge	Moderate; designed for admins with some training	Low to moderate	High (AI/ML knowledge often required)
Popularity in Tunisia	Low usage	Emerging, growing adoption worldwide	Low usage	Very low usage
Flexibility for Last-Minute Changes	Low (manual edits required)	Excellent; supports rapid re-scheduling and scenario testing	Low	High (dynamic re-optimization)

3.2. Critique of Existing Systems

Despite the advances in school management and timetable scheduling solutions, several significant limitations remain that impact their effectiveness and adoption:

Limitations of School Management Systems:

- **High Cost:** Many comprehensive systems require expensive licenses or subscriptions, making them inaccessible to schools with limited budgets, especially in developing countries.
- **Lack of Customization:** Off-the-shelf solutions often do not fully adapt to specific local educational policies, language needs, or unique institutional workflows.
- **Complexity and Usability:** Some platforms are too complex for non-technical users such as teachers or parents, leading to low adoption and underutilization of features.
- **Limited Integration:** Many systems do not seamlessly integrate with other tools, including timetable software, resulting in fragmented workflows and duplicated efforts.
- **Dependence on Internet Connectivity:** Cloud-based solutions require reliable internet access, which may be a challenge in certain regions.

Limitations of Timetable Management Tools:

- **Inflexibility:** Many tools struggle to handle last-minute changes such as teacher absences, room unavailability, or exam rescheduling without requiring manual intervention.
- **Insufficient Constraint Handling:** Basic timetable software cannot efficiently manage complex constraints (e.g., specialized classrooms, balanced teacher workloads) which are common in real-world scenarios.
- **Technical Expertise Required:** Advanced AI-based solutions often require specialized knowledge to configure and maintain, limiting their adoption in typical school environments.
- **High Implementation Costs:** Sophisticated timetable tools with AI optimization tend to have high development or licensing costs, which are prohibitive for many institutions.
- **Low Adoption in Local Contexts:** In many schools, especially in Tunisia, timetable management remains largely manual or semi-automated due to lack of awareness, training, or budget.

These limitations underline the need for an adaptable, and user-friendly school management and timetable solution tailored to the specific needs and constraints of local educational institutions.

4. Problematic

The analysis of existing systems reveals significant gaps in affordable, adaptable, and intelligent school management and timetable solutions, especially tailored for local contexts. Despite the growing importance of digital solutions in various sectors, many educational institutions particularly in Tunisia still rely on traditional methods for managing their daily operations. Timetable creation, for instance, often involves manual coordination efforts by administrators, who attempt to avoid conflicts by hand. This process is time-consuming, error-prone, and lacks the flexibility needed to accommodate last-minute changes or complex constraints.

These outdated methods often lead to delays, missed information, and ineffective engagement. The absence of a centralized data system makes information retrieval difficult and increases the risk of data inconsistency or loss.

Existing school management systems on the market tend to be either too complex, expensive, or insufficiently tailored to local educational needs. Many lack integrated timetable automation features or user interfaces that accommodate varying levels of technical proficiency. Additionally, most existing tools do not fully address the need for intelligent scheduling, comprehensive data management, and multi-user access.

These limitations highlight an urgent need for a digital solution that is affordable, adaptable, and smart combining automated timetable generation, centralized school management, and intuitive interfaces. The goal is to enhance efficiency, minimize administrative burden, and support better communication and decision-making in educational institutions.

5. Our solution

To address the limitations of existing school management and scheduling systems, we propose a web-based and a mobile application for School Management System designed to be intuitive, modular, and scalable. Our solution, named JeridSchool, an advanced and automated timetable scheduling system designed to simplify school management.

JeridSchool solution aims to centralize the management of students, teachers, classes, and academic resources in a single, user-friendly platform. One of its core features is an intelligent timetable scheduling module that takes into account multiple constraints such as teacher availability, classroom capacity, and subject distribution, reducing manual workload and avoiding conflicts.

The system also includes functionalities for attendance tracking, grade management, and internal communication between different stakeholders, including parents. Built using modern technologies like React.js for the frontend and Spring Boot for the backend, our platform ensures performance, security, and adaptability to meet the real needs of educational institutions, especially those still relying on manual methods.

5.1. Key Features

In the following sentence, we will present the most significant and distinguishing elements that are often highlighted to emphasize the unique value proposition of this project.

- Easy to Use: JeridSchool offers an intuitive and user-friendly interface, making it accessible to school administrators with minimal training.
- Fast and Automated Scheduling: Unlike traditional tools that require manual intervention, JeridSchool automatically generates optimized timetables in a matter of minutes.
- Real-Time Updates: The system allows schools to make instant modifications to schedules without the need to regenerate the entire timetable.
- No Prior Experience Required: School staff can efficiently manage timetables without needing specialized knowledge, reducing dependency on experts.

- Adaptive and Smart: The software intelligently allocates resources while avoiding conflicts, ensuring optimal distribution of classrooms, teachers, and student groups.
- Seamless School Management: Beyond scheduling, JeridSchool integrates other essential school management features to improve overall administrative efficiency.

By combining ease of use, speed, and flexibility, JeridSchool provides an innovative solution that eliminates the hassle of timetable scheduling and enhances school operations.

5.2. Project Specifications Document

Project Title: Development of a School Management System

Project Supervisor:

- Mr. Mohamed FOURATI (Software Engineer, Code Cooperation GmbH)
- Mrs. Haifa DEGACHI (Computer Science Professor, ISET Tozeur)

Project Team: Youcef DEGACHI

Project Duration: 3 months (February 20th, 2023 – May 30th, 2023)

Deliverables:

- Functional Web-based School Management System
- Functional mobile-based app
- Project Report / User Guide
- Final Presentation

Required Technical Environment:

- Frontend: React
- Backend: Node Js
- Database: PostgreSQL
- Tools: git/github

Requirements and Constraints:

- The system must be web-based and responsive.
- The interface must be user-friendly and accessible to users with varying levels of technical knowledge.
- Data integrity and security must be ensured, especially for sensitive student and staff information.

- Timetable scheduling must consider complex constraints like teacher availability, classroom usage, and subject distribution.
- The solution should allow future scalability and potential integration with other educational platforms.

Project Objectives: The objective is to develop a complete, centralized school management platform that allows educational institutions to:

- Manage student records, attendance, grades, and class enrollment efficiently.
- Facilitate communication between administrators, teachers, students, and parents.
- Automate timetable generation based on custom constraints.
- Improve decision-making by providing real-time data and reports.

Monitoring Procedures:

The project team must attend weekly progress meetings with the project supervisor.

A brief progress report is required after each milestone or phase completion.

5.3. Project Phases

Phase 1: Project Understanding

- Study of existing school management systems
- Bibliographic research and benchmarking

Phase 2: Needs Analysis

- Gather functional and technical requirements
- Define user roles and workflows
- Draft detailed specifications document

Phase 3: Design and Architecture

- Define system architecture (client-server)
- Design database schema and system modules
- Create mockups and UI wireframes

Phase 4: Development

- Backend and frontend implementation
- Implementation of timetable scheduling logic
- Integration of modules (attendance, grades, user management, etc.)

Phase 5: Testing and Validation

- Perform unit and integration testing
- Functional and user acceptance testing
- Debug and validate performance, security, and usability

Phase 6: Documentation and Presentation

- Write final report and user manual
- Prepare and deliver final project presentation

5.4. GANTT Diagram

Our project will start on February, 20th 2025 and will continue until the end of May. Throughout this period, we have chosen to break down our work according to the Gantt chart below, which clearly describes our estimate of the progress of the project.

Phase	durée											
	Février		Mars				Avril				Mai	
Semaine	1	2	3	4	5	6	7	8	9	10	11	12
Phase 1												
Phase 2												
Phase 3												
Phase 4												
Phase 5												
Phase 6												

Conclusion

In this chapter, we have outlined the overall context of the project by highlighting the limitations of traditional school management methods and analyzing the existing solutions available on the market. Our investigation revealed that many institutions, particularly in Tunisia, still rely on manual processes that are inefficient, error-prone, and poorly adapted to modern educational needs. We also identified significant gaps in current school management and timetable scheduling tools, especially in terms of affordability, adaptability, and intelligent automation.

Chapter 2: Functional and Conceptual Specification

Introduction

In this chapter, we delve into the functional and non-functional requirements of the system, based on a detailed analysis of user needs. This serves as the foundation for the technical and conceptual modeling of the system. Various diagrams are used to represent and formalize the structure and behavior of the application, including use case diagrams, sequence diagrams, activity diagrams, class diagrams, and more. These models help define the system architecture and ensure that development will be aligned with user expectations and technical constraints.

1. Analysis of needs

Requirements analysis is a critical and preliminary phase in any project, as it allows understanding the actual needs of the project, the objectives to be achieved, and the constraints to be considered in order to create a relevant and effective project that meets user expectations. This chapter therefore focuses on clearly identifying our functional, non-functional, and technological requirements.

1.1. Functional requirements

The functional requirements represent the specific features and services that our system must provide to meet the needs of its users. In other words, they define the tasks, operations, and functionalities that our **School Management System** must perform to ensure effective administration and learning experiences. In our case, the following list outlines these essential requirements:

- User Management
 - Create, update, and delete user accounts (admins, teachers, students, parents).
 - Assign roles and permissions based on user profiles.
 - Allow secure login and logout with role-based access control.
- Class and Subject Management
 - Create and manage classes and sections.
 - Assign subjects to each class.
 - Link teachers to subjects and classes.
- Timetable Management

- Automatically generate conflict-free timetables based on defined constraints (teacher availability, room capacity, subject distribution).
- Allow manual adjustments by administrators.
- Display personalized schedules for students and teachers.
- Attendance Tracking
 - Teachers can mark daily attendance.
 - Students and parents can view attendance history.
 - System can generate attendance reports.
- Grade and Evaluation Management
 - Teachers can input grades per subject and student.
 - Generate report cards and academic transcripts.
 - Allow parents and students to view academic results.
- Communication and Notifications
 - Send announcements and messages to specific user groups (e.g., parents, students).
 - Allow messaging between teachers and parents.
 - Notify users of schedule changes, events, or new grades.
- Document and Content Sharing
 - Teachers can upload learning materials, assignments, and resources.
 - Students can download and view shared documents.
- Dashboard and Reporting
 - Provide dashboards for admins and headmasters to monitor school operations.
 - Generate statistics and performance reports (attendance, grades, scheduling efficiency).
- Mobile Access
 - Offer mobile-responsive design or mobile app access.
 - Allow viewing schedules, grades, messages, and notifications on mobile devices.

1.2. Non-functional Requirements

Non-functional requirements are aspects that describe the quality and behavior of a system, rather than the functions it must perform. They are essential to ensure a system that is reliable, efficient, and user-friendly, even though they are not directly related to the core functionalities.

Security:

- The system must ensure secure authentication and role-based access control.

- All sensitive data (e.g., student grades, personal information) must be encrypted and protected from unauthorized access.
- Regular backups should be implemented to prevent data loss.

Performance

- The system should handle multiple users simultaneously without performance degradation.
- Timetable generation should respond in a reasonable time frame even with a high number of constraints.
- Pages and data must load within 2–3 seconds under normal conditions.

Responsiveness & Accessibility

- The system must be accessible from various devices (desktop, tablet, smartphone).
- User interfaces should be responsive and compatible with all modern browsers.
- The mobile version or app should provide access to core features like timetables, grades, and messages.

Usability

- The user interface must be intuitive and easy to use for all user profiles (admin, teacher, student, parent).
- Minimal training should be needed for users to interact with the system.
- Clear instructions, labels, and feedback messages should be provided.

Maintainability

- The system should be modular and easily maintainable for future updates or changes.
- Clean code structure and documentation should be ensured for further development or debugging.

Scalability

- The system should be scalable to support more users, classes, and schools if required in the future.
- The database and architecture must be designed to accommodate growth.

Availability

- The system should have high availability during school hours and beyond (24/7 recommended for parents/students access).
- Downtime must be minimized, especially during grading or scheduling periods.

Testability

- The system should be testable with unit, integration, and functional tests.
- Logs and errors must be traceable for debugging purposes.

2. Specification

In order to effectively build our application, during this section we will identify the needs that our application must fulfill. First, we will describe the functional and the non-functional needs of our system, followed by a presentation of the product backlog, and we end with identifying the different actors.

2.1. Actors identification

An actor is an entity with which our application must interact. In other words, the users who will need a help from the application to fulfill their tasks and run its functions. Accordingly, it can be a person, a system, or even a program. Therefore, we use this section to present our actor.

- **Super Admin (owner):** is the highest-level user with full control over the entire system. This actor is responsible for system initialization, global configurations, and managing multiple schools (if multi-tenant). He can create or deactivate administrator accounts and oversee all user roles.
- **Administrator (Headmaster/School Director):** the headmaster oversees the educational and administrative functioning of the school. He validates reports, monitors staff performance, and reviews academic progress. The administrator manages the day-to-day operation of the system. The admin can create user accounts for students, teachers, and parents. He handles timetable settings, subjects, classrooms, and manages school data.
- **Teacher:** The teacher manages assigned classes and subjects, marks attendance, enters grades, and uploads course materials and communicates with students and parents through the system.

- **Student:** The student can view personal timetable, grades, attendance, and assignments. He also accesses learning materials and receives announcements and can communicate with teachers and participate in classroom discussions.
- **Independent Student:** for users who sign up themselves and are not yet linked to any school.
- **Parent:** The parent of each student have the possibility to track their child's academic performance, attendance, and timetable. They receives updates, announcements, and messages from teachers and can contact teachers or administrators for follow-up.
- **Guest:** this is the simple user of our web application. He can contact support or sign up as a non-assigned student.

2.2. Use Cases Identification

Use case identification is a crucial step in any project, as it helps the development team gain a comprehensive understanding of user requirements and enables effective planning and implementation.

In this project, we have this Use cases as the functional objectives and associated use cases for the user:

Super Admin

- Authenticate (Sign in)
- Manage Profile (edit profile, edit password)
- List Administrators (add, edit, deactivate, delete)
- Manage Schools (add, edit, delete, list, view)
 - o Generate website and configure Global Settings

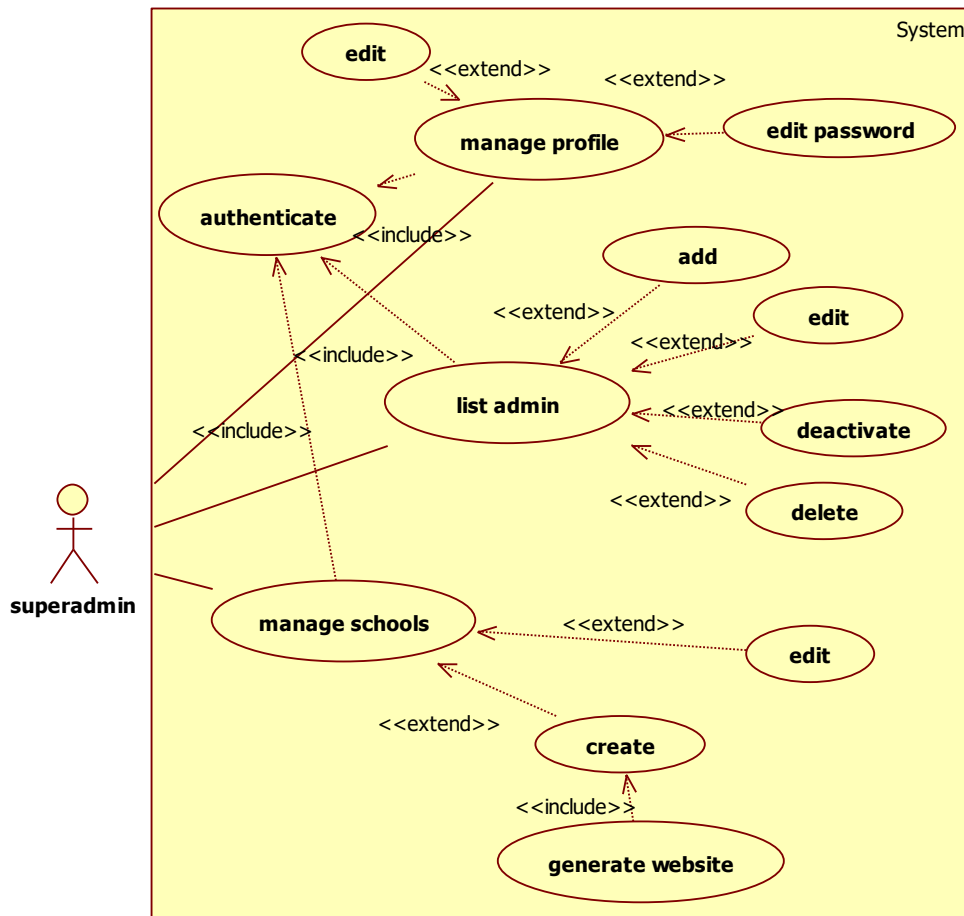


Figure 2. 1: Uses case diagram "superadmin"

Headmaster (School Admin)

- Authenticate (Sign in)
- Manage Profile (edit profile, edit password)
- Manage Users
 - o List Students (add, edit, delete, list, view)
 - o List Teachers (add, edit, delete, list, view)
 - o List Parents (add, edit, delete, list, view)
- Manage Classes (add, edit, delete, list, view)
- Manage Levels (add, edit, delete, list, view)
- Manage Subjects/Courses (add, edit, delete, list, view)
- Generate Timetable
 - o Adjust Timetable
- View Attendance

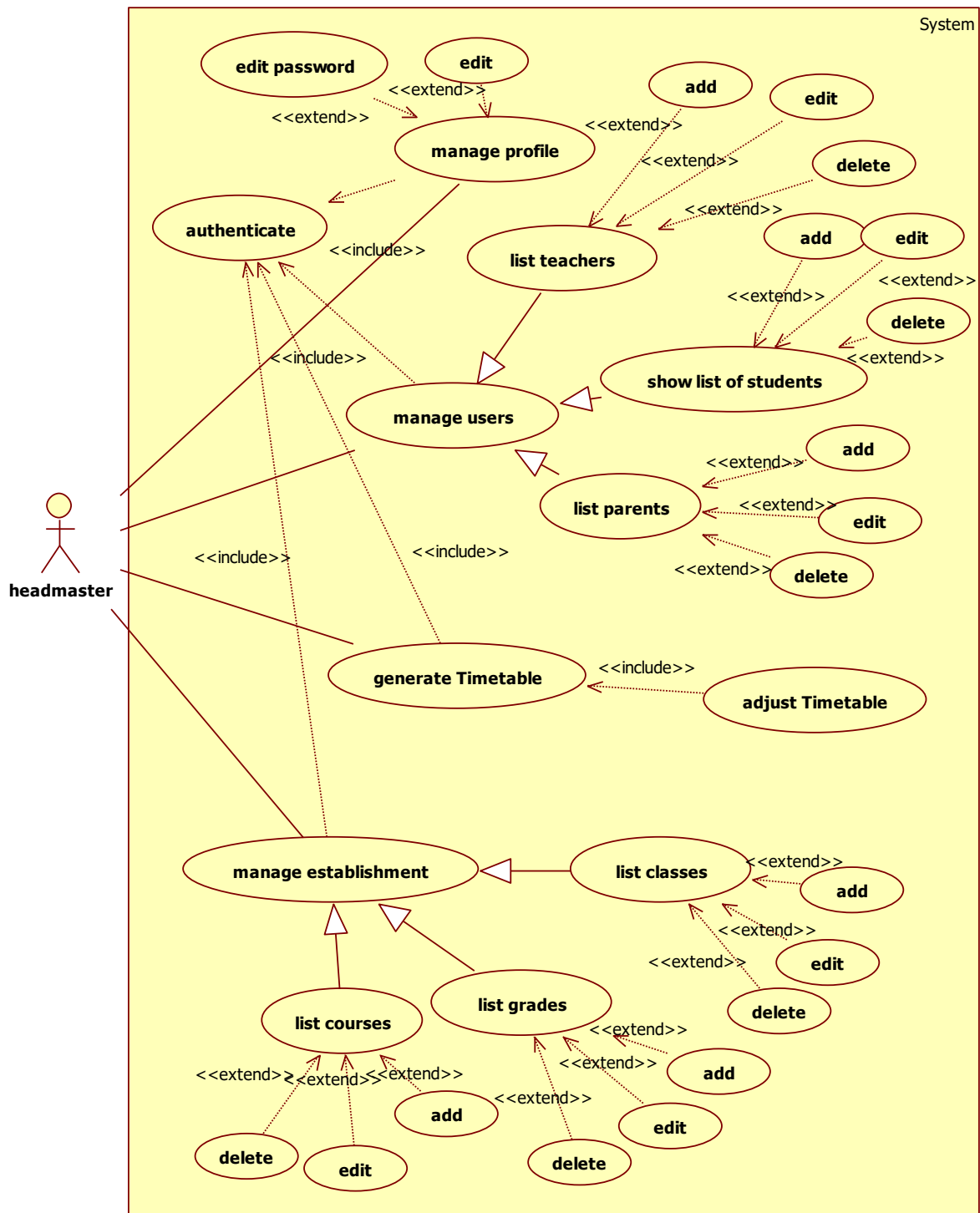


Figure 2. 2: Use Cases diagram "headmaster"

Teacher

- Authenticate (Sign in)
- Manage Profile (edit profile, edit password)
- View Personal Timetable
- List students
 - o Mark Absences
- Consult Attendance
- Manage and generate Quizzes (add, edit, delete, list, view)
- Use Whiteboard
- Communicate with Students and Parents (Chat, Messages)
- Manage Notes (add, edit, delete, list, view)
- Manage Focus Room (Study environment)
 - o Add activity
 - o List activities

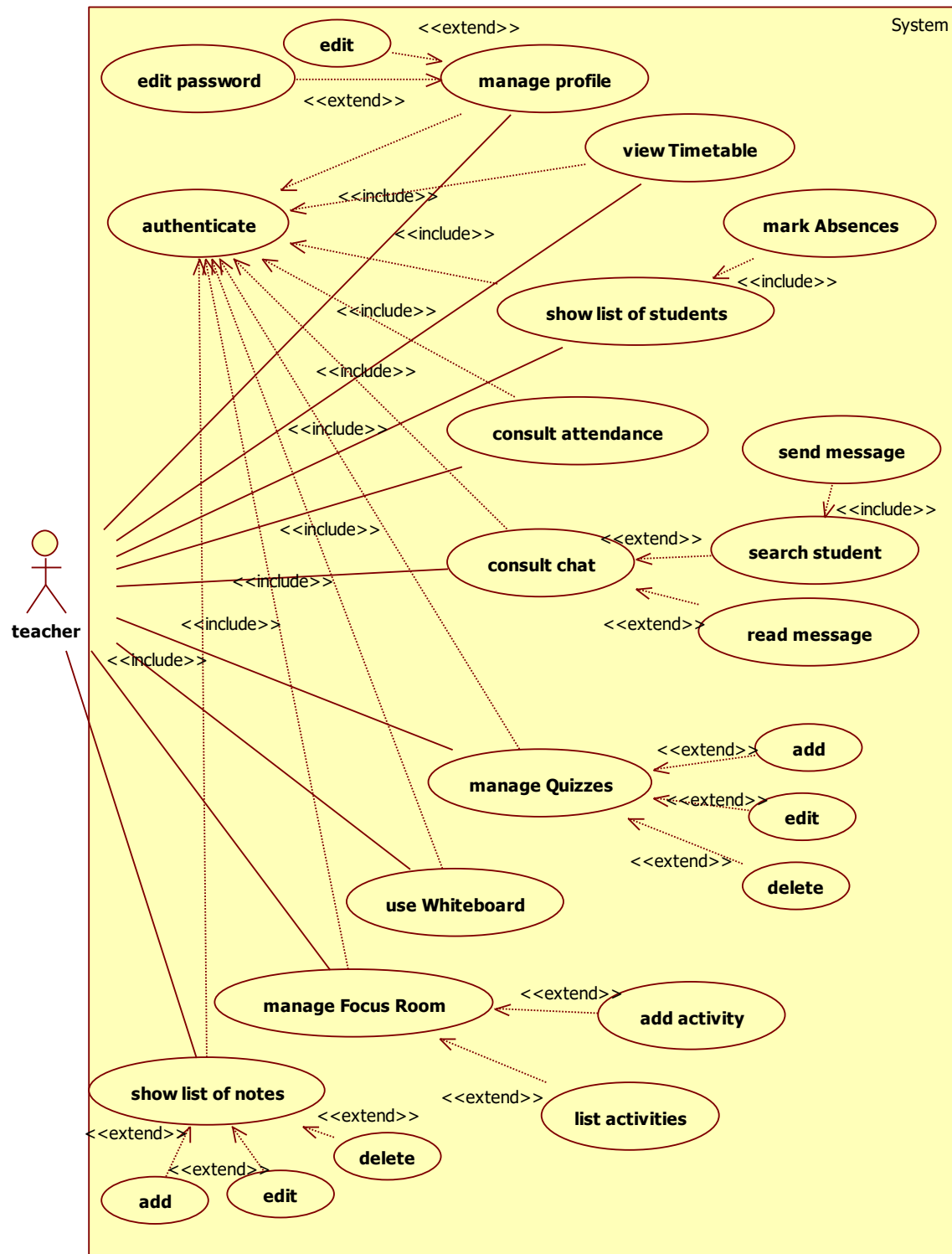


Figure 2. 3: Use Cases diagram "teacher"

Student

- Authenticate (Sign in)
- Manage Profile (edit profile, edit password)
- View Personal Timetable and Schedule
- Access Attendance
- Participate in Quizzes
- Use Flashcards for Learning (add, edit, delete, list, view)
- Use Whiteboard for Collaboration
- Communicate with Teachers (Chat)
- Manage Notes (add, edit, delete, list, view)
- Access Focus Room (Study environment)
- View academic performance
 - o View exam grades
 - o View semester/subjects averages
 - o Consult result analysis
- Use AI learning interface
 - o Upload file/add Youtube link
 - o Generate study cards

Students who register themselves without school assignment have limited access to the platform. They can manage their profile, authenticate, and use basic learning and communication tools, but they cannot access class-specific features, detailed academic performance data, or AI-powered personalized learning tied to a school. Interact with AI Chatbot

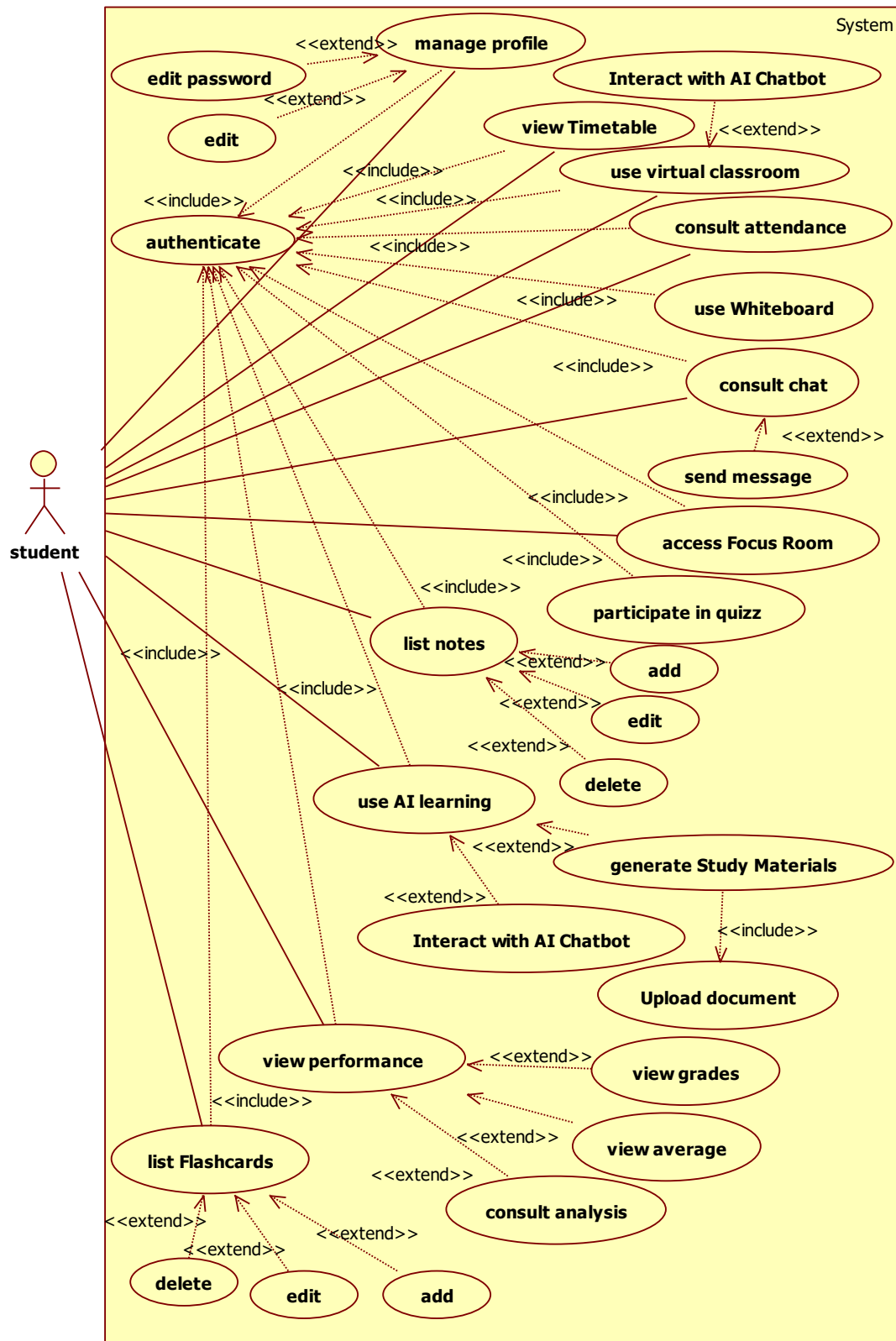


Figure 2. 4: Use Cases diagram "student"

Parents

- Authenticate (Sign in)
- Manage Profile (edit profile, edit password)
- View Child's Timetable
- Access Attendance
- Communicate with Teachers and School Staff

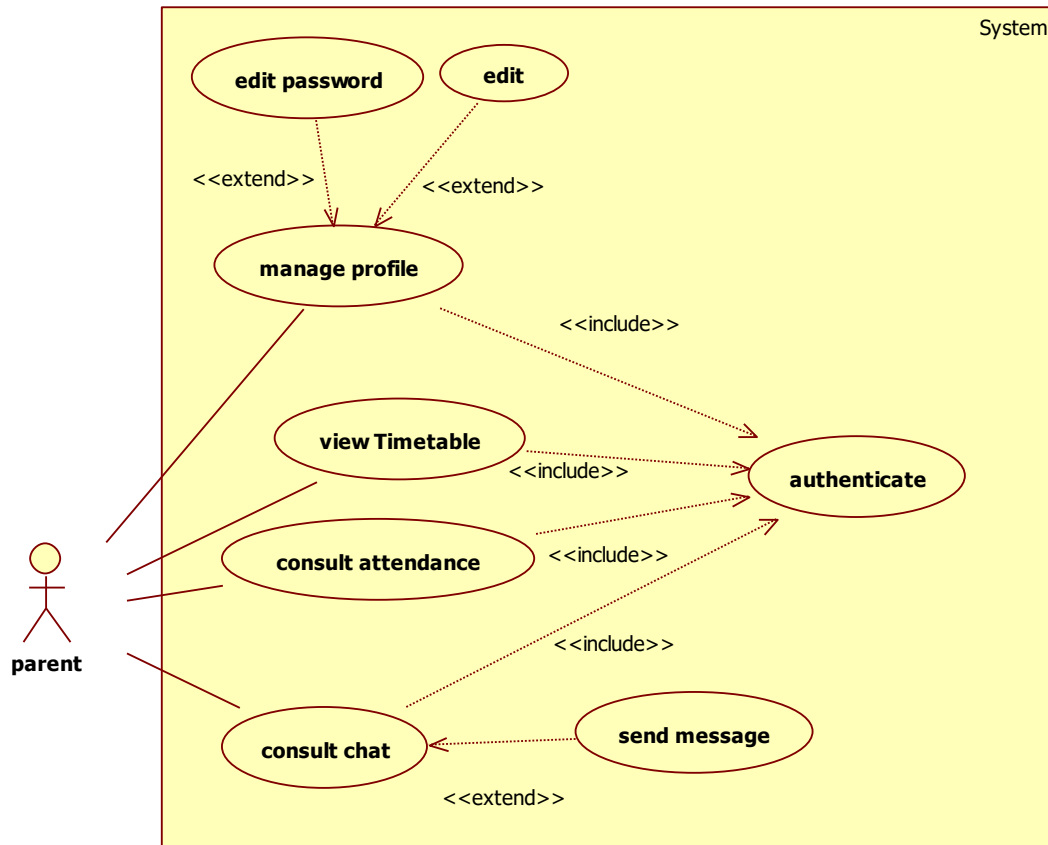


Figure 2. 5: Use Cases diagram "parent"

As for the simple user or the browser who use the platform, he can sign up to become a non-assigned student or he can contact admin.

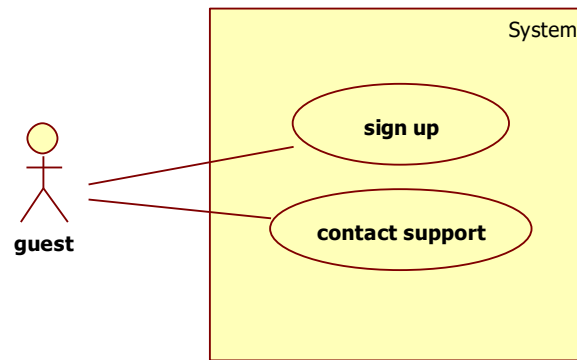


Figure 2. 6: Use Cases diagram "guest"

3. Sequences diagrams

In this section, we will present, for each use case, the different interactions between the actors and the system in a chronological order using sequence diagrams. These diagrams will visually illustrate the flow of actions and messages exchanged between the actors and the system, providing a clear understanding of the dynamic behavior and the sequence of steps involved in each use case. By presenting these sequence diagrams, we aim to enhance the comprehension of the system's functionality and how it fulfills the requirements of the identified use cases.

3.1. Sequence diagram of the use case "Authenticate"

Authentication allows a user to access their account on the platform. If they are already registered, they must enter their email address and password. The system performs a local check to detect the validity of the entries (empty or incorrect fields) and informs the user if an error occurs. A database check is then run to verify the validity of the data provided. If the login and password exist, they can access the platform interface and are authorized to perform the application tasks. Otherwise, the system displays an error message for invalid email and/or password, prompting them to re-enter them.

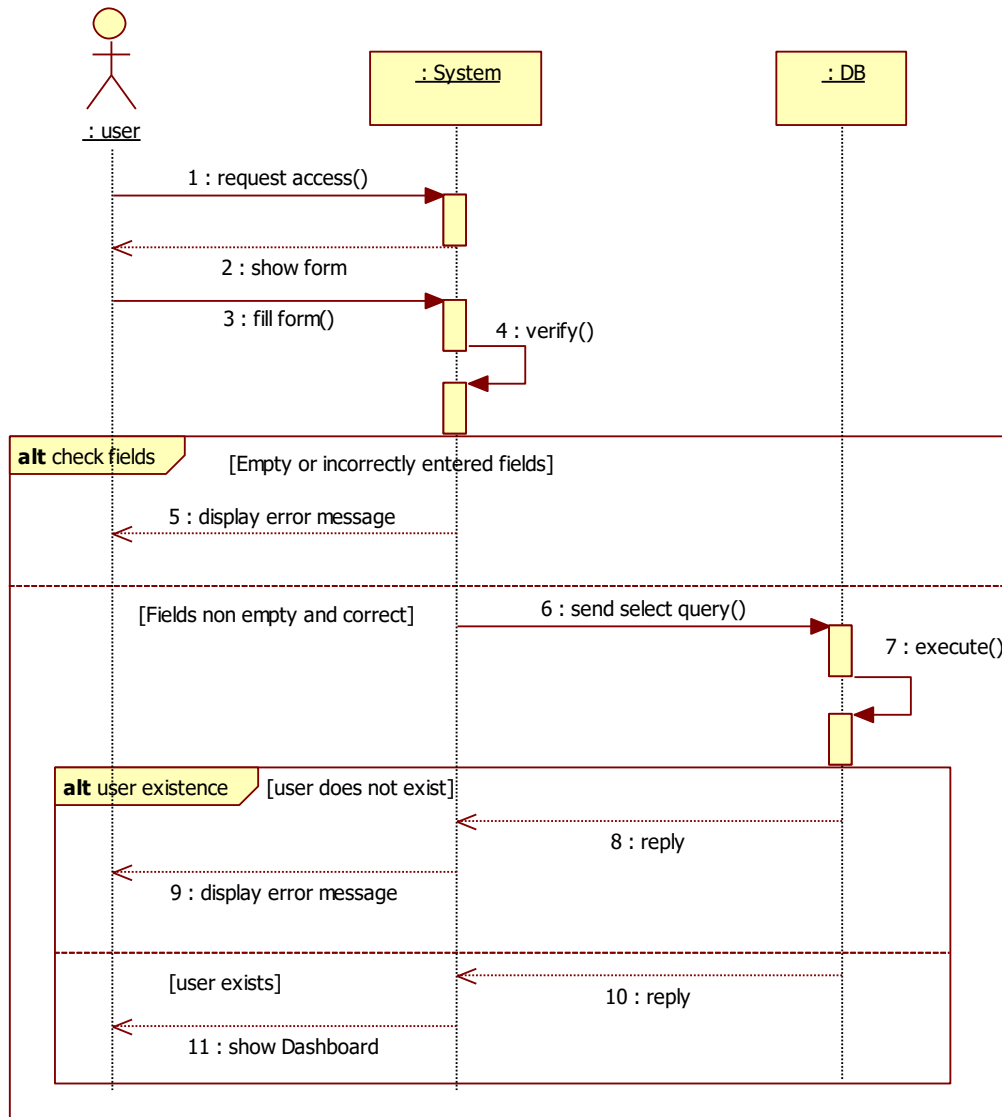


Figure 2. 7: Sequence diagram of the use case "authenticate"

3.2. Sequence diagram of the use case "display list"

To display the list of students, the user requests the list. The system sends a selection query to the database. After executing the query, the system then displays the list of students.

This process follows the same logic for listing other entities such as administrators, parents, classes, classrooms; grades...

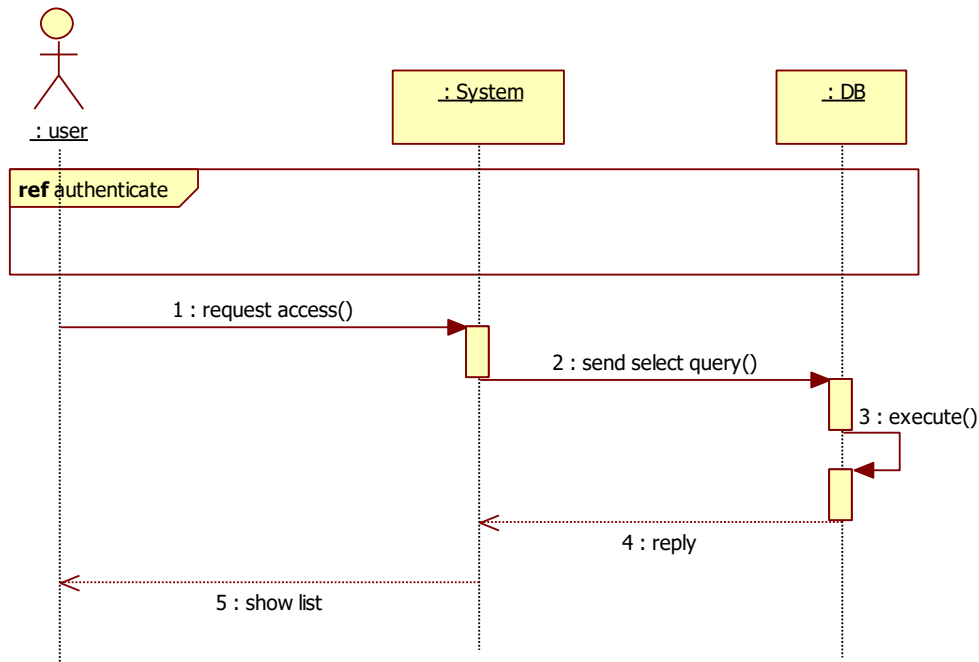


Figure 2. 8: Sequence diagram of the use case "display list"

3.3. Sequence diagram of the use case "add subject"

After displaying the list, to add a new subject, the administrator must complete a form displayed by the system. The system checks the validity of the fields and then sends an insert query to the database. The database, in turn, checks the validity of the data and executes the query, inserting the new subject into the "Subjects" table and sending a response to the system, which in turn displays an insert success message to the user, along with the new list of subjects.

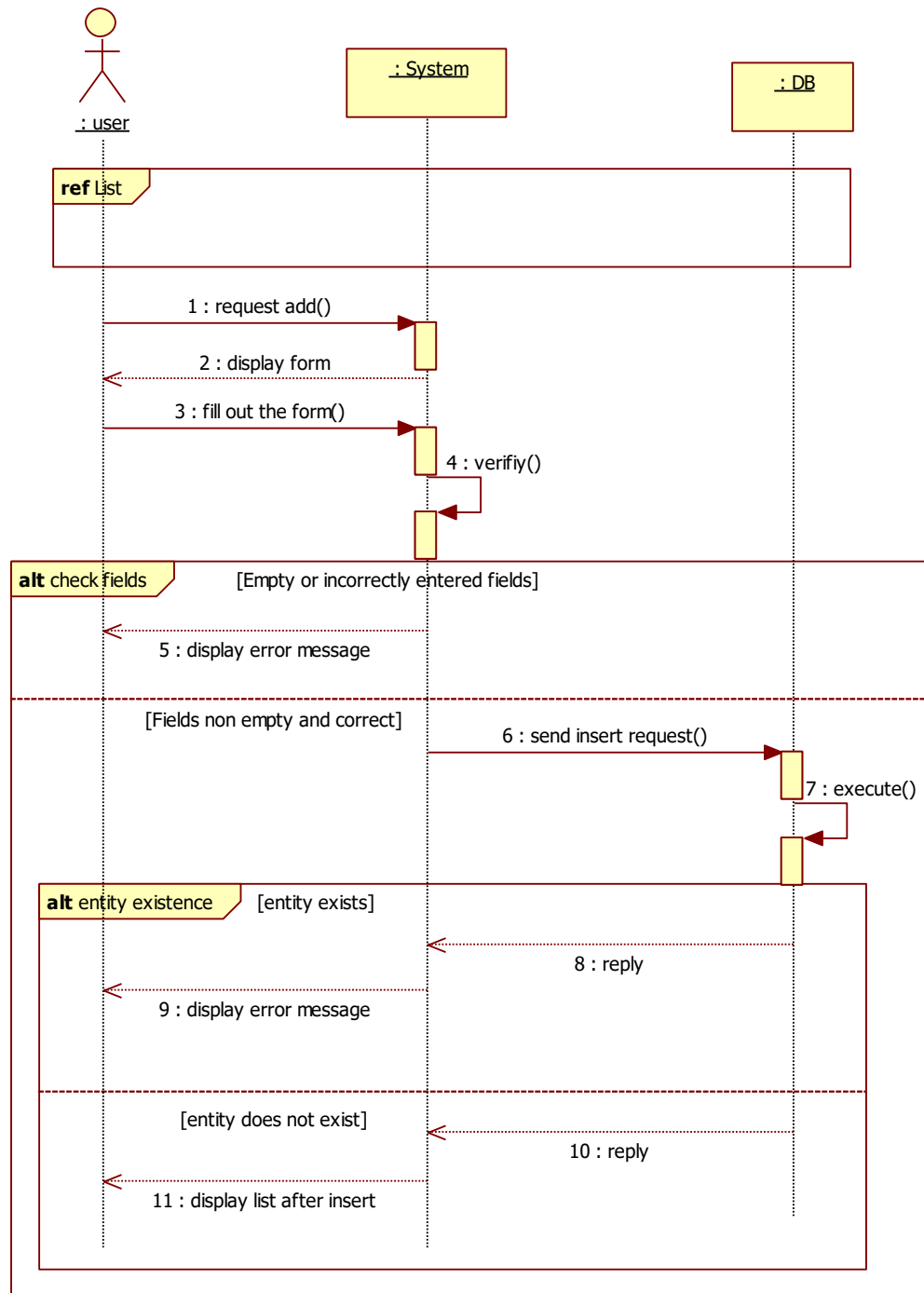


Figure 2. 9: Sequence diagram of the use case "add subject"

3.4. Sequence diagram of the use case "edit teacher information"

To modify a Teacher's information, the administrator must select the teacher to modify. The system will import this information from the database. After modifying and validating the fields, the system checks the validity of the fields and then sends an update request to the database.

The database management system will verify the validity of the data. After saving and executing the request, the system will receive confirmation of the modification and display an update confirmation message.

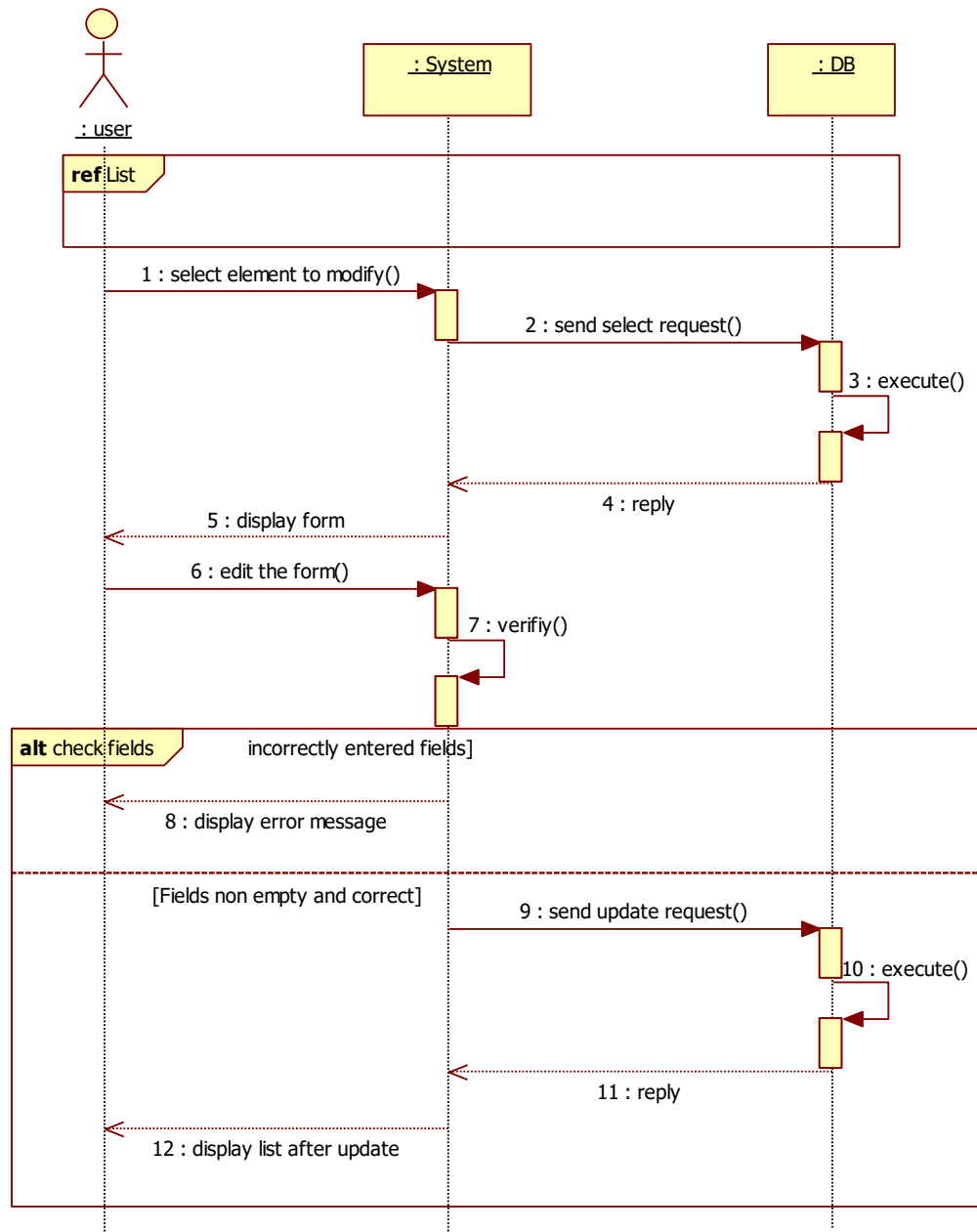


Figure 2. 10: Sequence diagram of the use case "edit teacher"

3.5. Sequence diagram of the use case "delete a class"

To delete a class, the user must select the class to be removed from the list.

The system will ask for confirmation. After confirmation, the system will send a deletion request to the database. After executing the request, the system will receive confirmation of the deletion and display the new list of classes.

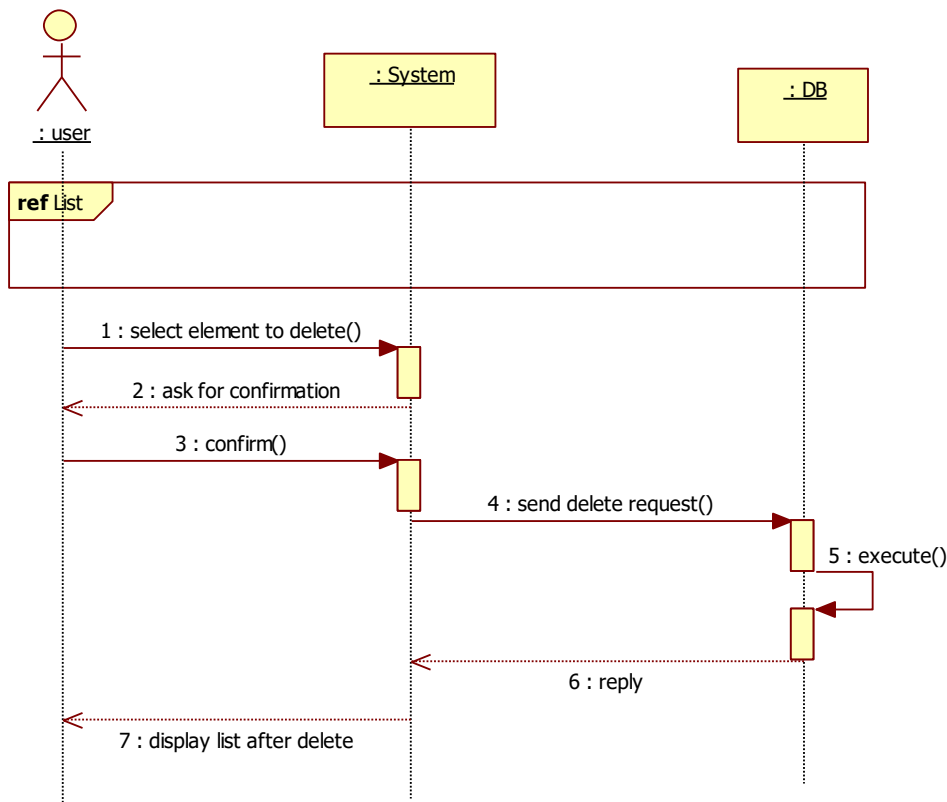


Figure 2. 11: Sequence diagram of the use case "delete class"

3.6. Sequence diagram of the use case "generate Timetable"

To generate a timetable, the headmaster must first ensure that all required academic data is correctly filled in (teachers, classes, subjects, periods, and lessons). Once verified, the user initiates the generation process by clicking the "Generate" button. The system retrieves the relevant data from the database and launches the scheduling algorithm. This algorithm processes constraints, availability, and optimization rules.

Once completed, the system stores the generated timetable in the database and displays it on the interface. The timetable has been successfully generated and shown.

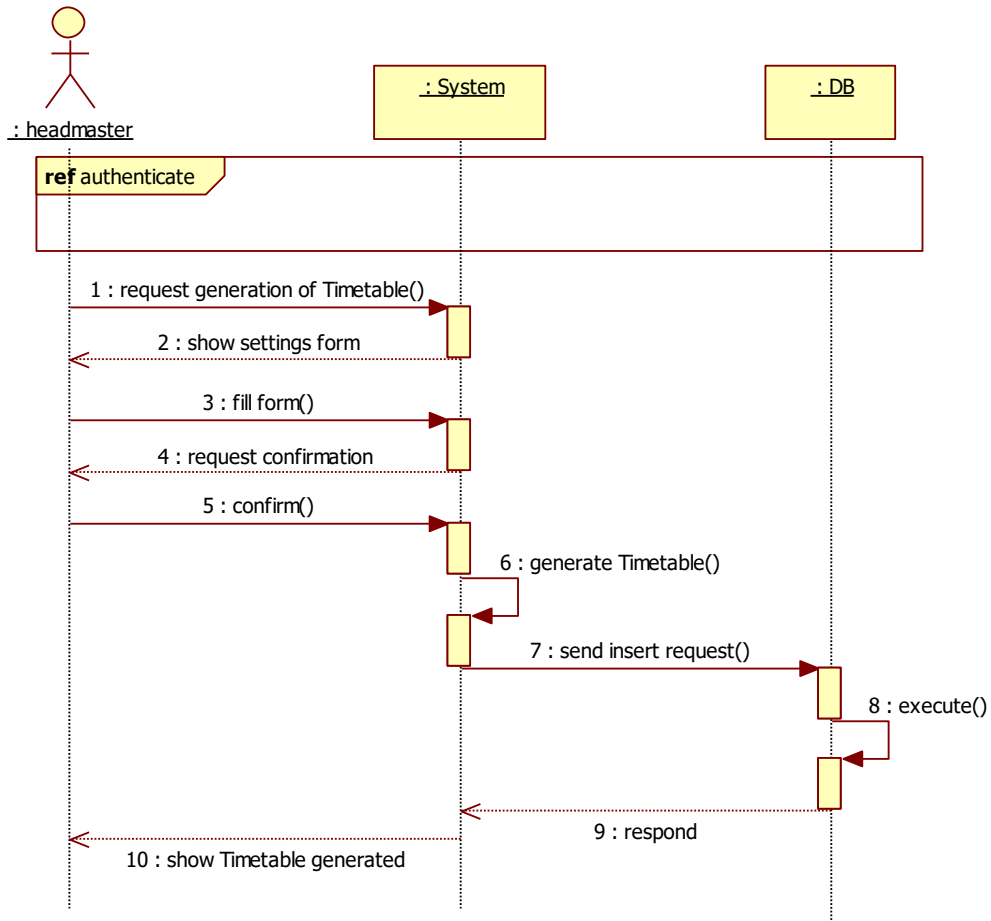


Figure 2. 12: Sequence diagram of the use case "generate Timetable"

4. Activity diagram

The activity diagram is intended to represent the dynamic views of the system. Indeed, it reveals the interactions between objects from a temporal point of view by focusing on the chronology of message sending.

In the following figure we present the overall activity diagram of our system.

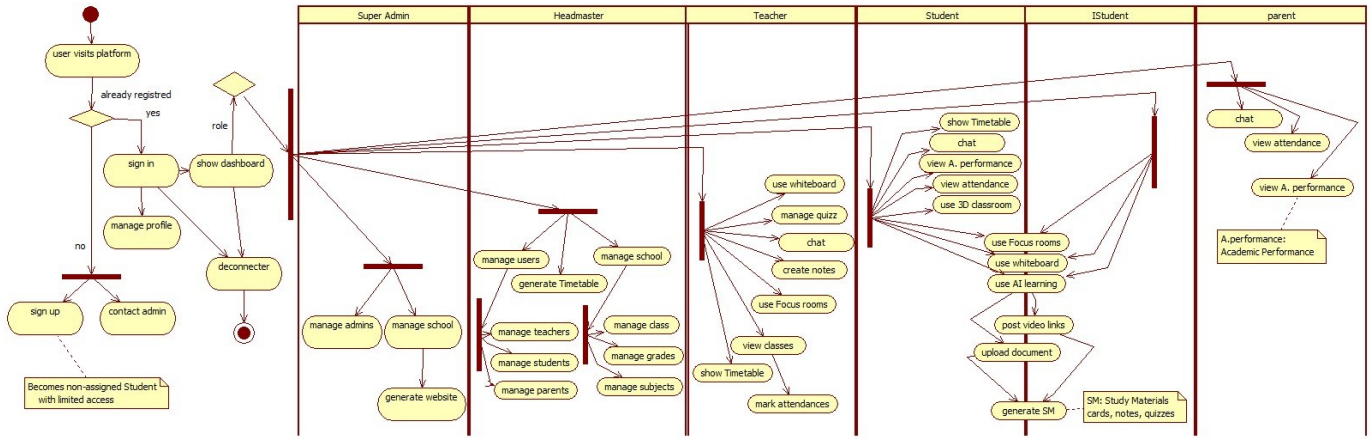


Figure 2. 13: Activity diagram

5. State diagram

A state diagram is a visual representation of the possible states an object or system can be in, along with the transitions between those states. It depicts how an object's behavior changes over time in response to events or stimuli. The state diagram below illustrates the lifecycle of two critical processes in the system: student creation by the headmaster and timetable generation.

In the case of student creation, the system progresses from form initialization to validation and confirmation.

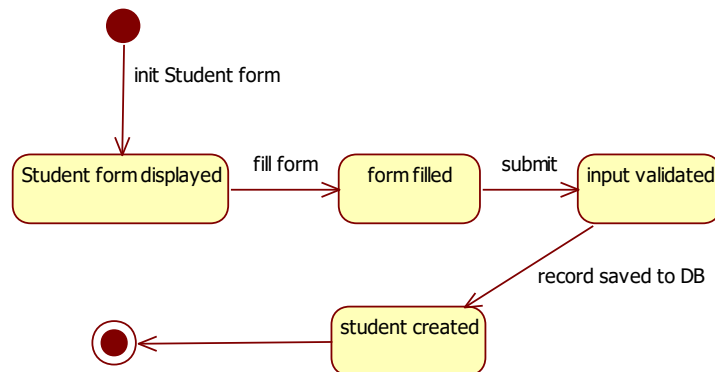


Figure 2. 14: Statechart diagram "create student"

For timetable generation, the system transitions through configuration, constraint verification, algorithm execution, and result presentation.

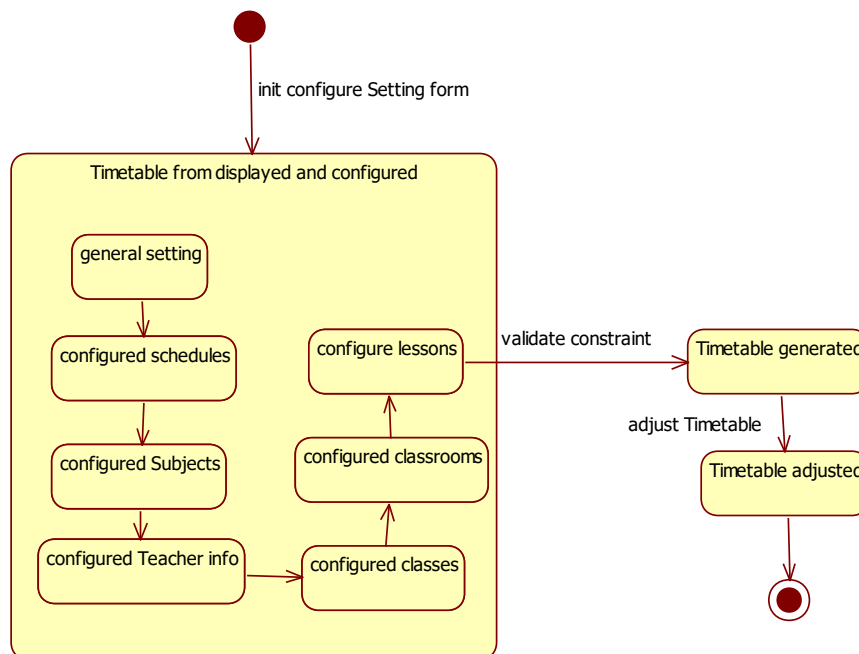


Figure 2. 15: Statechart diagram "Timetable generation"

6. Design Class Diagram

In the Unified Modeling Language (UML), a class diagram is a type of static structure diagram that depicts the structure of a system by showing the system's classes, their attributes, operations or methods, and the relationships between the classes. Below is the overall class diagram of our application. Here we will present all entities of system and the class diagram. At this stage, we will start the design phase, during which we illustrate our design by presenting class diagrams.

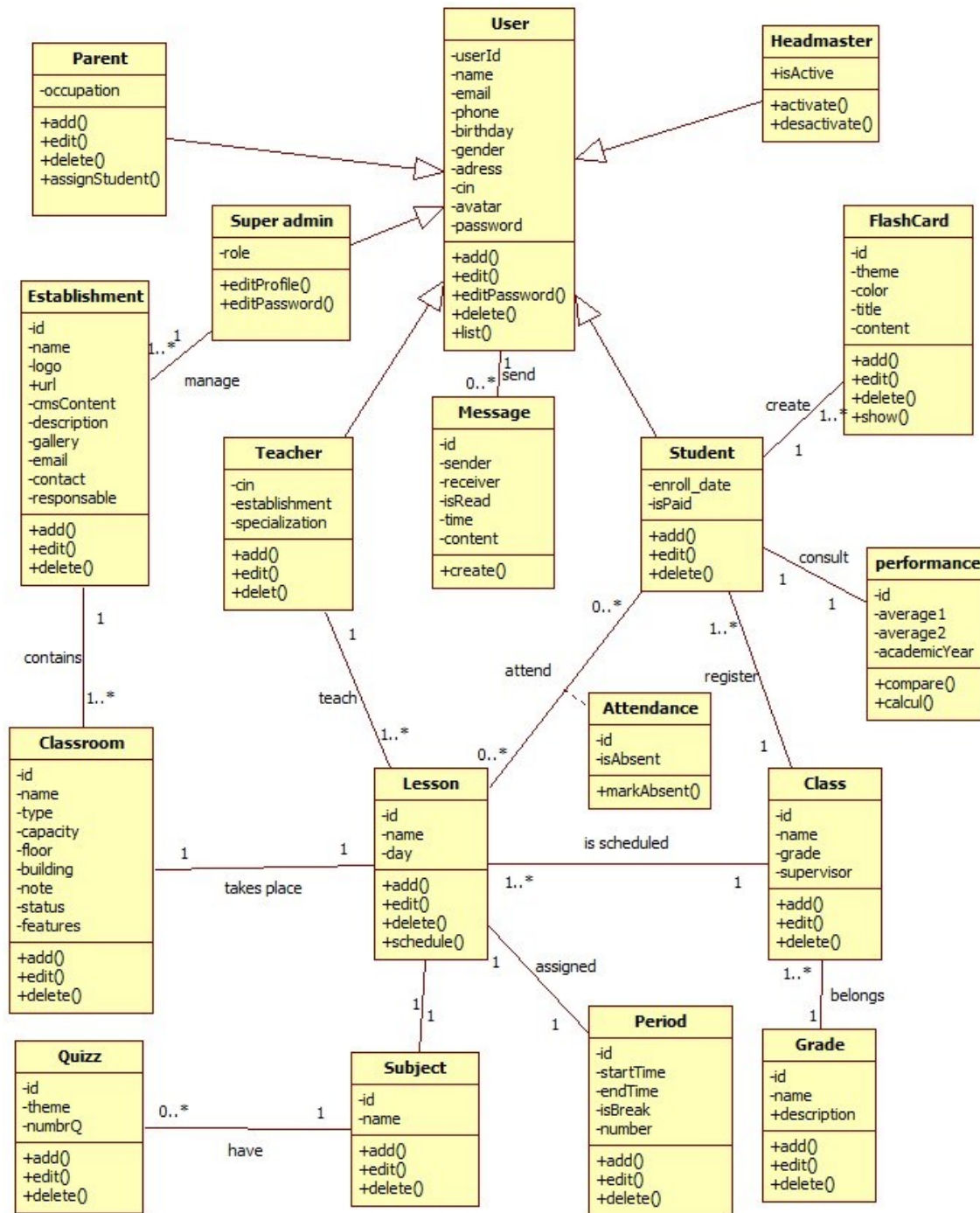


Figure 2. 16: Class Diagram

Conclusion

In this chapter, we examined a set of use cases and a set of usage scenarios describing the possible interactions between users and the application in terms of functionality. We presented the detailed architectural, static, and dynamic modeling of our future application. After completing this phase, we will then begin the present the project planning process. The following chapter highlights our methodological choice and the sprints to achieve during this project.

Chapter 3: Engineering Process & Technologies

Introduction

This chapter outlines the development methodology adopted for the project, with a focus on agile and iterative practices. We present the project planning process, the division into sprints, and the tasks allocated for each sprint. Each sprint is described in detail, including the weekly objectives, the features implemented, and the difficulties encountered. This chapter highlights how agile development allowed us to progressively build and validate the solution, adapting to feedback and changes along the way.

1. Adopted development methodology

For a successful project, one of the most important steps is choosing the right work methodology at the very beginning. It is like it is a critical phase to choose the most suitable methodology for our work. There are various methodologies to choose between and each one has her own principles, process and rules.

1.1. Work Methodology

Choosing the methodology varies from a project to another project based on specific the type and the size of the project and the team that will be working on it and it is used to increase client satisfaction and reduce the project complexity. During this project, in order to ensure an efficient and iterative development process, we have adopted the Agile methodology. This approach allows for continuous feedback, adaptability to changes, and improved collaboration between team members.

Agile: one of the most recognizable methodologies, it is a system in which demands and effort evolve through time and collaborative effort of both working team and clients. Therefore, it is the best choice for the iterative and incremental projects that requires flexibility. Made up of four fundamental values [2]:

- Iterative Development: The project is divided into small, manageable cycles (sprints) that ensure incremental improvements.

- Continuous Feedback: Regular interactions with stakeholders allow us to refine requirements and improve the final product.
- Flexibility and Adaptability: Agile allows quick adjustments based on evolving needs and priorities.
- Collaboration and Communication: Daily stand-up meetings and sprint reviews ensure team alignment and progress tracking.



Figure 3. 1: Agile methodology

1.2. Kanban Implementation

For this project, I chose to implement the Kanban methodology for task management and workflow visualization. Kanban offered several advantages that made it particularly suitable for the JeridSchool project: [3]

- Visual Workflow: Kanban’s visual board allowed the entire team to see the status of all tasks at a glance, improving transparency and accountability.
- Continuous Flow: Unlike time-boxed methodologies, Kanban enabled a continuous flow of work, allowing for more flexible prioritization and resource allocation.
- Work-in-Progress Limits: By implementing WIP limits, we prevented bottlenecks and ensured that tasks were completed before new ones were started.
- Just-in-Time Delivery: Kanban facilitated a pull-based system where work was pulled as capacity allowed, leading to more efficient resource utilization.
- Reduced Context Switching: Team members could focus on fewer tasks simultaneously, reducing the overhead of switching between multiple priorities.

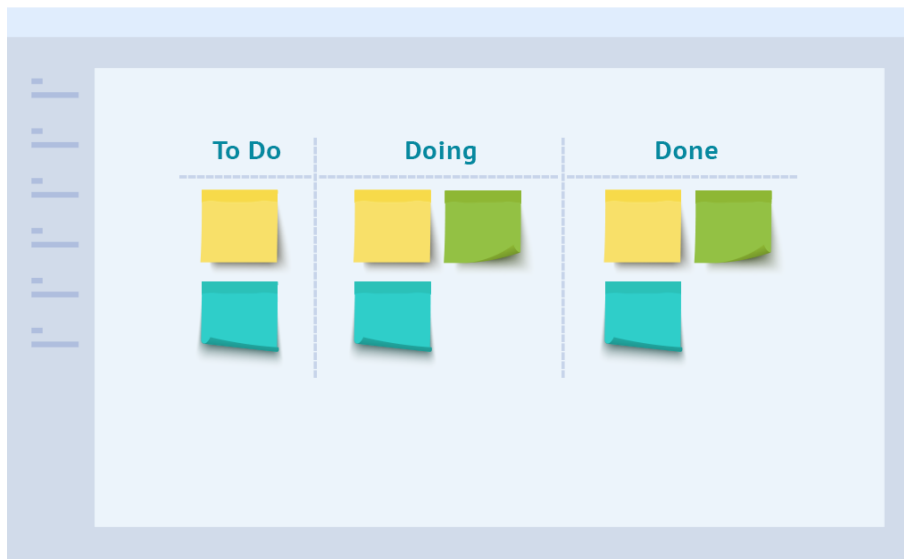


Figure 3. 2: Kanban Board Used for Project Management

1.3. Comparison with Other Methodologies

When compared to other agile methodologies like Scrum, Kanban offered several distinct advantages for our specific project needs:

- Greater Flexibility: Unlike Scrum's fixed sprint lengths, Kanban allowed us to adapt to changing priorities without waiting for the end of a sprint cycle.
- Smoother Workflow: The continuous delivery approach of Kanban eliminated the start-stop nature of sprint-based methodologies, providing a more consistent development pace.
- Reduced Planning Overhead: Kanban required less ceremonial meetings, allowing more time for actual development work.
- Faster Response to Changes: Without the commitment to a sprint backlog, we could quickly respond to stakeholder feedback and changing requirements.
- Evolutionary Approach: Kanban encouraged gradual improvement of existing processes rather than imposing a predetermined framework.

By adopting Kanban, we ensured that our development process remained efficient, adaptable, and focused on delivering value continuously, ultimately leading to a high-quality solution that met stakeholder expectations.

1.4. Sprint planning

This section provides a comprehensive overview of the sprint planning process used throughout the project. The detailed table below summarizes the timeline, objectives, and key deliverables of each sprint. The purpose is to give readers a clear and concise roadmap of the development phases and how the project milestones were organized to ensure steady progress and alignment with project goals.

Sprint #	Duration	Focus Area	Weekly Breakdown & Key Tasks	Deliverables / Outcomes
Sprint 1	Feb 3 – Feb 28	Core System Foundation	Week 1 (Feb 3–7): Planning, requirements analysis, UML diagrams, system architecture design	Project scope clarified, UML diagrams, system architecture blueprint
			Week 2 (Feb 10–14): Backend setup with Node.js & Express, JWT authentication, CRUD APIs, role-based access control	Secure backend, tested APIs with Postman, role-based API routes
			Week 3 (Feb 17–21): Frontend with React/Vue, mock data, UI components, login/register forms, role-based views	Functional frontend UI with mock integration, role-based dashboards
			Week 4 (Feb 24–28): Frontend-backend integration, form submission, image upload with CDN setup, debugging	Fully integrated front and back ends, image upload support, deployment preparation
Sprint 2	Mar 3 – Mar 29	Timetable Scheduler	Week 1 (Mar 3–7): Analyze user needs, constraints, and existing tools; design JSON configuration for scheduling	Scheduling constraints & config defined, problem well understood
			Week 2 (Mar 11–15): Implement brute force timetable generation (inefficient but baseline)	Brute force prototype created, identified performance issues
			Week 3 (Mar 18–22): Develop Greedy Algorithm for faster scheduling	Faster, scalable scheduler with partial constraint satisfaction
			Week 4 (Mar 25–29): Hybrid Greedy + Genetic Algorithm to optimize scheduling results	Smart scheduler achieving ~90% success, balancing hard & soft constraints

Sprint 3	Apr 1 – Apr 26	Interactive Learning & Communication	Week 1 (Apr 1–5): Research & design chat architecture; real-time communication tech evaluation	Chat module design, backend and frontend chat UI foundation
			Week 2 (April 8–12): Building the Real-Time Chat Feature (Message Encryption + UI Polish)	Real-time chat system implemented
			Week 3 (Apr 15–19): Quiz generator backend, flashcard system design & frontend implementation	Quizzes and flashcards features ready
			Week 4 (Apr 22–26): Collaborative whiteboard, final integration, QA testing	Interactive whiteboard, fully integrated learning tools
Sprint 4	Apr 28 – May 25	Mobile Development & Deployment	Week 1 (Apr 28 – May 3): Mobile app foundation, authentication, API integration	Mobile app skeleton with login & role-based navigation
			Week 2 (May 5–10): Core features on mobile: schedules, profiles, chat, learning tools	Functional mobile app with core features
			Week 3 (May 12–17): Hosting and deployment of web app; security and performance optimizations	Live web app hosted on Vercel (frontend) and Render (backend)
			Week 4 (May 19–25): Final testing, monitoring, mobile API integration, CI/CD setup, analytics	Full production deployment, monitoring, continuous deployment pipeline

2. Key concepts and Engineering Foundations

Introduce this section as a summary of the modern engineering concepts used throughout the development process.

2.1. DevOps

DevOps aims to bridge the gap between developers and system administrators by promoting a set of practices that streamline the software lifecycle from code development and testing to deployment and monitoring [4].

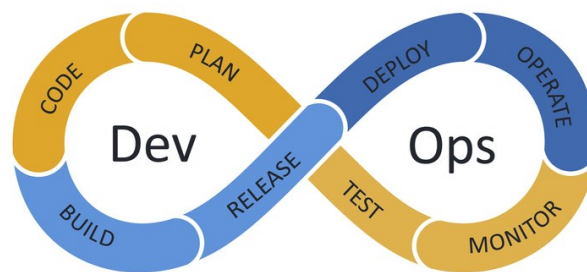


Figure 3. 3: Visual representation of DevOps Life Cycle

- Plan: Define project requirements, scope, and goals in collaboration with all stakeholders to align development with business needs.
- Develop: Write, review, and manage source code using version control systems, ensuring code quality and collaboration.
- Build: Compile source code and dependencies into deployable packages using automated build tools.
- Test: Automatically or manually verify that the application behaves as expected, identifying bugs and regressions early.
- Release: Prepare the application for deployment, including tagging versions, managing environments, and ensuring approval workflows.
- Deploy: Deliver code to production or staging environments using automated deployment pipelines for consistency and speed.
- Operate: Monitor application performance, manage infrastructure, and ensure uptime and responsiveness of services.
- Monitor: Continuously collect metrics and logs to gain insights into system health, performance, and user behavior.

2.2. Containerization and Environment Management

Containers have become a central element in modern DevOps practices, offering a lightweight and consistent way to run applications across different environments. A container is a compact unit that bundles an application with its dependencies and configuration, ensuring it runs reliably regardless of the host system. Unlike virtual machines, containers share the host OS, making them faster and more efficient.

Container management involves controlling the lifecycle of containers—creating, running, updating, and scaling them. Tools like Docker, LXC/LXD (Linux Containers) and Podman simplify this process by offering command-line and API access to container operations.

For more complex setups involving multiple containers, Docker Compose allows developers to define and run multi-container environments using a single YAML file. This improves organization and speeds up development [5].

2.3. CI/CD

CI/CD, which stands for Continuous Integration/Continuous Delivery, is a set of practices and processes aimed at automating software development and deployment, enabling fast and reliable deliveries. It is essential in DevOps approaches to accelerate development cycles and improve software quality [6].

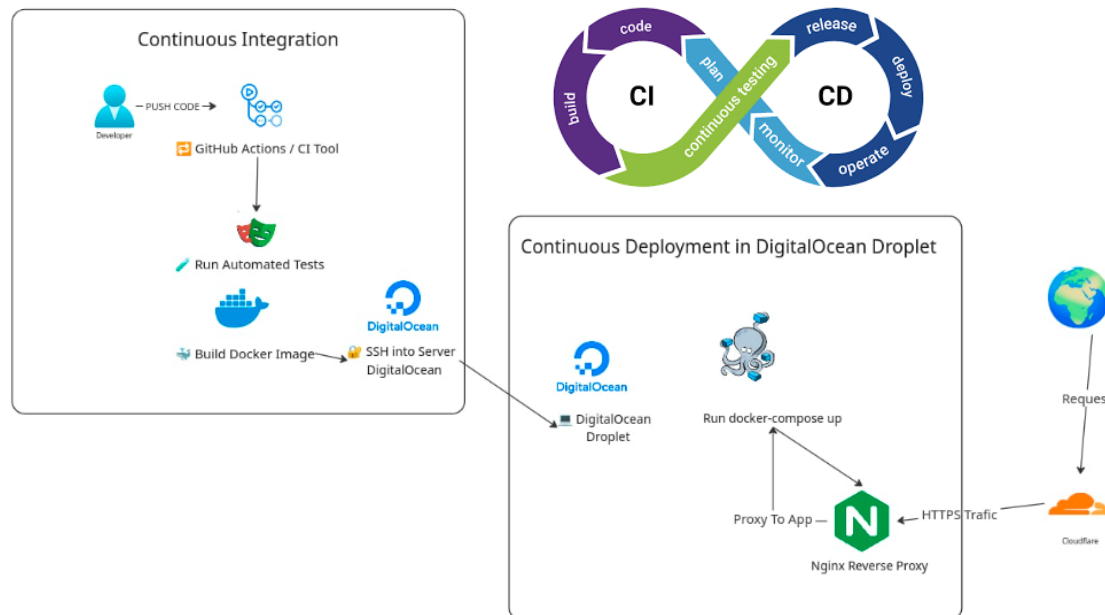


Figure 3. 4: CI/CD process

Continuous Integration (CI) involves the regular integration of developers' code into a central repository, followed by automated testing to quickly detect errors. This helps prevent compatibility issues and maintain code quality. Continuous Delivery/Deployment (CD) automates the process of testing and delivering code changes, while continuous deployment also automates the process of deploying final versions to end users.

CI/CD Accelerated development, improved quality, reduced errors, increased release frequency, and better collaboration between teams.

A CI/CD pipeline is a set of automated steps that handle merging code, testing it, and pushing it to production. CI (Continuous Integration) helps teams catch bugs early by testing code every time it's updated. CD (Continuous Deployment) automatically releases the tested code, reducing the need for manual steps [6].

This process speeds up development, lowers the risk of errors, and improves team efficiency.

2.4. Devops and deployment tools

To deploy our project we used the following technologies:

2.4.1. *Docker*

Docker is a software platform that makes it easy to develop, deploy, and run containerized applications launched in 2013. Containers are units of software that package code and all its dependencies into a standardized format, enabling faster and more reliable execution across different computing environments.



Figure 3. 5: docker logo

According to industry research firm 451 Research, "Docker is a tool that can package an application and its dependencies into an isolated container, which can be run on any server." This isn't virtualization, but containerization, a lighter-weight form that relies on certain parts of the host machine for its operation. This approach increases the flexibility and portability of an application's execution, allowing it to run reliably and predictably on a wide variety of host machines, whether on-premises, in a private or public cloud, on bare metal, etc [7].

2.4.2. *Web Server*

nginx ("engine x") is an HTTP web server, reverse proxy, content cache, load balancer, TCP/UDP proxy server, and mail proxy server. Originally written by Igor Sysoev and distributed under the 2-clause BSD License. Enterprise distributions, commercial support and training are available from F5, Inc [8].



Figure 3. 6: Nginx logo

NGINX is an asynchronous system, as opposed to synchronous servers where each request is handled by a dedicated process. Instead of leveraging a parallel architecture and time-based multiplexing of tasks by the operating system, NGINX uses state changes to handle multiple connections at the same time; the processing of each request is broken down into many mini-tasks, thus enabling efficient multiplexing between connections. To take advantage of multiprocessor computers, multiple processes can be started. This architectural choice results in very high performance, as well as much lower load and memory consumption than traditional HTTP servers like Apache [8].

2.4.3. *Cloud Infrastructure*

DigitalOcean is a cloud computing service provider that offers virtual machines, managed databases, and scalable hosting solutions. It simplifies deployment with pre-configured environments and automated scaling. In this project, DigitalOcean was used to deploy and manage the backend and database services efficiently [9].



Figure 3. 7: DigitalOcean logo

Regarding the backend, we chose DigitalOcean. While it is neither the cheapest nor the most expensive provider, it offers excellent value for money, strong customer trust, and a well-balanced pricing model, making it a popular choice among developers worldwide.

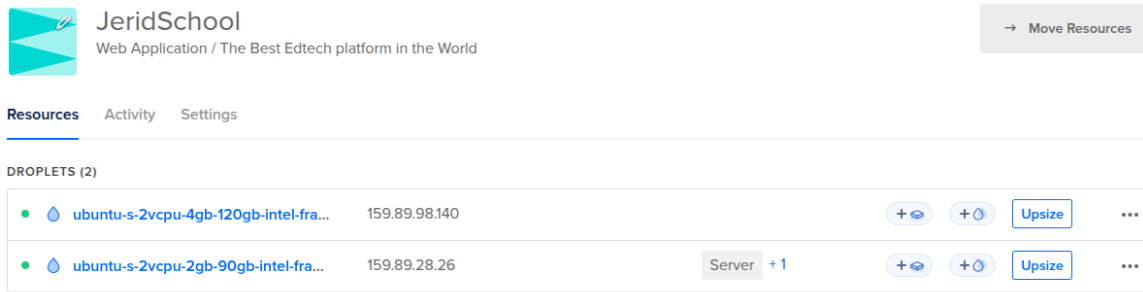


Figure 3. 8: DigitalOcean Droplets

We used two DigitalOcean Droplets as our core compute infrastructure, virtual CPU servers deployed in the cloud to host our backend services and applications.

We integrated Cloudflare to secure our domain and handle DNS resolution, benefiting from features like SSL encryption, DDoS protection, and performance optimization all within the free plan. Cloudflare's dashboard allowed us to easily configure HTTPS enforcement, firewall rules, and caching policies to enhance both security and speed. DigitalOcean provides easy-to-use monitoring tools that keep us informed about incoming traffic and help us track resource consumption in real time for free

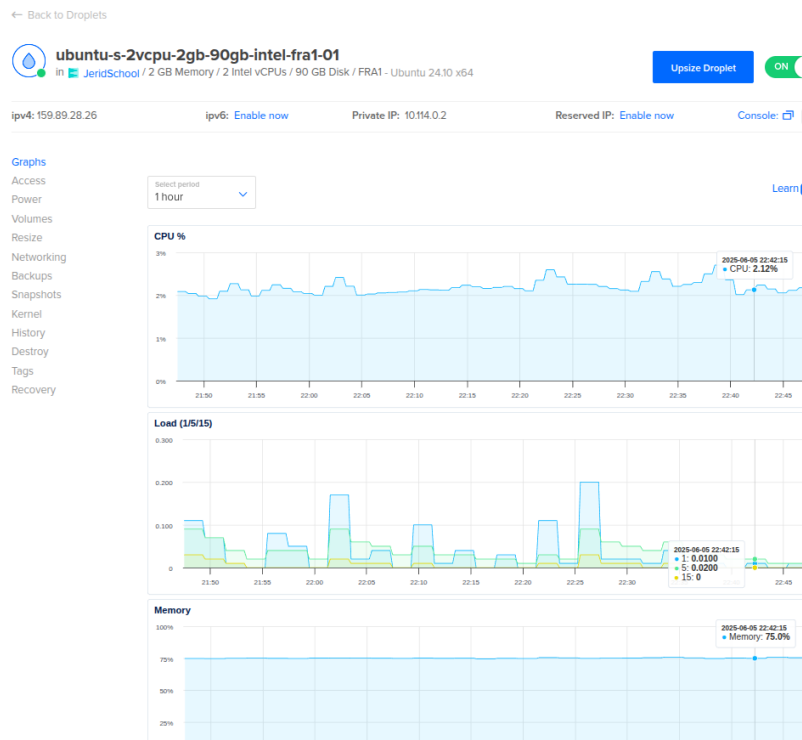


Figure 3. 9: DigitalOcean Monitor Tools

This image below displays the available VM prices and their corresponding specifications in the DigitalOcean dashboard. As a fresh startup, we selected two machines: one for the core features, such as our NestJS servers, and a more powerful one dedicated to the timetable generator and AI tasks. The latter requires significantly more resources, so we opted for a web-based solution hosted on two separate machines: one with 4 GB of RAM and 2 CPUs dedicated to AI processing, and another with 2 GB of RAM and 2 CPUs to handle standard application services efficiently.

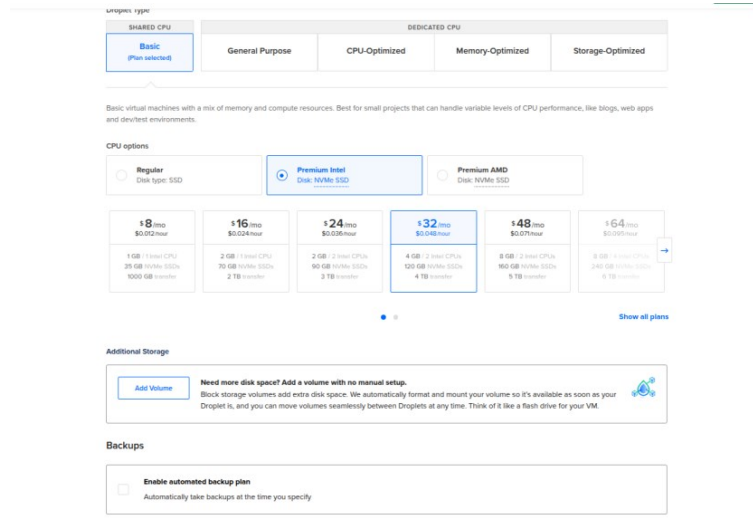


Figure 3. 10: DigitalOcean Machine Prices

For AI-related services, we partnered with Azure AI Studio (Foundry). In April 2025, we received a \$150,000 startup grant, which enabled us to leverage Azure as our primary AI provider. While we do not use Azure servers for our core infrastructure, opting instead for more cost-effective and powerful DigitalOcean droplets, we reserve Azure resources specifically for AI workloads due to their specialized requirements.

Azure's model offerings are competitively priced compared to proprietary alternatives such as OpenAI and Claude. Additionally, Azure provides affordable token-based pricing and managed models, which are more practical for our needs than self-hosted open-source solutions like DeepSeek-R1 or Phi. The cost structure of this AI model will be detailed in the next section.

3. Technological choices

The choice of the technical environment and development tools is importance to ensure the success of the project. This section provides an overview of the key technologies used in this project, detailing their purpose and role in the development process.

3.1. Operating system: Arch Linux.

A lightweight and flexible Linux distribution that tries to Keep It Simple. Arch Linux is an open source, rolling release Linux distribution. Arch Linux is kept up-to-date by regularly updating the individual pieces of software that it comprises. Arch Linux is intentionally minimal, and is meant to be configured by the user during installation so they may add only what they require. Arch Linux provides monthly "snapshots" which are used as installation media [10].



Figure 3. 11: Arch Linux logo

3.2. Development Tools

In this section, we will present all development tools adopted to realize our project:

3.2.1. *Modeling tool*

StarUML is a software engineering tool for system modeling using the Unified Modeling Language, as well as Systems Modeling Language, and classical modeling notations. It is published by MKLabs and is available on Windows, Linux and MacOS [11].



Figure 3. 12: StarUML logo

3.2.2. *Visual Studio Code*

Visual studio code or VS Code is a code editor developed by Microsoft in 2015. Unlike what Microsoft was used to for years, it is one of the first open source and free products, and especially available on Windows, Linux and Mac operating systems. The code is developed with the Electron framework and is designed mainly to develop projects with JavaScript, Node.js or TypeScript. It is a lightweight but powerful code that has essential features for writing code such as automatic indentation,

syntax highlighting, auto-completion, code refactoring, etc. It provides built-in support for debugging, syntax highlighting, Git integration, and extensions. For this project, VS Code was used as the primary code editor for development and debugging [12].

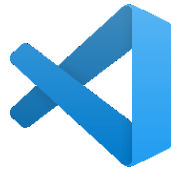


Figure 3. 13: Visual Studio Code logo

3.2.3. Version control system

Git is a distributed version control system that tracks changes in source code during software development. It enables developers to collaborate, revert to previous versions, and work on multiple branches simultaneously. In this project, Git was used to maintain version control, manage feature branches, and integrate updates [13].



Figure 3. 14: git logo

3.2.1. Code repository and Version Control and Collaboration

GitHub is a cloud-based platform for version control and collaborative software development using Git. It allows multiple developers to work on the same project, track changes, and manage issues. For this project, GitHub was used to host the code repository and facilitate team collaboration [14].



Figure 3. 15: GitHub logo

3.2.2. *GitHub Actions*

GitHub Actions is a cloud-based continuous integration and continuous delivery (CI/CD) platform offered by GitHub that allows you to automate workflows, build, test, and deploy your code directly from your repository. It utilizes YAML files to define workflows, allowing you to automate tasks like testing, building, and deployment. GitHub Actions makes it easy to automate all software workflows, now with world-class CI/CD. Build, test, and deploy code right from GitHub. Make code reviews, branch management, and issue triaging work the way we want.

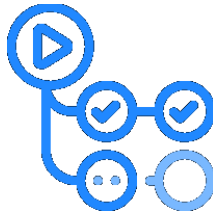


Figure 3. 16: Github action logo

3.2.3. *UI/UX Design*

Figma is a cloud-based collaborative design tool for creating user interfaces, prototypes, and design systems. It allows real-time collaboration, making it easy for designers and developers to work together. In this project, Figma was used to design the application's wireframes and UI components [15].



Figure 3. 17: Figma logo

3.2.4. *Test tools*

Postman is a versatile API platform for building, testing, and documenting APIs. It offers a comprehensive set of tools to streamline the API lifecycle, from design to deployment and monitoring. Postman is used by over 30 million developers and is known for its collaborative features and ease of use. Postman is an all-in-one API-Platform for building and working with APIs. It takes the pain out of every stage of the API lifecycle from designing and testing to delivery and monitoring. Built for teams, Postman makes it easy to collaborate, stay organized, and build secure, reliable APIs faster [16].



Figure 3. 18: Postman logo

3.2.5. *Documentation tool*



Figure 3. 19: Overleaf logo

Overleaf is an online, collaborative LaTeX editor. It allows users to write, edit, and produce documents, especially research papers and reports, using LaTeX. It offers real-time collaboration, visual and code editing, and access to a vast library of templates and resources.

LaTeX is a software system for typesetting documents. LaTeX markup describes the content and layout of the document, as opposed to the formatted text found in WYSIWYG word processors like Google Docs, LibreOffice Writer, and Microsoft Word. The writer uses markup tagging conventions to define the general structure of a document, to stylize text throughout a document (such as bold and italics), and to add citations and cross-references. A TeX distribution such as TeX Live or MiKTeX is used to produce an output file (such as PDF or DVI) suitable for printing or digital distribution [17].

3.3. Frontend technologies

In this section, we will present frontend technologies adopted to realize our project:

3.3.1. *React & React native*

React is a JavaScript library for building fast, interactive, and reusable user interfaces with a component-based architecture. It enables efficient updates and rendering with its virtual DOM mechanism. For this project, React was used to create a responsive and dynamic user interface for the application [18].

React Native is a framework for building native apps using JavaScript. React Native compiles native app components, allowing you to build native mobile apps.



Figure 3. 20: React logo

3.3.2. *TanStack*

TanStack (React Query) is a powerful data-fetching and state management library designed for React applications. It simplifies server-state management, caching, and background updates, improving performance. In this project, TanStack was used to optimize API calls and handle asynchronous data efficiently [19].



Figure 3. 21: TanStack logo

3.3.3. *Interactive 3D Components Three.js*

To enhance user engagement and visualize virtual learning environments, we integrated Three.js into the platform. This JavaScript 3D library allowed us to design an interactive 3D Classroom experience, where students can explore a virtual study space. The implementation supports visual context for learning, providing an immersive feel. This feature was integrated directly into the frontend and optimized for responsiveness across devices [20].

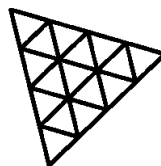


Figure 3. 22: three js logo

3.4. Backend technologies

In this section, we will present backend technologies adopted to realize our project:

3.4.1. *Node.js*

Node.js is a JavaScript runtime built on Chrome's V8 JavaScript engine for building fast, scalable server-side applications. It utilizes an event-driven, non-blocking I/O model that makes it lightweight and efficient for data-intensive applications. For this project, Node.js was used as the foundation of our backend services, providing a consistent JavaScript environment across the stack [21].



Figure 3. 23: Node js logo

3.4.2. *NestJS*

NestJS is a progressive Node.js framework for building efficient, scalable, and maintainable server-side applications. It leverages TypeScript and incorporates concepts from object-oriented programming, functional programming, and reactive programming.

In this project, NestJS was used to develop RESTful APIs, manage HTTP requests, and implement modular, scalable authentication mechanisms through its powerful dependency injection system and built-in decorators [22].



Figure 3. 24: Nest Js logo

3.4.3. *Fastapi*

FastAPI is a modern, fast (high-performance), web framework for building APIs with Python based on standard Python type hints [23].

The key features are:

- **Fast:** Very high performance, on par with **NodeJS** and **Go** (thanks to Starlette and Pydantic). One of the fastest Python frameworks available.
- **Fast to code:** Increase the speed to develop features by about 200% to 300%.
- **Fewer bugs:** Reduce about 40% of human (developer) induced errors.
- **Intuitive:** Great editor support. Completion everywhere. Less time debugging.
- **Easy:** Designed to be easy to use and learn. Less time reading docs.
- **Short:** Minimize code duplication. Multiple features from each parameter declaration. Fewer bugs.
- **Robust:** Get production-ready code. With automatic interactive documentation.
- **Standards-based:** Based on (and fully compatible with) the open standards for APIs: OpenAPI (previously known as Swagger) and JSON Schema



Figure 3. 25: Fastapi logo

3.4.4. Python

Python is a high-level, general-purpose programming language. Its design philosophy emphasizes code readability with the use of significant indentation.

Python is dynamically type-checked and garbage-collected. It supports multiple programming paradigms, including structured (particularly procedural), object-oriented and functional programming. It is often described as a "batteries included" language due to its comprehensive standard library [24].



Figure 3. 26: Python logo

3.5. Database Management

PostgreSQL is a powerful, open-source object-relational database system known for its reliability, robustness, and feature-rich SQL support. It stores data in structured tables with defined schemas, enabling strong data integrity, complex queries, and transactional consistency. In this project, PostgreSQL was used to store and manage the application's data, providing a reliable foundation for structured data and complex relational queries [25].

Neon is a serverless open-source alternative to AWS Aurora Postgres. It separates storage and compute and substitutes the PostgreSQL storage layer.



Figure 3. 27: PostgreSql and Neon logos

4. Algorithmic Strategies

An algorithmic strategy refers to the specific method or approach used to solve a computational problem, particularly when there are multiple ways to reach a solution. These strategies define how a problem is tackled, the logic behind the solution, and the trade-offs between performance, accuracy, and complexity.

In the context of our project, algorithmic strategies play a crucial role because we are solving a constraint satisfaction problem that requires optimization and decision-making under multiple conditions and we are addressing a highly complex constraint satisfaction problem, the automatic generation of school timetables. This task is computationally intensive and widely recognized as NP-hard, involving a vast number of variables and constraints such as teacher availability, subject hour distribution, classroom types, and balanced scheduling. The exponential growth in possible schedule combinations makes naïve approaches infeasible in real-world scenarios.

Designing an effective and intelligent timetable generation system was therefore one of the most technically demanding components of the project. To overcome these challenges, we explored multiple algorithmic strategies with increasing levels of sophistication starting from brute-force enumeration to heuristic methods and finally to hybrid metaheuristic algorithms.

In this section, we explain the core algorithmic design decisions and the reasoning behind the evolution of our approach, aiming to deliver a fast, conflict-free, and adaptable timetable generation engine.

4.1. Timetable Generation Problem

Before starting implementation, we took time to study how scheduling is typically handled in educational systems. We analyzed tools like ASC Timetable and Timetable Master. These tools attempt to automate class scheduling, but often fall short in flexibility, integration, or constraint support.

To meet our specific needs, we built a JSON-based configuration that defines:

- Time Slots : Study times per day and rest periods
- Classroom Types : Normal rooms, labs, and specialized labs
- Class Subjects : Defined with hour splits, alternative subject logic, and required room type
- Teachers : With subject expertise, availability, and hour limits

Example excerpt from our configuration:

```
{
  "timeSlots": {
    "Monday": {
      "studyTimes": ["8 => 9", "9 => 10", "10 => 11", ...],
      "restTime": "12 => 14"
    }, ...
  },
  "salles": [
    { "name": "Salle 1", "type": "room" },
    { "name": "lab I", "type": "lab I" }
  ],
  "classes": [{
    "className": "1A",
    "subjects": [
      { "subject": "Math", "hours": 4, "divide": [2,2], "teacherID": 17 },
      ...
    ]
  } ],
  "teachers": [{
    "teacherId": 1,
    "unavailableTimes": {
      "Monday": ["8 => 9", "9 => 10", ...]
    },
    "minimumHours": 10,
    "maximumHours": 15
  } ]
}
```

4.2. Hard and Soft Rules in Scheduling

To guide the scheduler, we separated constraints into two categories:

- Hard Constraints (Must Be Satisfied)
 - No teacher or student can have overlapping classes.
 - Teachers can only teach within their availability.
 - Classes must use appropriate rooms (labs for IT, etc.).
 - Each subject must fulfill its required weekly hours.
 - A subject can't be resumed in the same day once finished.
- Soft Constraints (Preferred Conditions)
 - Classes should be evenly spread during the week.
 - Avoid idle time or “gaps” in schedules.
 - Respect teacher preferences for morning or afternoon sessions.
 - Keep students from having very short or overloaded days.

These constraints form the foundation for evaluating the quality of any generated schedule.

4.3. Algorithm Development and Evolution

Before finding the best solution and implementing it successfully, we tried several methods.

4.3.1. Brute Force Approach

Our first solution attempted to build a full timetable using brute force. This meant trying every possible permutation of class–time–room–teacher combinations through nested loops.

Brute Force Works as follow:

- For each class:
- For each subject:
- For each available time slot:
- For each room of the appropriate type:
- Check all hard constraints.
- If valid, assign it. If not, backtrack.

Although theoretically complete, brute force is impractical in real-world scenarios. As the number of classes, rooms, and constraints increases, the number of combinations becomes astronomical. For example, for 10 classes with 6 subjects and 40 available time slots, possible configurations exceed billions.

Even for a small dataset, generating a full schedule took over 20 minutes and still often failed to complete or violated constraints.

4.3.2. Greedy Algorithm

To overcome brute force limitations, we implemented a Greedy Algorithm. Instead of trying every option, the greedy algorithm makes the locally optimal choice at each step:

- Sort subjects by number of required hours (longer subjects scheduled first).
- For each subject, assign the first available valid slot that satisfies hard rules.
- Once a subject is placed, mark the slot as occupied for both the teacher and the class.

As results, we obtained:

- Schedule generation dropped from 20+ minutes to a few seconds.
- Success rate improved significantly, though not all subjects were always scheduled.
- Soft rules were often ignored due to local decisions.

4.3.3. Hybrid Greedy + Genetic Algorithm

Greedy works better than brute force in speed and scalability, but it's still limited in solution quality. To improve further, we developed a hybrid approach combining greedy initialization with genetic evolution.

Genetic Algorithm Basics

- Population: Start with a set of greedy-generated schedules.
- Fitness: Evaluate each schedule using a fitness function based on constraint violations and soft rule satisfaction.
- Crossover: Mix good schedules to produce children schedules.
- Mutation: Randomly alter some schedules to introduce diversity.
- Selection: Keep the best-performing schedules and repeat.

The hybrid algorithm achieves over 90% subject scheduling success with zero hard constraint violations, excellent soft rule compliance (balanced loads, minimal gaps), and reasonable generation time (around 30 seconds for medium datasets).

The hybrid algorithm offers an excellent balance of correctness, performance, and usability.

5. AI-Powered Features & Integrations

To enhance the intelligence and usability of our platform, we integrated advanced AI components that go beyond traditional school management functionalities.

5.1. Intelligent Document Processing with AI

As part of enhancing the digital capabilities of our solution, an AI-powered document processing module was designed and integrated using IBM's open-source toolkit Docling, combined with semantic search and LLMs (Large Language Models).

The main objective is to enable smart document querying and automatic extraction of contextual information from various file types (PDF, images, youtube links, etc.), especially for institutional archives or student records [26].



Figure 3. 28: Docling logo

Technical Workflow is as follows:

- Data Extraction: Inputs can be PDFs, image or youtube links; OCR is applied if needed to extract text content.
- Content Chunking: Documents are split into semantically coherent sections to facilitate accurate retrieval.
- Embedding Generation: Each chunk is vectorized using transformer-based models to build a semantic representation.

- Semantic Indexing: An index is created using vector databases to allow fast and accurate similarity search.
- Query Response via LLM: When users input a natural language query, the system retrieves the most relevant document chunks and feeds them to an LLM to generate a clear, human-like answer.

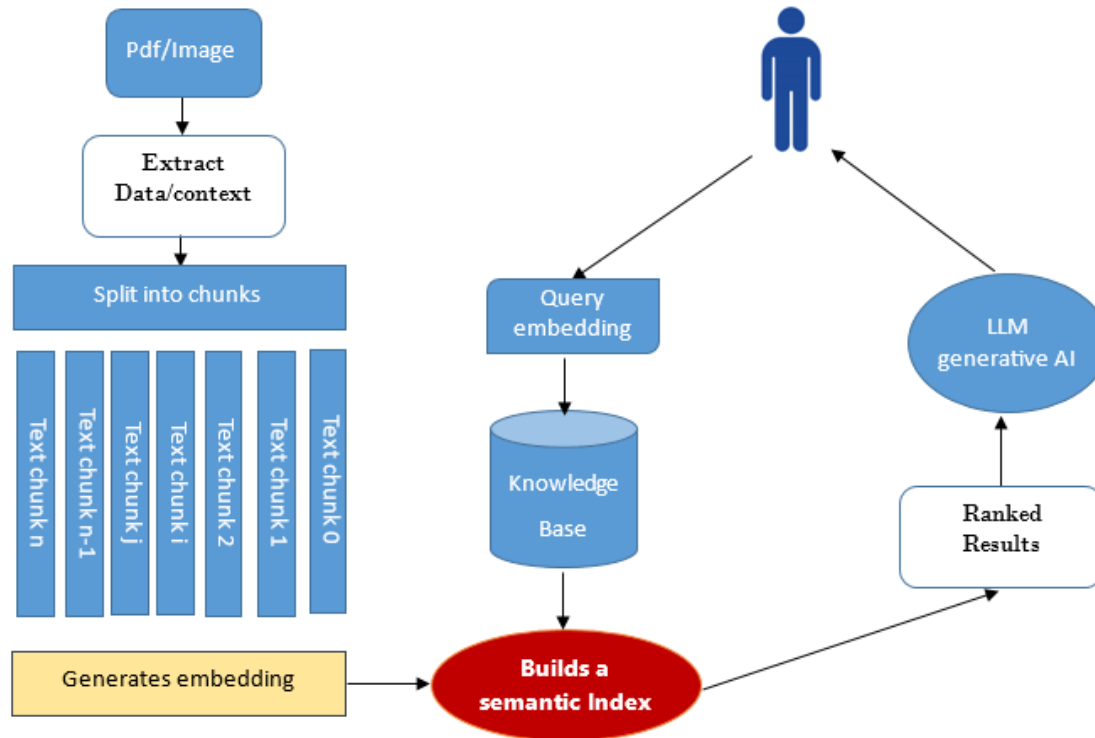


Figure 3. 29: AI-powered document processing pipeline

5.2. Speech-to-Text Integration with OpenAI Whisper

To enhance accessibility and input flexibility, we integrated OpenAI Whisper, a state-of-the-art automatic speech recognition (ASR) system. This system enables the platform to convert audio recordings into searchable, structured text, allowing users (e.g., teachers or administrators) to upload or record voice memos or YouTube videos that are transcribed and analyzed automatically [27].



Figure 3. 30: Whisper AI logo

6. Authentication & Security

JWT (JSON Web Token) authentication is a stateless, token-based authentication mechanism. It uses a compact, self-contained token (a JWT) to securely transmit information between parties, typically for authentication and authorization. This token contains claims, like user ID and permissions, and is signed for verification and security.

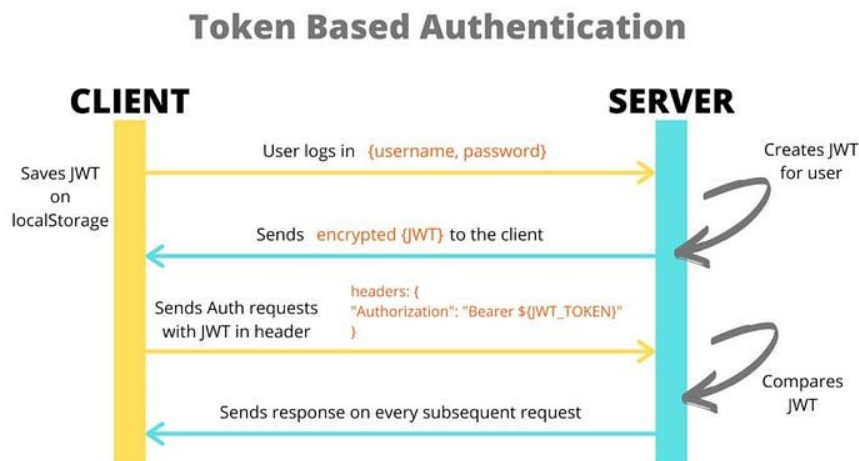


Figure 3. 31: Token based authentication

7. Real-time Communication

To carry out the chat and ensure real-time communication we need.

7.1.1. WebSockets

WebSockets are a protocol that enables a full-duplex communication channel over a single TCP connection, allowing for real-time data exchange between a client and a server. Unlike HTTP, which

typically uses a request-response pattern, WebSockets establish a persistent connection, enabling bidirectional communication without the need for frequent HTTP requests.

7.1.2. *Socket.IO*

WebSockets are a low-level, bi-directional communication protocol for real-time web applications, while Socket.IO is a higher-level library built on top of WebSockets, offering additional features like automatic reconnection and fallback to HTTP long-polling.

Socket.IO is a JavaScript library that builds on top of WebSocket and fallback transports. Though often compared, they serve different purposes and offer distinct tradeoffs when you're building large-scale, low-latency realtime systems.

8. Architecture Concept

Our solution is built using a modular, cloud-based architecture, designed to be scalable, secure, and customizable for multiple schools through a multi-tenant model. The architecture separates responsibilities across different layers and services to ensure maintainability, high availability, and performance.

The system includes authentication services, management modules (users, establishments, features, chat), and a Timetable Generator Service. Document processing and AI capabilities are handled by FastAPI services that parse documents, transcribe videos, and implement RAG functionality. The frontend consists of React-based interfaces for login, dashboards, AI interactions, and chat. Data storage is managed through MongoDB for chunks and PostgreSQL for structured data, with all services communicating through the centralized API Gateway.

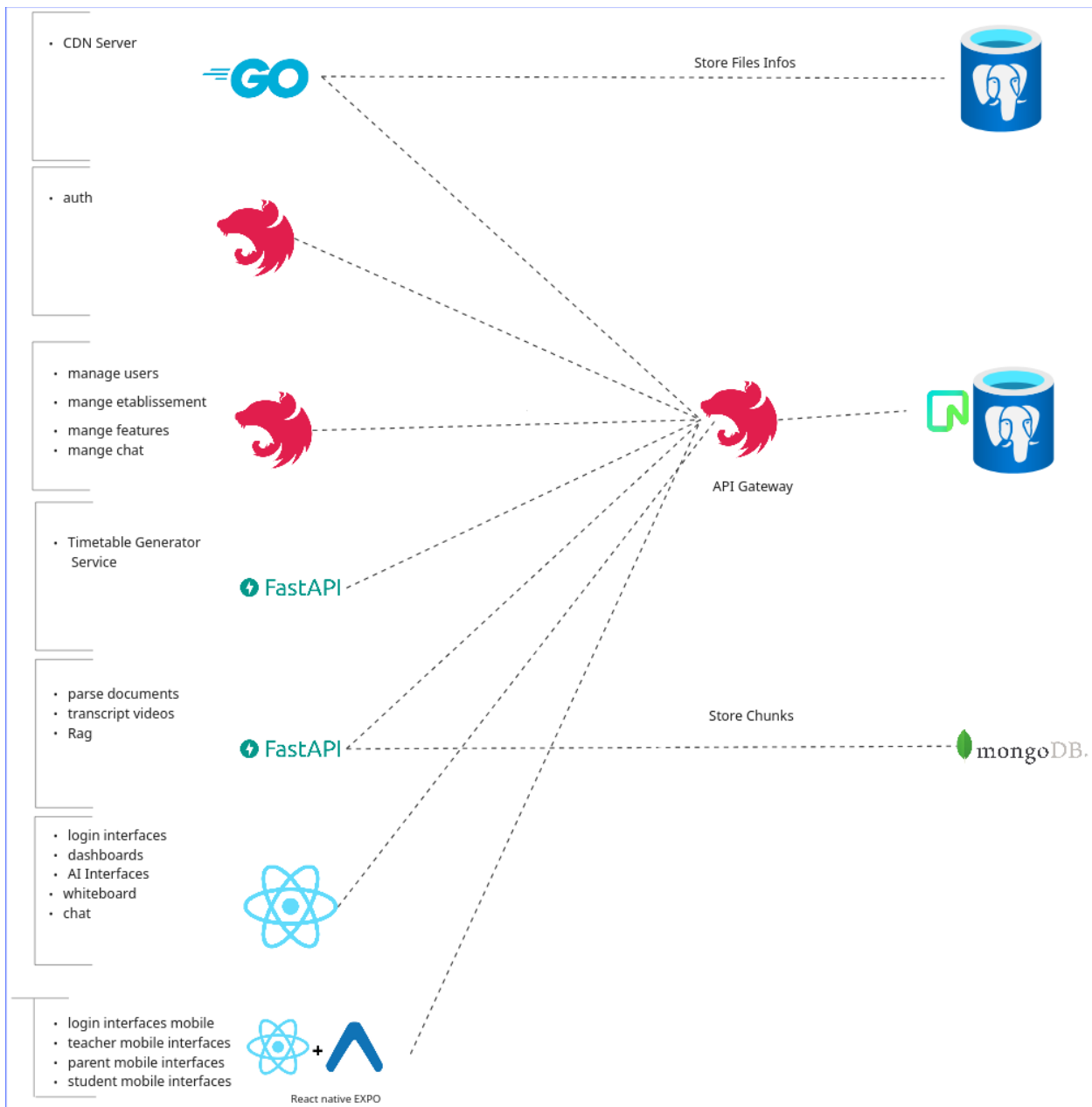


Figure 3. 32: General solution architecture

To support scalability and individualized access for different institutions, we adopted a multi-tenancy architecture that allows each school to have its own customized website under the same core platform. This approach enables multiple schools to coexist within a single application instance, while isolating their data, user base, branding, and timetable configurations.

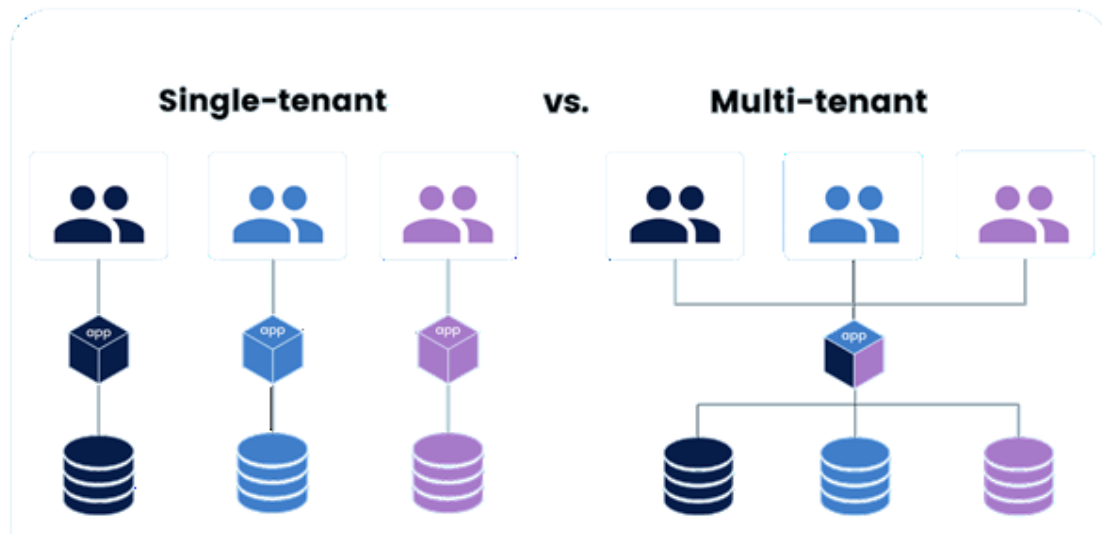


Figure 3. 33: Multi-tenant model

Upon creating a new school through the super admin interface, the system automatically:

- Registers the school entity and its metadata (name, logo, URL slug, color theme, etc.),
- Generates a dedicated URL (schoolname.jeridschool.tech),
- Applies a selected theme for the front-end,
- Sets up an isolated data context, ensuring full data separation across tenants.

The backend handles this by dynamically resolving tenant contexts via subdomains or URL prefixes, and routes each request through a middleware layer that loads the appropriate configuration and database scope. Each tenant accesses a fully functional, branded interface with features such as timetable management, attendance tracking, and communication tools all while being securely isolated from other institutions.

This strategy ensures:

- Efficient resource usage, as all tenants share the same codebase,
- Low maintenance overhead,
- And rapid deployment of new school instances.

Conclusion

This chapter concerns the development methodology adopted and the work organization and planning of the project. The next chapter is devoted to the project's implementation phase.

Chapter 4: Implementation and validation tests

Introduction

In this final chapter, we describe the working environment and the technologies chosen for the implementation of the system. We showcase the key features developed and illustrate the result through screenshots of the application. This chapter also covers the testing process carried out to ensure the reliability, functionality, and usability of the application. It includes user feedback, outlines future improvements, and reflects on the technical and organizational challenges faced during the implementation phase.

1. Application Screenshots & Key Features

For the realization of our application, we used a computer with the following characteristics:



- ✓ **OS:** Arch Linux
- ✓ **Processor:** Intel i7 8500 2.1GHz.
- ✓ **RAM:** 16 GB.
- ✓ **Hard disk:** 512Go SSD.

The platform is already live and accessible online at the following address:
<https://www.jeridschool.tech>.

It has been deployed using cloud-based infrastructure, ensuring high availability, performance, and scalability. The platform supports **both English and French**, making it accessible to a wider audience. Its public availability enables real-world testing and continuous user feedback, which contribute directly to its ongoing improvement and evolution.

The following figure shows the deployment diagram of our project.

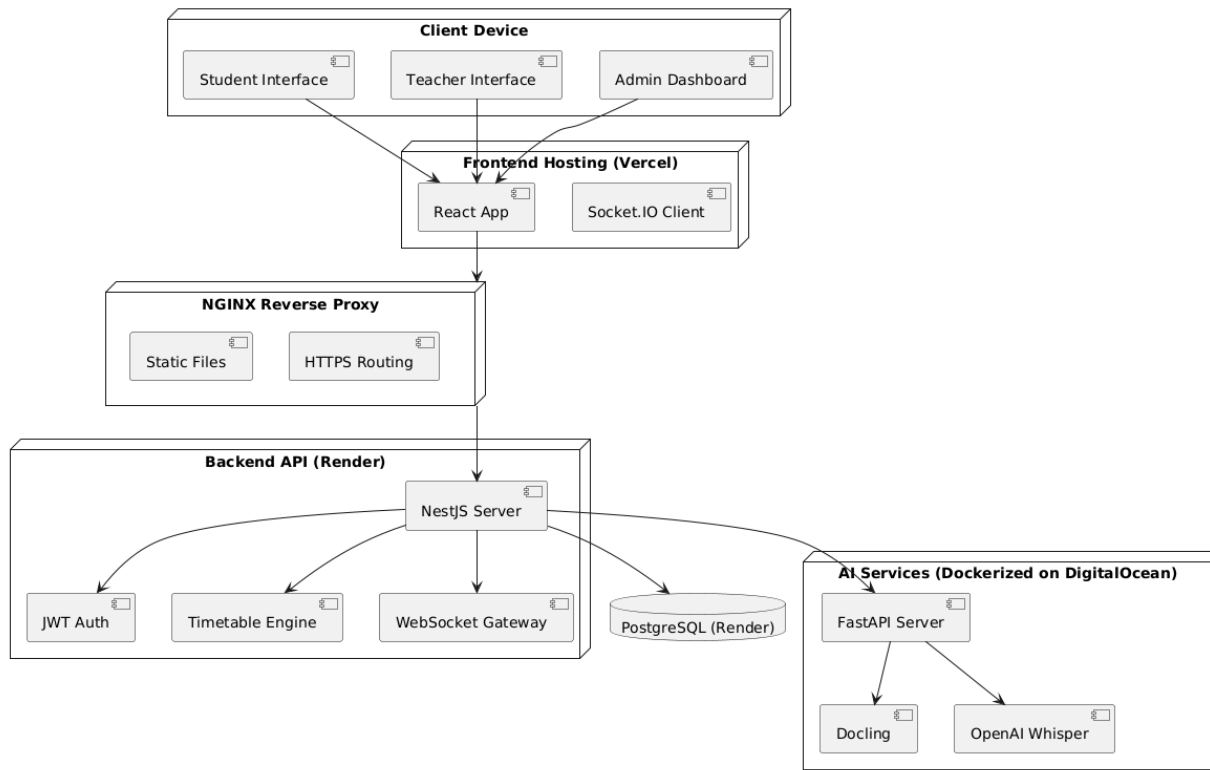


Figure 4. 1: Deployment diagram

1.1. Navigation Flow & Pages management

A clear and logical navigation flow is essential for guiding users through the application efficiently. This section presents the structure and interconnection of different pages beginning from the homepage and branching into role-specific dashboards. The platform is structured to begin with a public homepage, from which users can access authentication pages (login/register). Once logged in, users are redirected to role-specific dashboards tailored to their account type (Super Admin, Headmaster, Teacher, Student, or Parent). Each dashboard provides access to core features relevant to the user's role. This hierarchical and role-based page organization ensures a smooth user experience and intuitive system navigation.

1.2. Graphic Charter (UI Guidelines)

To ensure a consistent and user-friendly experience across the entire application, a comprehensive graphic charter was defined at the beginning of the design phase. This charter outlines the visual identity of the platform, including color schemes, typography, spacing, and iconography. These design elements were carefully selected to reflect modern educational technology standards while ensuring readability and accessibility across devices.

1.2.1. Logo and Branding

As for name, we chose “JeridSchool”. And our logo is shown below. The logo of Jerid School symbolizes digital transformation in education. It is always placed in the top-left corner and links to the homepage. Consistent branding is maintained across web and mobile platforms.



Figure 4. 2: Solution logo

The visual identity of Jerid School was designed to be modern, minimalistic, and intuitive, in order to create a pleasant and consistent user experience across all devices and user roles.

1.2.2. Color Palette

The color scheme is based on soft and professional tones that reflect trust and clarity. The main palette includes:

- ▶ **Primary Color:** #1F487E (Deep Blue) – used for headers, action buttons, and highlights.
- ▶ **Secondary Color:** #F2F6FC (Light Greyish Blue) – used for backgrounds and contrast.
- ▶ **Accent Color:** #FFB703 (Golden Yellow) – used for alerts, notifications, and highlights.

This combination ensures readability, visual harmony, and accessibility.

1.2.3. Typography

The platform uses the '**Inter**' font family for its clean, modern look and excellent legibility on screens. Font weights are used as follows:

- **Bold** for titles and key actions,
- **Medium/Regular** for body text and labels.

1.2.4. Iconography

Icons are used throughout the application to support navigation and actions. The design follows a **line-style** (Lucide/Feather icons) for simplicity and coherence.

1.2.5. Layout and Spacing

All pages follow a grid-based layout with a consistent header, sidebar/menu, and content area. Margins and paddings are applied generously to avoid visual clutter and improve focus.

1.2.6. Responsive Design

The platform is fully responsive and adapts seamlessly to desktop, tablet, and mobile devices. Layout elements are repositioned or resized to maintain usability on all screens.

1.2.7. Consistency Across Roles

Although each user profile (student, teacher, admin) has specific functionalities, the overall visual structure remains consistent. This ensures that users can navigate easily when switching devices or roles.

1.3. Common interfaces

Some pages and features are universally accessible regardless of the user's role. These shared components, such as the login page, user profile, and messaging system, provide core functionalities and maintain design consistency throughout the platform. This section introduces these common pages and highlights their importance in creating a unified user experience.

1.3.1. Homepage

The homepage serves as the main entry point to the platform and provides visitors with a clear overview of the solution's purpose and key features. It includes a descriptive introduction to the platform, outlining how it helps streamline school timetable management and enhance communication between stakeholders.

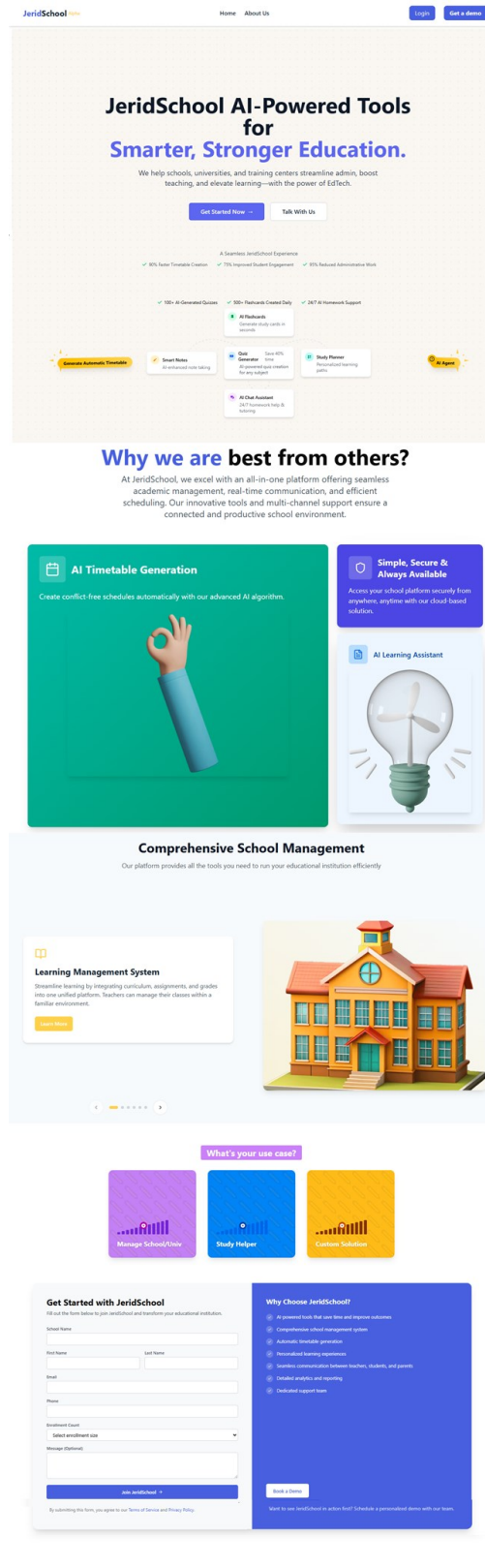


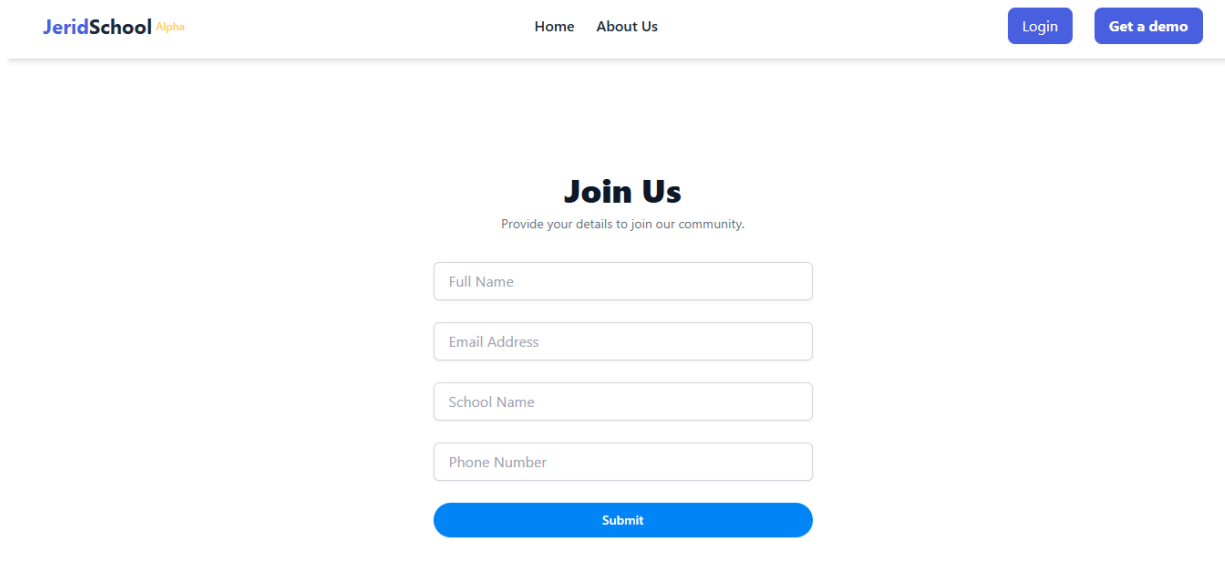
Figure 4. 3: Home Page

1.3.1. Navigation Bar & Side Menu

The landing page of the platform provides a general overview of the product, including its core functionalities, value proposition, and call-to-action buttons (e.g., Sign Up / Log In). It introduces the project to new visitors and invites them to explore or create an account.

A top navigation menu is present, offering quick access to essential sections such as:

- **Home:** Redirects users back to the main landing page.
- **About Us:** Provides information about the development team, project vision, and the story behind the platform.
- **Login:** Redirects users to the “Sign Up / Log In” interface
- **Get demo/contact:** redirects user to a contact page



The screenshot shows the top navigation bar of the JeridSchool Alpha platform. The navigation bar includes the logo "JeridSchool Alpha" on the left, and links for "Home" and "About Us" in the center. On the right, there are two blue buttons: "Login" and "Get a demo". Below the navigation bar is a horizontal line. The main content area features a "Join Us" heading, followed by the text "Provide your details to join our community." Below this text are four input fields: "Full Name", "Email Address", "School Name", and "Phone Number". At the bottom of the form is a blue "Submit" button.

Figure 4. 4: Contact Interface

Each user, once logged in, is presented with a navigation structure that provides access to their role-specific features. However, some elements such as "Home," "Notifications," and "Messages" are common across all user types.

1.3.2. The footer

The footer appears at the bottom of every page on the platform and provides users with essential links and quick access to important resources. It includes:

- **Contact Information:** Email address and/or phone number for support or inquiries.
- **Social Media Links:** Icons linking to the platform's official pages on platforms such as LinkedIn, Facebook, or Twitter (if applicable).
- **Legal and Policy Pages:** Links to the Privacy Policy, Terms of Service, and Cookies Policy to ensure transparency and compliance.
- **Navigation Shortcuts:** Quick links to key sections such as Home, About Us, and Login for easy navigation.

The footer enhances user experience by making key information easily accessible from anywhere on the site and reinforces the professionalism of the platform.

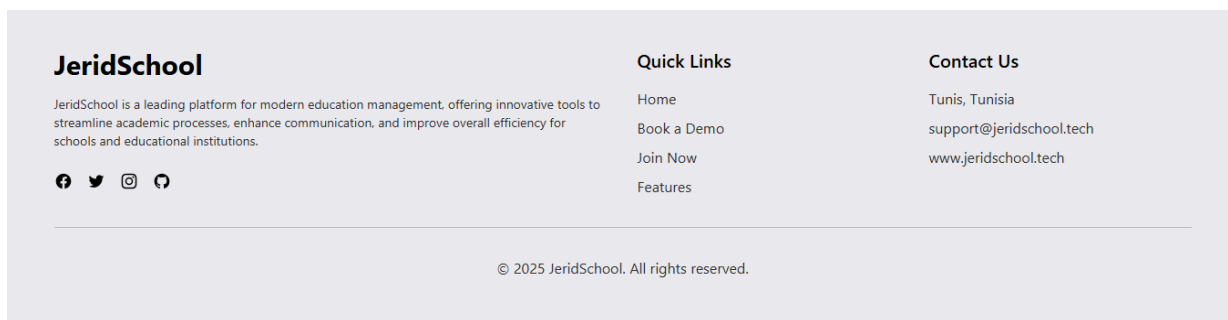


Figure 4. 5: Footer

1.3.3. Authentication Pages (Login / Registration)

All users pass through these interfaces. The login and registration pages are designed with user-friendliness in mind, with validation, password reset, and role selection integrated into the flow.

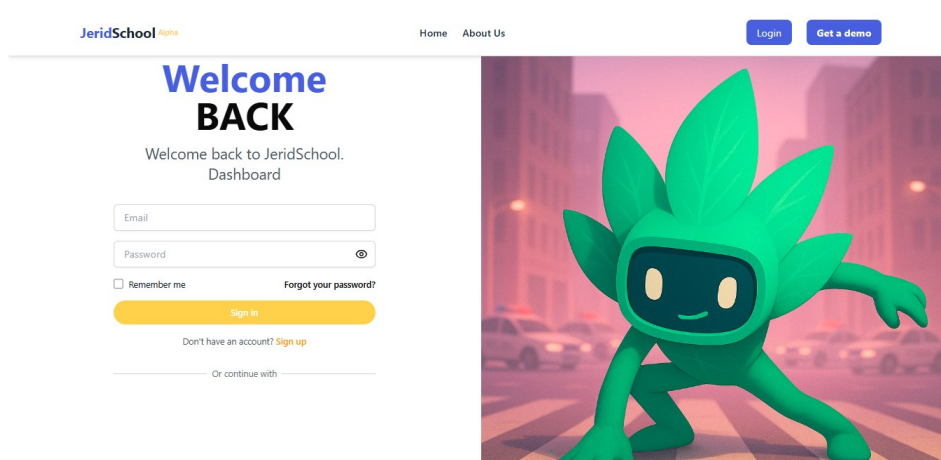


Figure 4. 6: Authentication page

If the user does not have an account, he can click on the "Sign up" link and will be redirected to the registration page. After filling in his information correctly in the "sign up" form, he will have an account.

The figure displays two sequential steps of a user registration form titled "Create ACCOUNT".

Step 1 of 2: Basic Information

- Fields: First Name, Last Name, Email, Password, Confirm Password.
- Buttons: Next Step, Back to Login.

Step 2 of 2: Additional Details

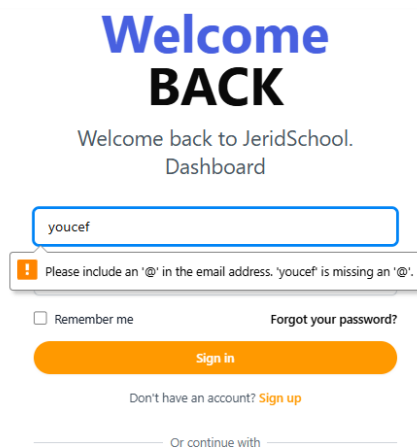
- Fields: Gender (dropdown), Birthday (date picker), Address, Phone, School Name, Enrolled Date (date picker), Profile Picture (upload area with "Change Image" button).
- Buttons: Previous, Register, Back to Login.

Figure 4. 7: Sign up Interface

If the user already has an account, he must correctly fill out the "login" form. During the authentication process, users are required to enter their login credentials, specifically their email address and password. The system performs validation to ensure data correctness:

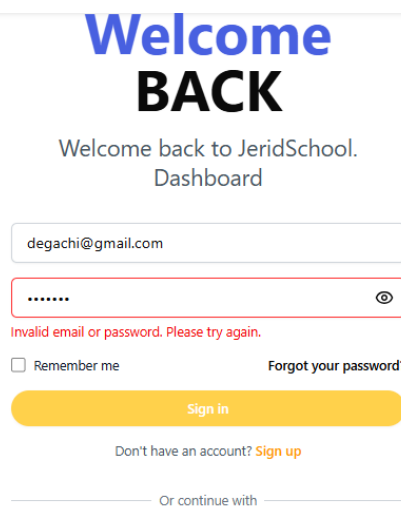
- **Email Format Validation:** The email input must contain the “@” symbol. If the user enters an email without this symbol, an error message will be displayed prompting them to enter a valid email address.
- **Credential Verification:** If the email or password is incorrect, the system will show an error message indicating that the login attempt failed due to invalid credentials.

This validation helps prevent common input mistakes and improves the overall user experience by providing immediate feedback.



The screenshot shows a login page titled "Welcome BACK" with the subtitle "Welcome back to JeridSchool. Dashboard". The email input field contains the text "youcef". Below the input field, a red error message is displayed: "Please include an '@' in the email address. 'youcef' is missing an '@'." Below the error message, there is a checkbox labeled "Remember me" and a link labeled "Forgot your password?". A blue "Sign in" button is positioned below these elements. At the bottom, there is a link that says "Don't have an account? Sign up" and a line of text that says "Or continue with".

Figure 4. 8: Email format validation



The screenshot shows a login page titled "Welcome BACK" with the subtitle "Welcome back to JeridSchool. Dashboard". The email input field contains the text "degachi@gmail.com". The password input field contains six dots. Below the password field, a red error message is displayed: "Invalid email or password. Please try again." Below the error message, there is a checkbox labeled "Remember me" and a link labeled "Forgot your password?". A blue "Sign in" button is positioned below these elements. At the bottom, there is a link that says "Don't have an account? Sign up" and a line of text that says "Or continue with".

Figure 4. 9: Credential Verification

1.3.4. Settings and Profile Management

Every user can update their personal information, change their password, and customize notification settings through the profile settings page.

1.4. Super admin profile

The Super Admin interface provides comprehensive control and oversight over the entire platform. This profile is designed for users who manage high-level system configurations, user roles, and permissions. The Super Admin can monitor system activity, manage institutional settings, and ensure the platform operates smoothly across multiple schools or branches. This interface prioritizes security, scalability, and administrative efficiency.

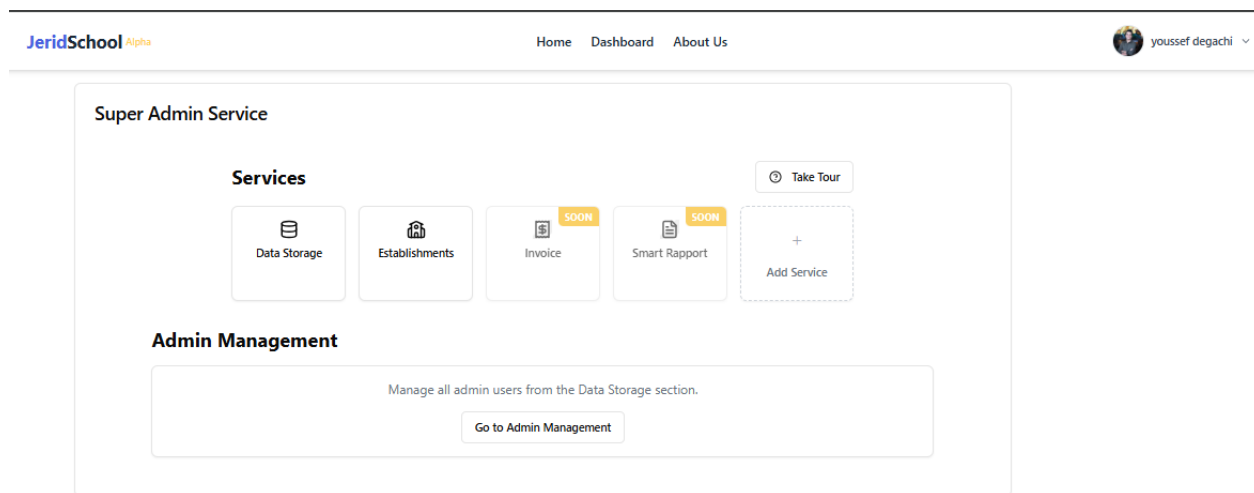


Figure 4. 10: Super admin Dashboard

From its interface, the super admin can manage the list of administrators and the list of schools.

1.4.1. Database Overview

The "Database Overview" interface allows you to display the number of students and teachers registered per school and the quota of space reserved for each school's site.

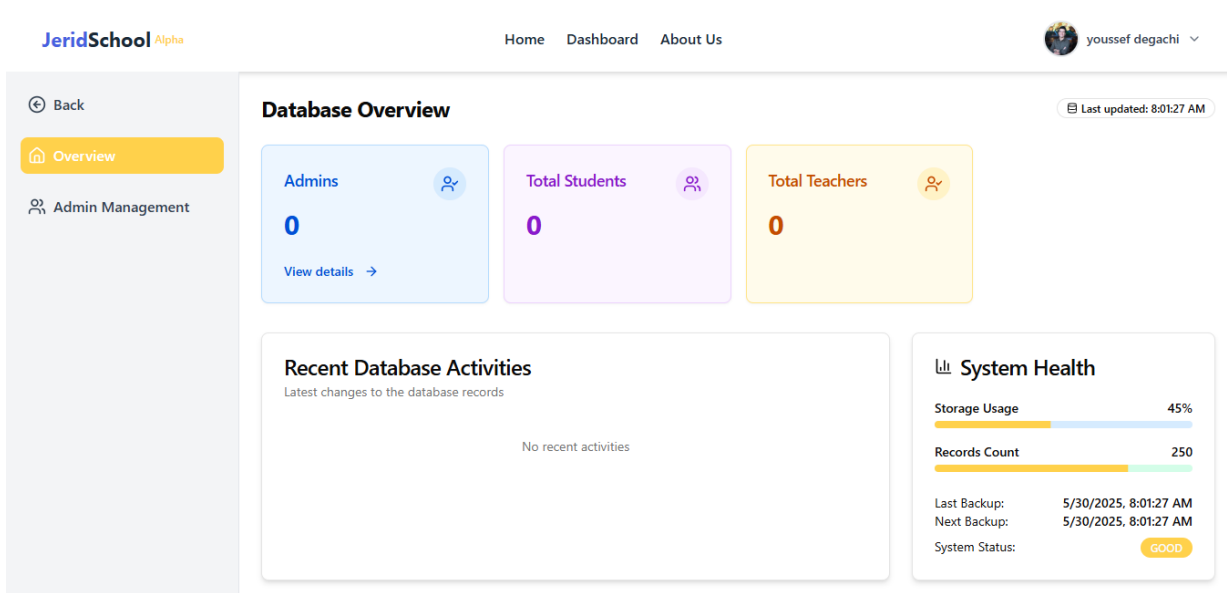


Figure 4. 11: The "Database Overview" interface

1.4.2. Admin management

The Administrator Management page allows the Super Admin to view and manage all registered school administrators. A structured table displays a list of current administrators, showing key details such as name, email, role, and account status.

The super administrator can manage the list of administrators by adding a new administrator, modifying or deleting an existing administrator.

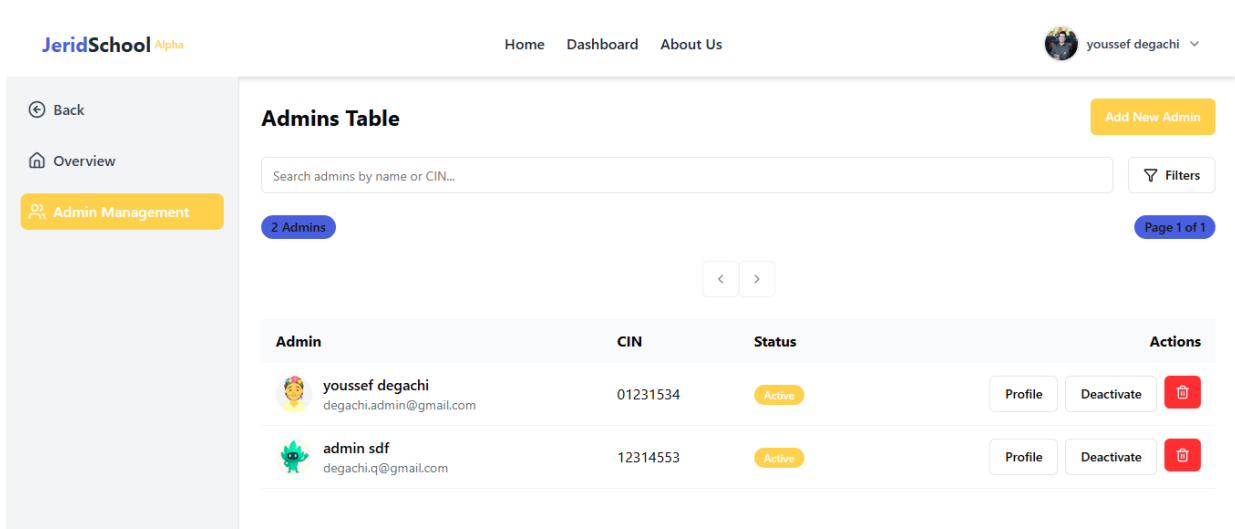


Figure 4. 12: User management

To add a new admin, just click on the link “Add new Admin”.

The following form will appear. The super admin will need to fill it out carefully. After confirmation, a notification message will be displayed on the screen indicating that the new admin has been successfully added, and the new admin will be added to the list.

A yellow rectangular button with the text "Add New Admin" in white, bold, sans-serif font.

Figure 4. 13: "Add New Admin" button

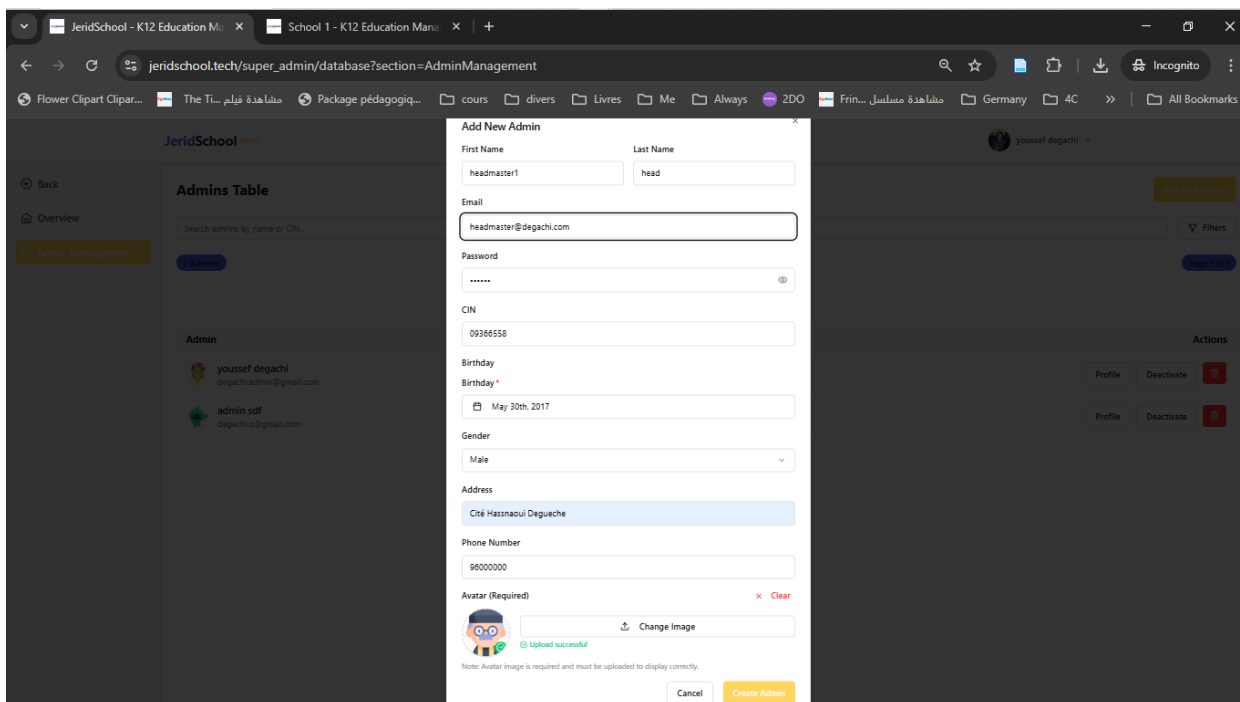
A screenshot of a web browser showing the "Add New Admin" form. The form is a white modal box with a title bar. It contains several input fields: "First Name" (filled with "headmaster1"), "Last Name" (filled with "head"), "Email" (filled with "headmaster@degachi.com"), "Password" (masked with dots), "CIN" (filled with "09366558"), "Birthday" (filled with "May 30th, 2017"), "Gender" (a dropdown menu showing "Male"), "Address" (filled with "Cité Hassnaoui Deguache"), "Phone Number" (filled with "96000000"), and "Avatar (Required)" (with a "Change Image" button and a "Clear" link). Below the form fields, there is a note: "Note: Avatar image is required and must be uploaded to display correctly." At the bottom of the form are "Cancel" and "Create Admin" buttons. In the background, a sidebar shows a list of admins: "youssef degachi" and "admin sdf".

Figure 4. 14: "Add new Admin" form

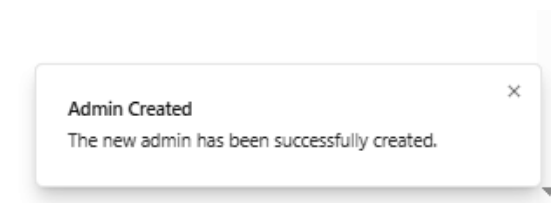


Figure 4. 15: Notification message

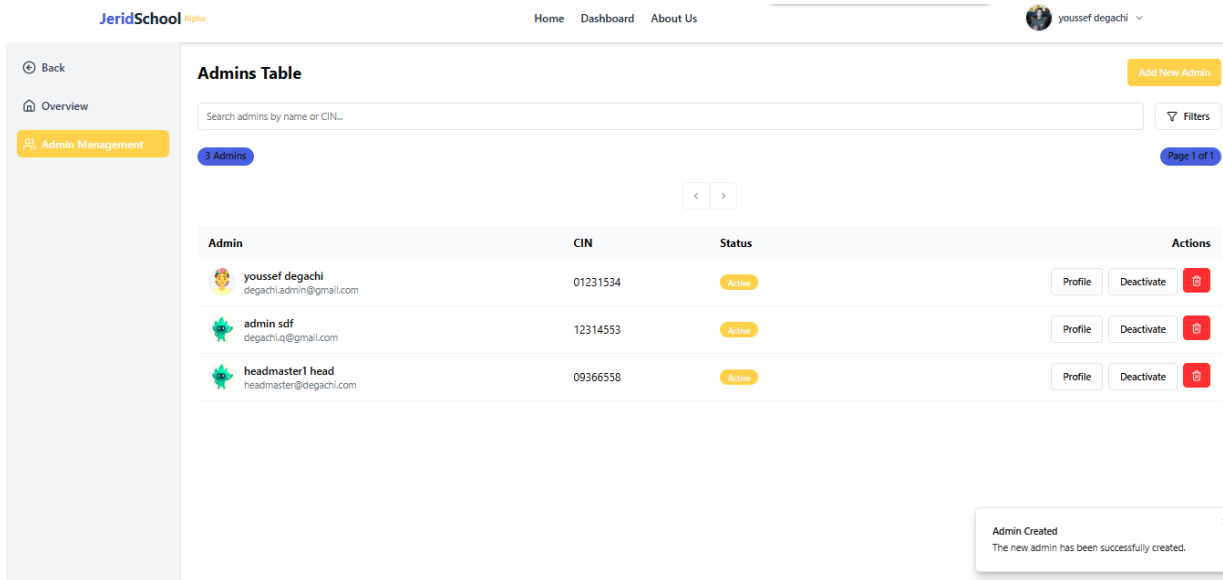


Figure 4. 16: Admin list updated

In front of each administrator, three essential action buttons are available:

- Profile/Edit: Opens a form that allows the user to modify the administrator's information (e.g., name, email, assigned permissions).
- Deactivate: Temporarily disables the administrator's account without permanently deleting their data. This is useful for managing inactive users or restricting access during a specific period.
- Delete: Permanently removes the administrator's account from the system, along with all associated access rights.

If we want to deactivate an administrator, we simply click on the **"Deactivate"** link next to their name. Once clicked, the link changes to **"Activate"**, and the administrator is visually marked as deactivated in the list (inactive red label). This allows administrators to be temporarily disabled without removing them permanently from the system.



Figure 4. 17: Activate/desactivate button

To delete an administrator, we have simply to click on the "Delete" link next to their name. A confirmation dialog will appear, asking the user to confirm the action. Once confirmed, a notification message is displayed indicating that the deletion was successful, and the administrator is permanently removed from the list.



Figure 4. 18: delete button

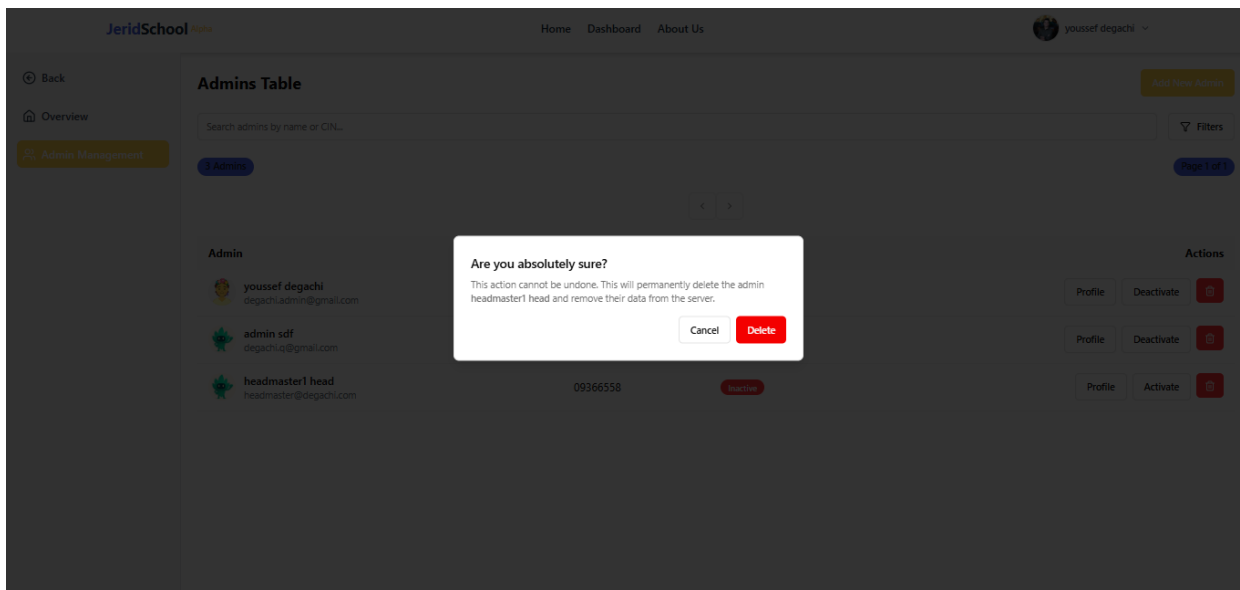


Figure 4. 19: Confirmation dialog for deleting a user

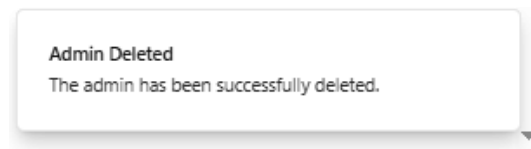


Figure 4. 20: Notification message

1.4.3. Schools management

The School Management page allows the Super Admin to add, edit, and delete schools from the system. When creating a new school, in addition to entering basic information, the Super Admin can select a custom theme for the school's website and define a unique subdomain URL (schoolname.jeridschool.tech). Once the form is submitted, the system automatically generates and

deploys the school's personalized website with the selected visual theme, allowing instant access for administrators, teachers, and students.

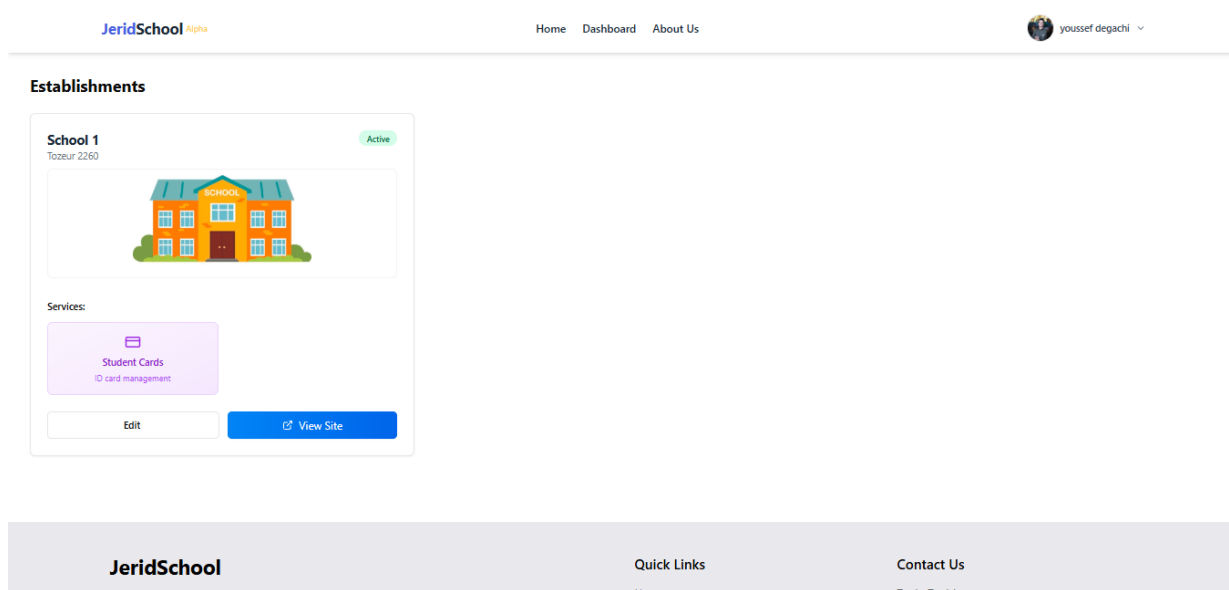


Figure 4. 21: School management interface

JeridSchool Alpha
Home Dashboard About Us
youssef degachi

+ Edit Establishment

Edit Establishment

Update the details for School 1

General Student Cards

Name
School 1

Address
Tobeur 2260

Logo



X Clear

Change Image

URL Slug
degachischool

Description

Enter establishment description

CMS Content
Select CMS Template

Programs Template
Our educational programs and curriculum

Custom Template
Create your own custom content

Enter your custom HTML content here

Hero Image


Click to browse

Select Image

Changes to gallery images will only be saved when you submit the form.

Gallery Images

+ Add Image

No images added yet

Add your first image

Contact Email
youssefdegachi@gmail.com
Contact Phone

Enter contact phone

Changes to social links will only be saved when you submit the form.

Social Media Links

Facebook URL

Twitter URL

Instagram URL

LinkedIn URL

School Statistics

Students
0

Teachers
0

Courses
0

Awards
0

Services

Add a service

Active Status

Cancel Update Establishment

Figure 4. 22: School editing interface

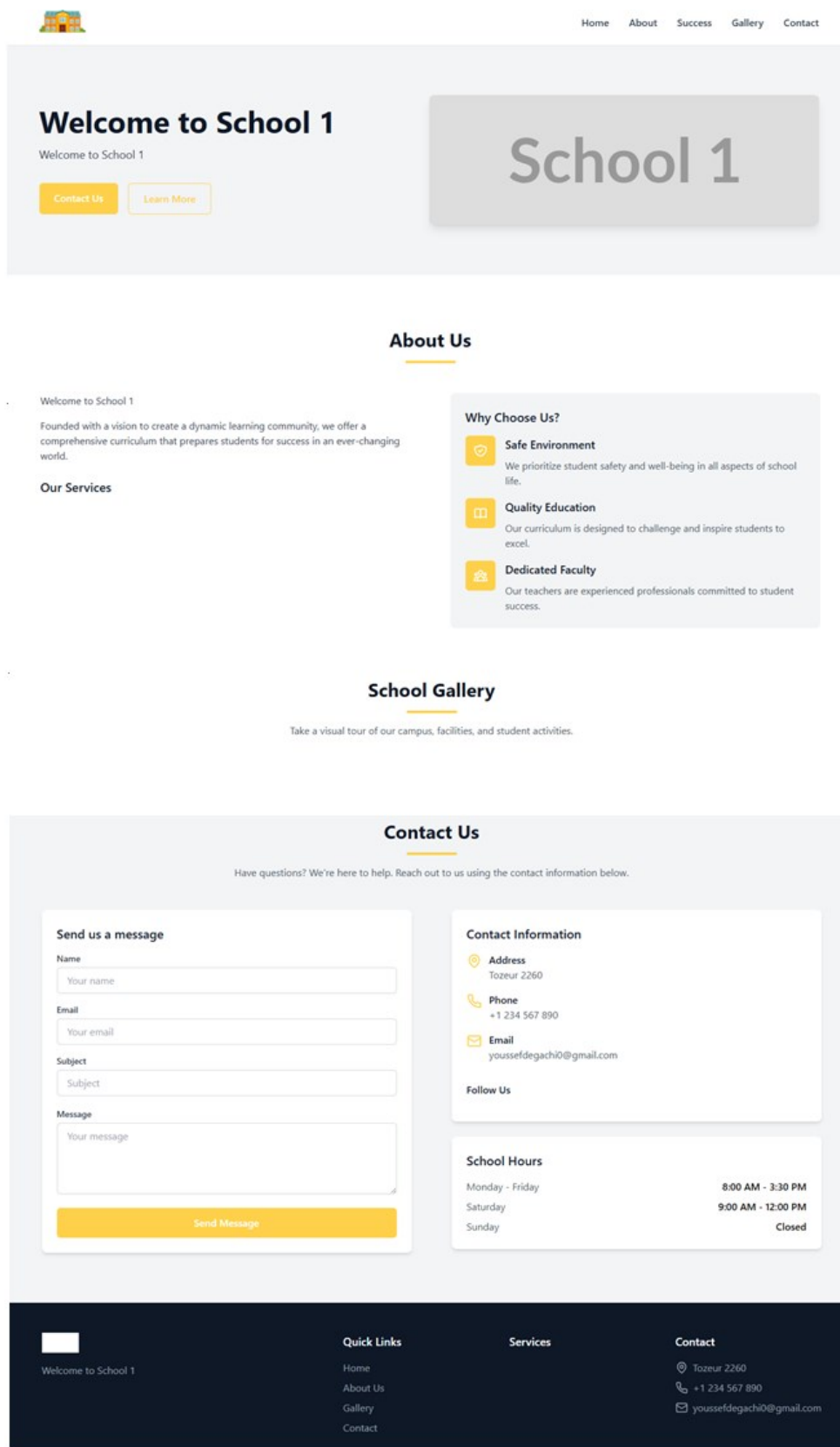


Figure 4. 23: School website created automatically

1.4.4. Profile management

By clicking on the administrator's profile photo, a dropdown menu appears, allowing the user to manage their profile, change their password or log out of the session.

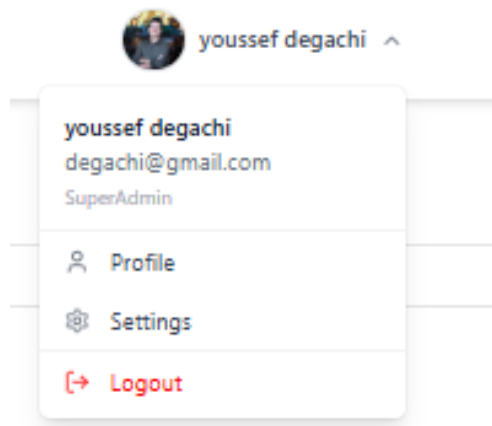


Figure 4. 24: Profile dropdown menu

A screenshot of the 'Profile' management interface. The page has a header with 'JeridSchool Alpha' on the left, navigation links 'Home', 'Dashboard', and 'About Us' in the center, and a user profile 'youssef degachi' with a dropdown arrow on the right. The main content area is titled 'Profile' and contains a form with the following fields: 'First Name' (youssef), 'Last Name' (degachi), 'Email' (degachi@gmail.com), 'CIN' (098761234), 'Birthday' (February 6th, 1995), 'Gender' (Male), 'Address' (Enter your address), 'Phone Number' (Enter your phone number), and 'New Password (Optional)' (Enter new password). An 'Update Profile' button is located at the bottom right of the form.

Figure 4. 25: Profile management interface

JeridSchool Alpha Home Dashboard About Us yousef degachi

Settings

Profile **Security** Preferences

Change Password

Current Password

New Password

Confirm New Password

Update Password

Two-Factor Authentication

Enhance your account security by enabling two-factor authentication.

Enable 2FA

Figure 4. 26: change Password interface

1.5. Headmaster/admin profile

The Head Master interface is tailored to meet the specific needs of school principals and administrators responsible for daily operations. This profile enables efficient management of school schedules, staff assignments, and student information. It offers tools to oversee timetable creation, handle exceptional cases such as classes, and courses and facilitate communication between teachers, students, and parents. The Head Master interface focuses on practicality and streamlined school management.

JeridSchool Alpha Home Dashboard About Us youcef degachi

Admin Service

Services

Take Tour

Data Storage

TimeTable

Elimination

Score

Quick Stats

Quick Timetable View

View the current timetable by class, teacher, or room **Delete**

Class: Teacher Room

Class 1 A

Time	Monday	Tuesday	Wednesday	Thursday	Friday
8h30 - 10h00	-	-	-	-	math teacher math
10h15 - 11h30	-	-	-	-	math

Restart Tour

Figure 4. 27: Headmaster Dashboard

1.5.1. Database Overview

From the headmaster's dashboard, the user gains centralized access to the Database Overview, which provides a comprehensive summary and management interface for key academic resources. Through this section, the headmaster can easily manage the lists of students, teachers, parents, subjects, classes, and educational levels.

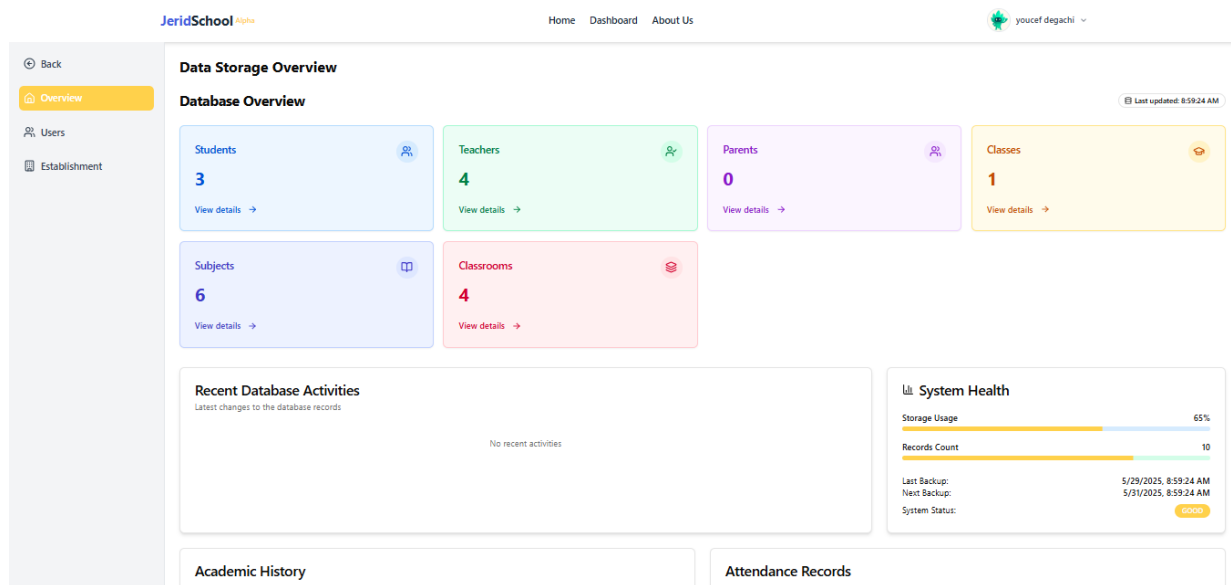


Figure 4. 28: Database Overview interface

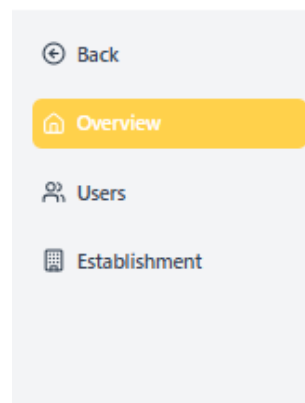


Figure 4. 29: Database Overview Menu

This centralized dashboard ensures that the headmaster has full control over school operations and can streamline academic planning with ease.

The administrator can thus manage the list of users, namely teachers, students and participants.

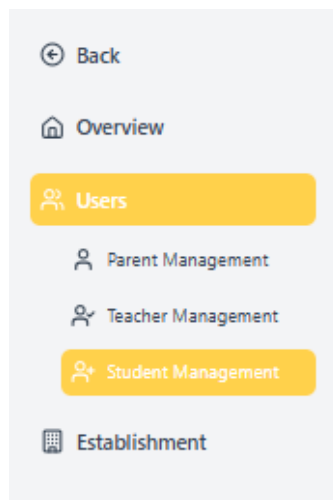


Figure 4. 30: User management submenu

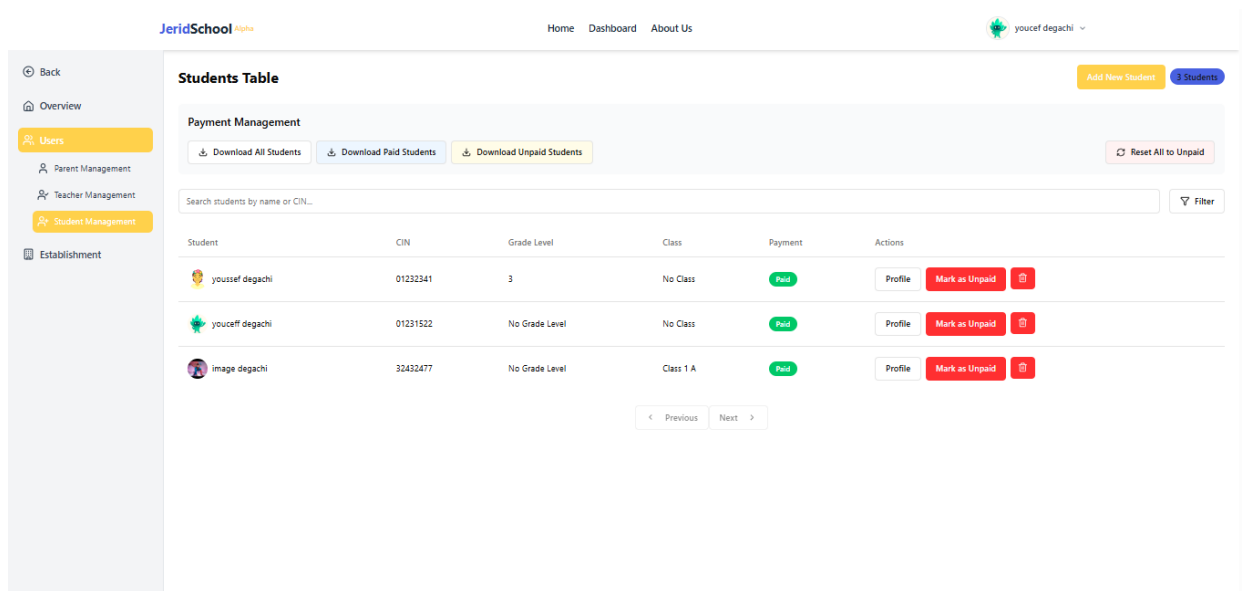


Figure 4. 31: Students management interface

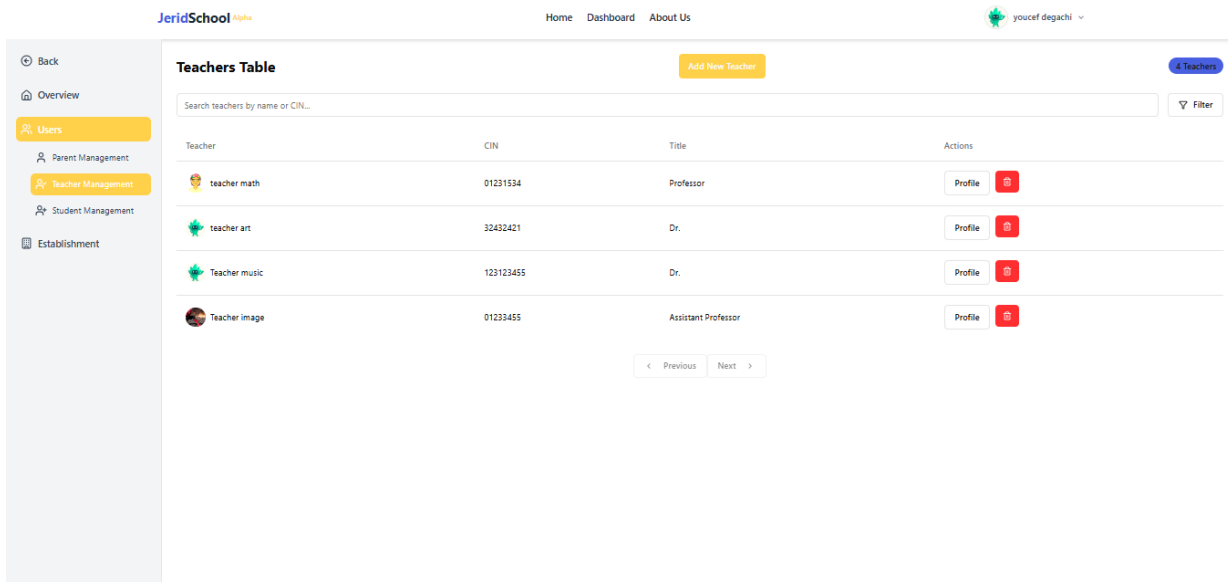


Figure 4. 32: Teachers management interface

The Establishment Management section allows the headmaster to define and organize the academic structure of the institution. This includes:

- Grades/ Class Levels: Manage different academic levels such as Grade 1, Grade 2, etc. The platform allows adding, editing, or deleting levels according to the school's curriculum.
- Classes: The headmaster can add, edit, or delete classes. This section plays a crucial role in structuring the academic organization of the school, as it directly connects to timetable creation, student assignment, and teacher responsibilities.
- Classrooms: Define and manage physical rooms where classes are held. Each classroom can be identified by a unique name or code and assigned to specific levels or subjects.
- Subjects: Add and configure the subjects taught at the school. Each subject can be linked to specific levels and assigned to available teachers.

This section ensures that the foundational components of the school are correctly set up before generating timetables or assigning teachers and students. Proper configuration here is essential for the platform's automation features to function effectively.

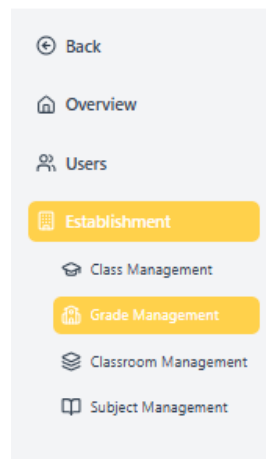


Figure 4. 33: Establishment management submenu

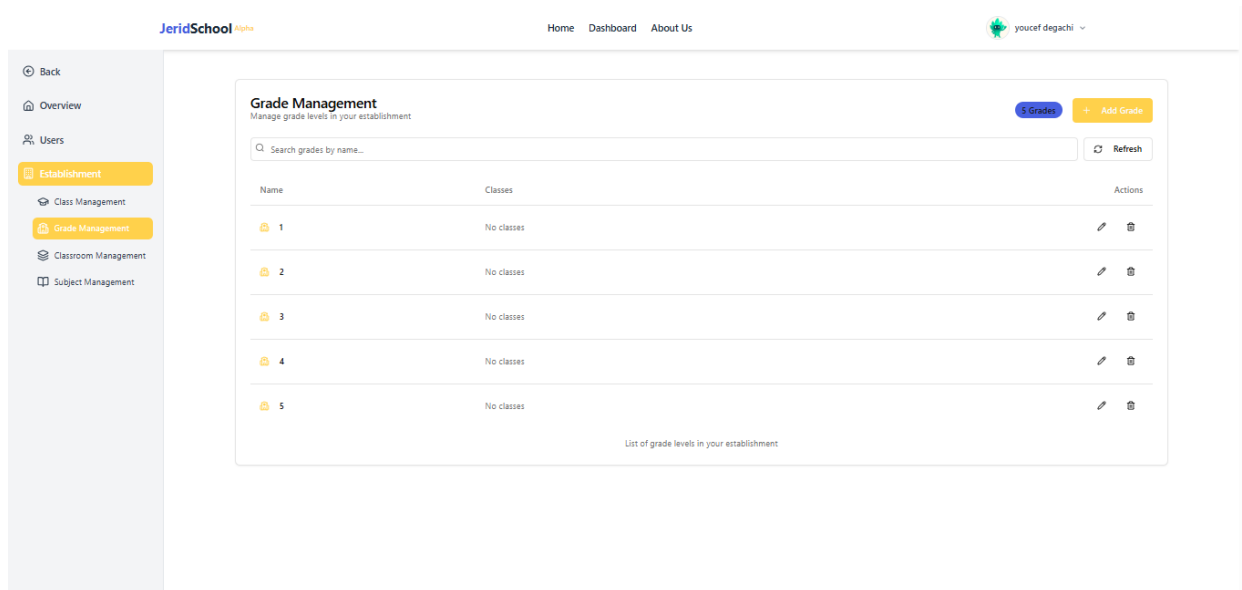


Figure 4. 34: Grade management interface

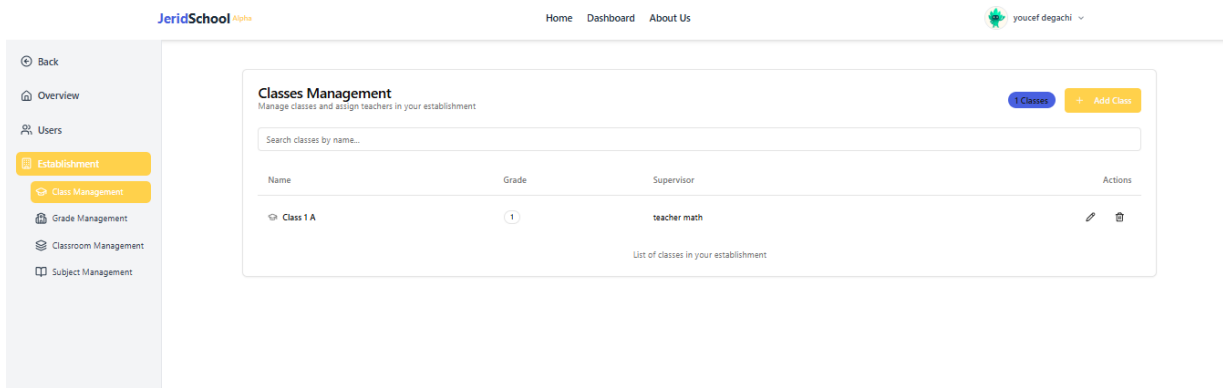


Figure 4. 35: Classes management interface

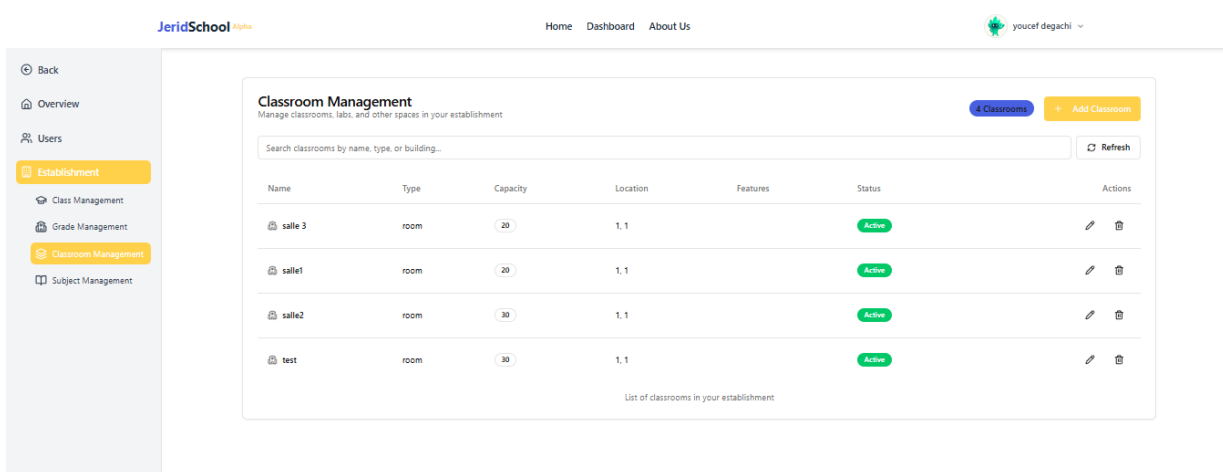


Figure 4. 36: Classrooms management interface

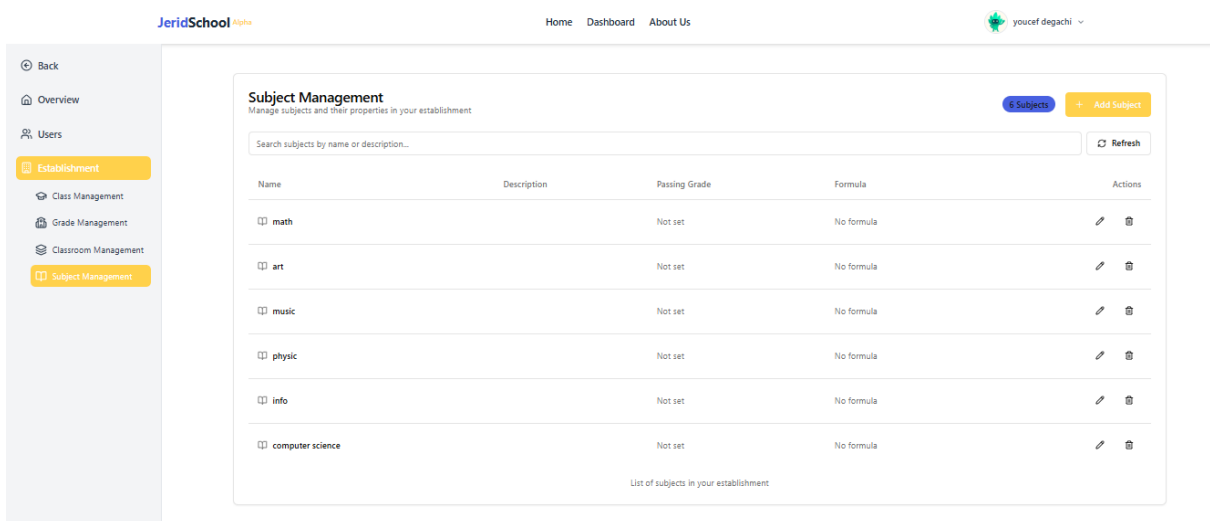


Figure 4. 37: Subjects management interface

1.5.2. Timetable module

In addition to data management, the headmaster can navigate to the Timetable module. Once all related forms have been properly filled out (including teachers' availability, class details, and subject constraints), the timetable can be generated automatically using the platform's built-in scheduling engine.

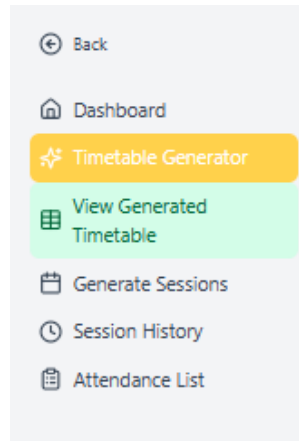


Figure 4. 38: Timetable menu

Before generating a timetable, the user must configure a set of general **settings** that define the structure and rules of the school's weekly schedule. These settings allow for a high degree of customization to adapt to the specific needs of each institution. The main configuration options include:

- **Number of School Days:** Define how many days per week the school operates (e.g., 5 or 6).
- **Timetable Name:** Assign a name to the current timetable for identification and version tracking.
- **School Day Start and End Time:** Set the daily operational hours of the school, which helps in generating realistic and usable time slots.
- **School Days:** Select which days of the week are considered working days (e.g., Monday to Friday).
- **Optimization Constraints:** These include rules such as "**Optimize Free Time**", which aims to minimize gaps between classes for both students and teachers, ensuring a compact and efficient schedule.

These settings serve as the foundation for the timetable generation process and directly influence the quality and accuracy of the resulting schedules. Once configured, the user can proceed to generate the timetable according to the defined parameters.

The screenshot displays the 'General Settings' page for a timetable generator. The sidebar on the left contains navigation links: Back, Dashboard, Timetable Generator (highlighted), View Generated Timetable, Generate Sessions, Session History, and Attendance List. The main content area has a top navigation bar with links: General Settings, Period Timings, Subjects/Courses, Teachers, Classes/Grades, Classrooms, Lessons, and Review & Generate. Below this is a 'Basic Timetable Settings' section. It starts with a 'Getting Started' card. The 'Timetable Name' field is pre-filled with 'e.g., Fall 2023 Timetable'. The 'School Day Start Time' is set to '08:00 AM' and the 'School Day End Time' is '05:00 PM'. Under 'School Days', there are toggle switches for Monday, Tuesday, Wednesday, Thursday, Friday, Saturday, and Sunday. At the bottom, there are three 'Optimization Constraints' with toggle switches: 'Avoid back-to-back lessons of the same subject', 'Optimize free time (minimize gaps)', and 'Balance teacher workloads across days'.

Figure 4. 39: Timetable General Settings

Within the timetable setup, the "Period Timing" section allows administrators to define the structure of each school day by setting up the number and duration of class periods. By default, the system is initialized with 8 periods per day, but users can add or remove periods based on the institution's specific scheduling needs.

Each period can be customized with:

- Start and End Time: Define the exact scheduling of each class session.
- Break Period Flag: A toggle option labeled "This is a break period", which lets the system know that a certain slot should be considered a rather than a teaching period. These periods will be excluded from teaching assignments during timetable generation.

Figure 4. 40: Period Timing configuration

In the timetable configuration process, the Subjects/Courses Configuration section allows the headmaster or administrator to manage the list of subjects taught within the institution. Each subject must be defined with a distinct name, and to enhance visual clarity throughout the scheduling interface, each subject is also assigned a unique color.

Subject Name	Color	Actions
math	#33FF33	
art	#FF3333	
music	#FF3333	
physic	#FF3333	
info	#33FF33	
computer science	#3333FF	

Figure 4. 41: Subjects / courses configuration

With “Teachers Configuration”, the headmaster configure the teachers and their subjects for the timetable

The screenshot shows the 'Teacher Configuration' screen. The left sidebar contains navigation links: Back, Dashboard, Timetable Generator (highlighted), View Generated Timetable, Generate Sessions, Session History, and Attendance List. The top navigation bar includes: General Settings, Period Timings, Subjects/Courses, Teachers (active), Classes/Grades, Classrooms, Lessons, and Review & Generate. The main content area is titled 'Teacher Information' and 'Teachers Configuration'. It includes a form to 'Configure the teachers and their subjects for the timetable' with fields for 'Teacher Name' (placeholder: Enter teacher name (e.g., John Smith)) and 'Subjects' (placeholder: Select subjects this teacher can teach). Below the form is a table with columns 'Teacher Name', 'Subjects', and 'Actions'.

Teacher Name	Subjects	Actions
teacher math	Unknown	[Add] [Edit] [Delete]
teacher art	Unknown	[Add] [Edit] [Delete]
Teacher music	Unknown	[Add] [Edit] [Delete]
Teacher image	Unknown	[Add] [Edit] [Delete]

Figure 4. 42: Teacher configuration

The headmaster must then define the classes or grades that will be included in your timetable

The screenshot shows the 'Classes Configuration' screen. The left sidebar is identical to the previous screen. The top navigation bar is identical, with 'Classes/Grades' now active. The main content area is titled 'Define Classes or Grades' and 'Classes Configuration'. It includes a form to 'Define the classes or grades that will be included in your timetable' with a section for 'Classes/Grades Information' (placeholder: Add all the classes or grades that need to be scheduled in your timetable. Each class will have its own unique schedule.) and a 'Class Name' field (placeholder: e.g., Class 10A or Grade 5). Below the form is a table with columns 'Class Name' and 'Actions'.

Class Name	Actions
Class 1 A	[Delete]

Total classes: 1

Figure 4. 43: Classes Configuration

Then, the admin should add classrooms that will be used for scheduling classes

Figure 4. 44: Classrooms/Salles Configuration

The Lesson Configuration section enables the administrator or headmaster to define the core teaching structure for each class. This step is essential for generating an accurate and conflict-free timetable.

For each class, the administrator can assign subjects to be taught, link each subject to a teacher and define the number of periods each subject should occupy within the timetable cycle. This ensures that each subject receives the appropriate instructional time based on the curriculum.

Figure 4. 45: Lessons configuration

To add a lesson, we can also choose one of these option:

- **Split Classes into Groups:** This option is used when a single class is divided into multiple groups (e.g., Group 1 and Group 2), each with their own subjects and teachers. These groups are scheduled in parallel time slots, allowing differentiated instruction to occur simultaneously. For example use, a science class is split into two groups, each with a different lab instructor.

- **Alternating Subjects:** Some subjects may alternate on a weekly or bi-weekly basis such as Art vs. Music depending on the school's curriculum. This feature allows for defining alternating subjects within the same period slot and ensuring that students receive exposure to both subjects over the course of the timetable cycle.

Figure 4. 46: Alternating subject example

The following figure shows the list of subjects that will be added to our timetable.

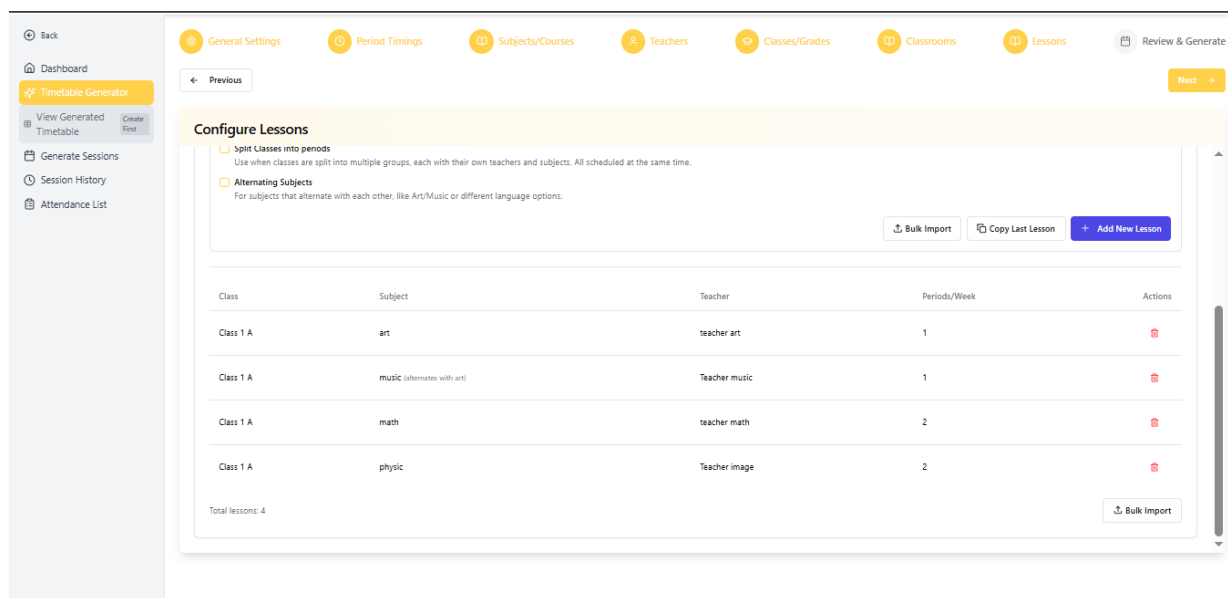


Figure 4. 47: Configure Lessons example

After verifying that all subjects and their associated lessons have been correctly configured, the user can proceed to the next step by clicking on the "Next" link. This leads to the timetable generation section.

By clicking the "Generate" button, the system automatically creates an optimized timetable based on all the defined parameters and constraints (such as teacher availability, subject periods, and classroom limitations). This process may take a few seconds, depending on the complexity of the data.

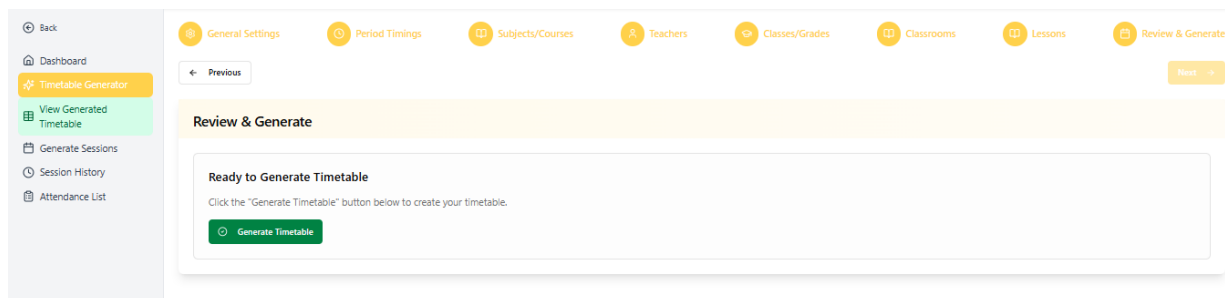


Figure 4. 48: "Generate Timetable" interface

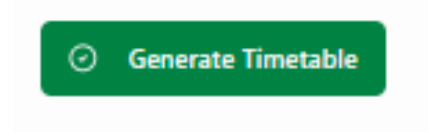


Figure 4. 49: "Generate Timetable" button

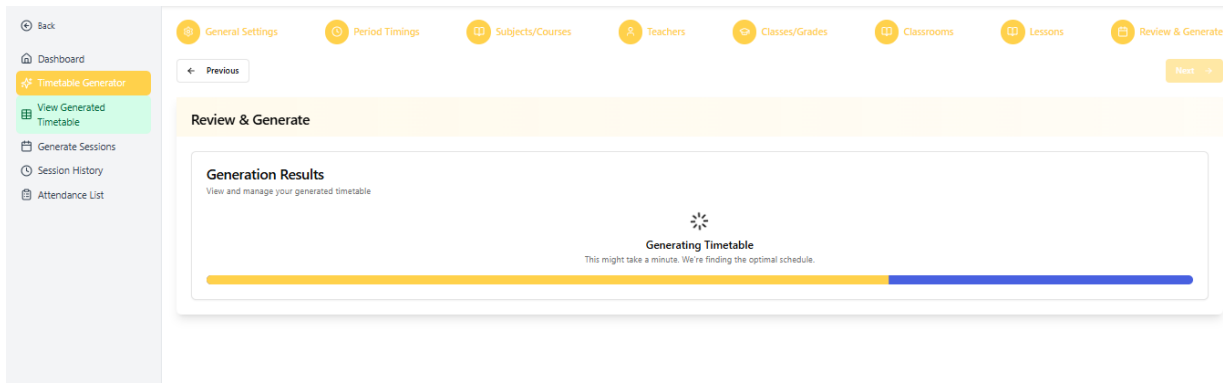


Figure 4. 50: Generating in progress

The screenshot displays the 'Timetable View' interface, which shows a structured timetable grid. The grid is organized by time slots (rows) and days of the week (columns). The time slots range from 8:00 to 15:00. The days of the week are Monday through Sunday. The timetable is color-coded by subject: Physics (light blue), Chemistry (yellow), Biology (light green), History (light orange), Geography (light purple), and English (light pink). Each cell in the grid contains the subject name, the teacher's name, and the room number. For example, on Monday, Physics is taught by A. Physics Teacher in Salle 2 from 8:00 to 9:00. The interface also includes a sidebar with navigation options, a top navigation bar, and a bottom section with buttons for 'Analyze Schedule' and 'Save to Server'. A notification at the bottom right states 'Timetable Saved: The timetable has been saved to the server.'

Time	Monday	Tuesday	Wednesday	Thursday	Friday	Saturday	Sunday
8:00 + 9:00	Physics A. Physics Teacher Salle 2	Free	Physics A. Physics Teacher Salle 2	History A. History Teacher Salle 1	History A. History Teacher Salle 4	English A. English Teacher Salle 2	Free
9:00 + 10:00	Physics A. Physics Teacher Salle 2	Free	Physics A. Physics Teacher Salle 2	History A. History Teacher Salle 1	Geography A. Geography Teacher geo room	Free	Free
10:00 + 11:00	Chemistry A. Chemistry Teacher Salle 3	Free	Geography A. Geography Teacher geo room	Biology A. Biology Teacher Salle 4	Geography A. Geography Teacher geo room	Free	Free
11:00 + 12:00	Chemistry A. Chemistry Teacher Salle 3	Free	Biology A. Biology Teacher Salle 2	Biology A. Biology Teacher Salle 4	Chemistry A. Chemistry Teacher Salle 3	Free	Free
12:00 + 13:00	Free	Free	Biology A. Biology Teacher Salle 2	Free	Chemistry A. Chemistry Teacher Salle 3	Free	Free
14:00 + 15:00	Free	Free	Free	Free	English A. English Teacher Salle 2	Free	Free
15:00 + 16:00	Free	Free	Free	Free	English A. English Teacher Salle 2	Free	Free

Figure 4. 51: Timetable generated

Once the timetable is generated, it is displayed in a structured and interactive format. Users can then perform manual adjustments using a drag-and-drop interface. This allows for fine-tuning the schedule to accommodate last-minute changes or special requests without regenerating the entire timetable.

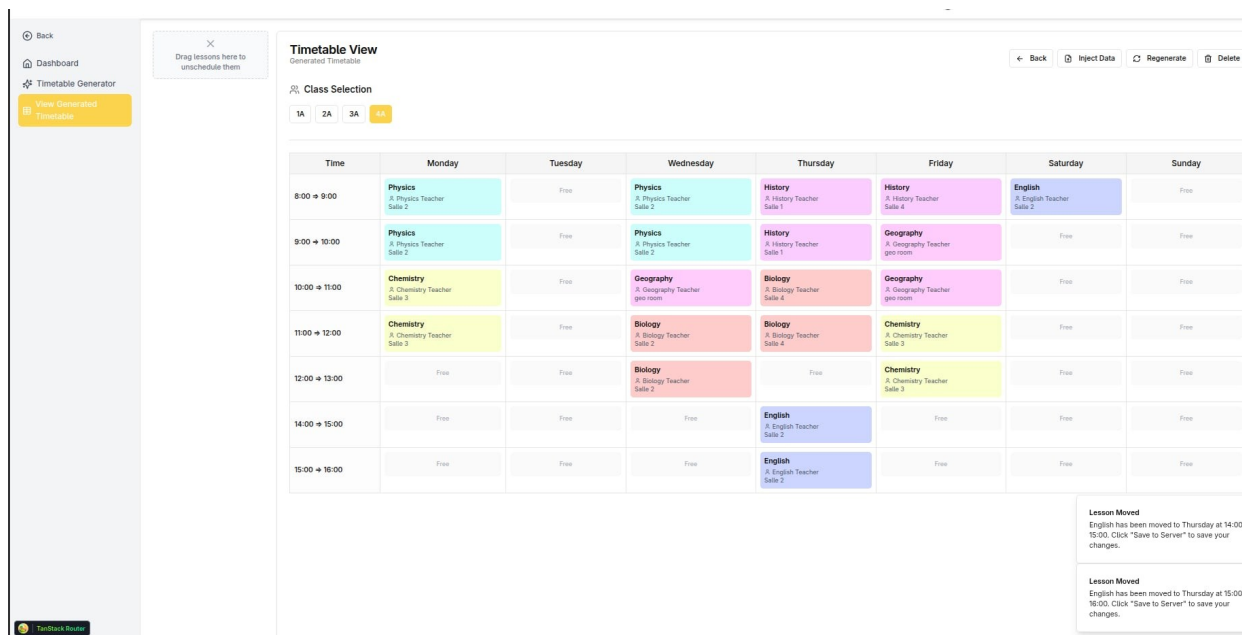


Figure 4. 52: Timetable updated

1.6. Teacher profile

Teachers interact with the platform to access their schedules, communicate with students, and manage digital learning tools. This section focuses on the teacher-facing features, such as creating quizzes, launching interactive sessions, and accessing their teaching materials. The interface prioritizes simplicity and productivity, allowing teachers to focus on instruction rather than navigation.

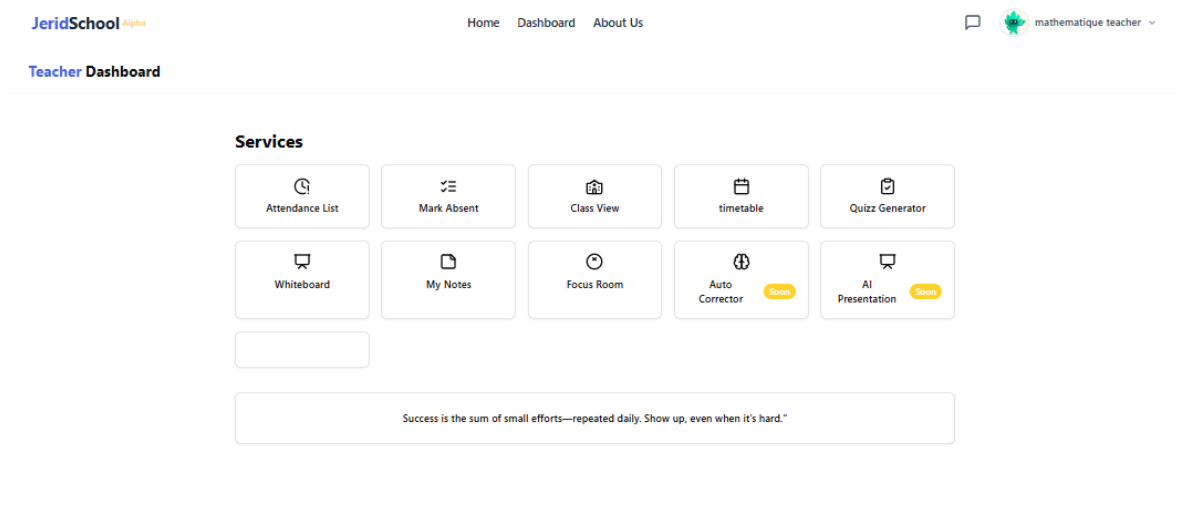


Figure 4. 53: Teacher Dashboard

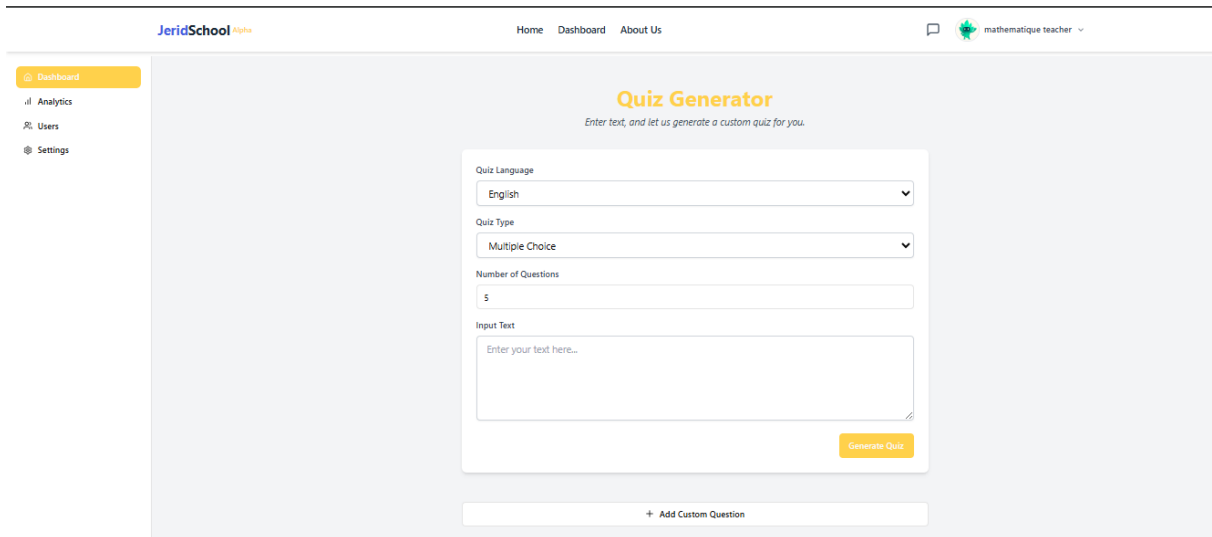
Upon logging into the platform, teachers are presented with a personalized dashboard offering a range of essential tools designed to streamline their daily teaching activities. The main menu includes the following features:

- **Attendance List:** View the list of students for each class and keep track of their attendance.
- **Mark Absent:** Quickly mark students as absent with just a few clicks, enabling real-time attendance tracking.
- **Class View:** Access detailed information about each class, including enrolled students and subjects taught.
- **Timetable:** View the teacher's personalized schedule, including classes, periods, and room assignments.
- **Quiz Generator:** Create interactive quizzes with customizable questions to assess student understanding.
- **Whiteboard:** Use the integrated digital whiteboard for interactive teaching and real-time collaboration during lessons.
- **My Notes:** Save and organize personal notes or teaching materials for future reference.
- **Focus Room:** A distraction-free environment designed for focused teaching sessions or private interactions with students.

These tools are accessible from the teacher's interface and are designed to enhance productivity, simplify administrative tasks, and support engaging digital learning experiences.

Day	Time	Subject	Class	Room
Sunday	No lessons			
Monday	No lessons			
Tuesday	8:00 -> 8:45	math	Class 1 A	salle 3
	8 -> 9	math	Class 1 A	salle 3
Wednesday	No lessons			
Thursday	8:00 -> 8:45	math	Class 1 A	salle2
	8 -> 9	math	Class 1 A	salle2
Friday	No lessons			
Saturday	No lessons			

Figure 4. 54: Teacher Timetable interface



The Quiz Generator interface is part of the JeridSchool Alpha application. It features a sidebar with navigation links: Dashboard, Analytics, Users, and Settings. The main content area is titled "Quiz Generator" with the subtitle "Enter text, and let us generate a custom quiz for you." The form includes a "Quiz Language" dropdown set to "English", a "Quiz Type" dropdown set to "Multiple Choice", a "Number of Questions" input field with a value of 5, and an "Input Text" area with the placeholder "Enter your text here...". A "Generate Quiz" button is located at the bottom right of the form. Below the form is a link to "+ Add Custom Question".

Figure 4. 55: Quizz generator interface

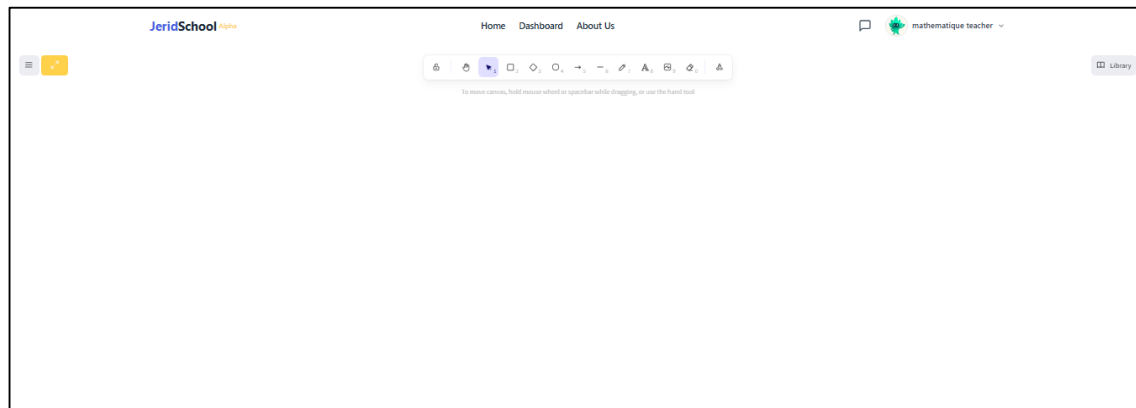


Figure 4. 56: Whiteboard interface

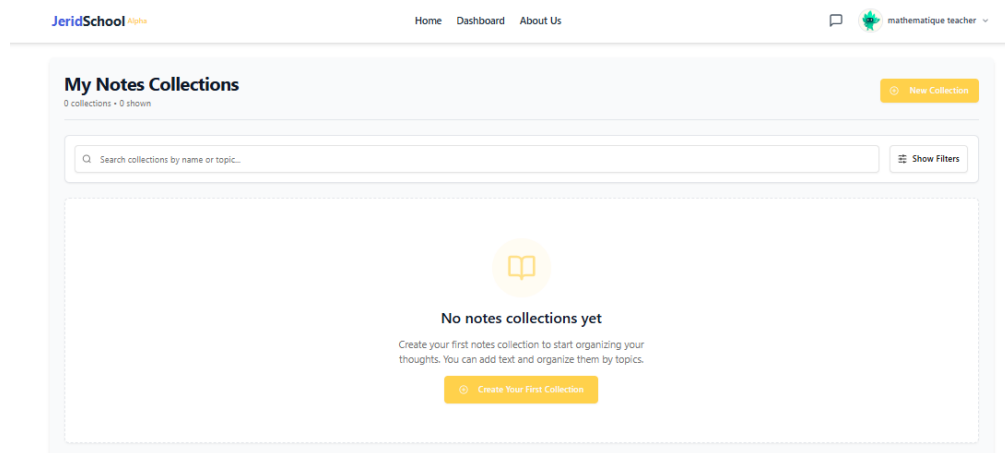


Figure 4. 57: My Notes interface

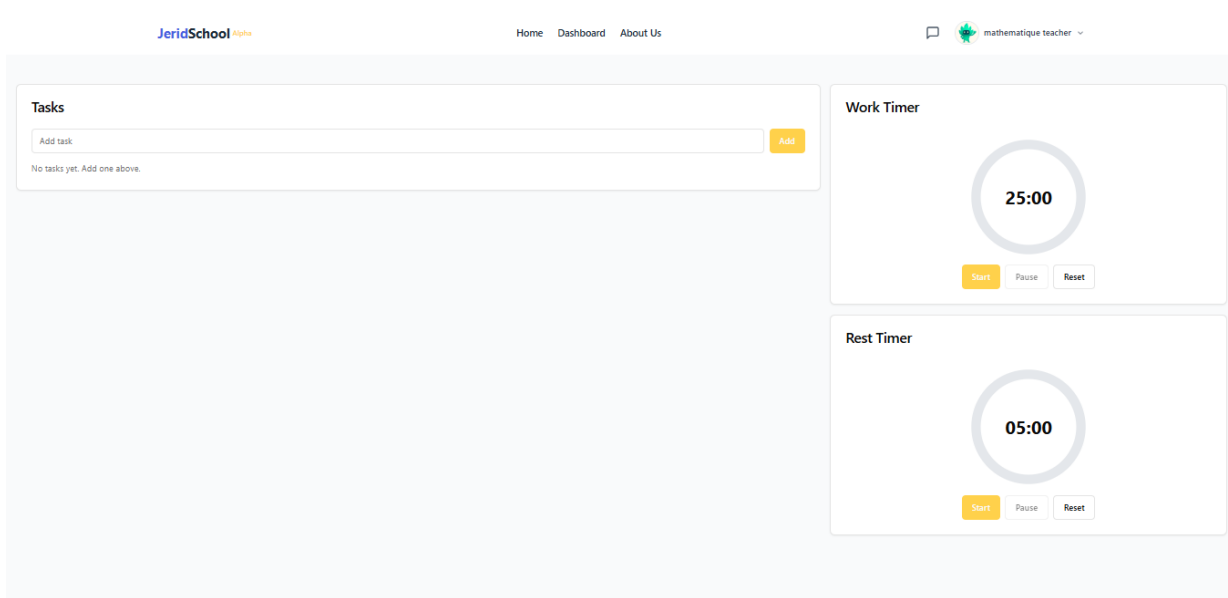


Figure 4. 58: Focus Room interface

To communicate with other users, the teacher must click on the “chat” button, choose or search for the student he wants to talk to and send him his message.

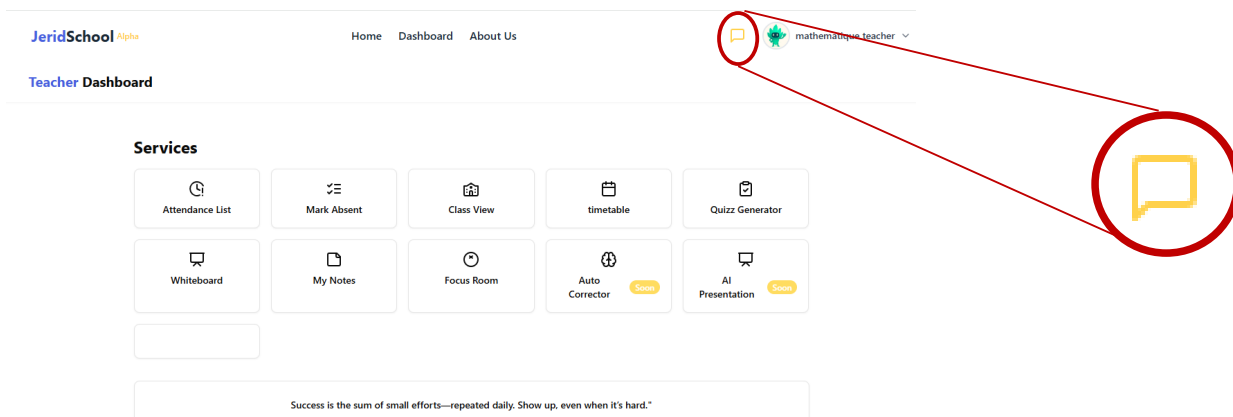


Figure 4. 59: Chat button

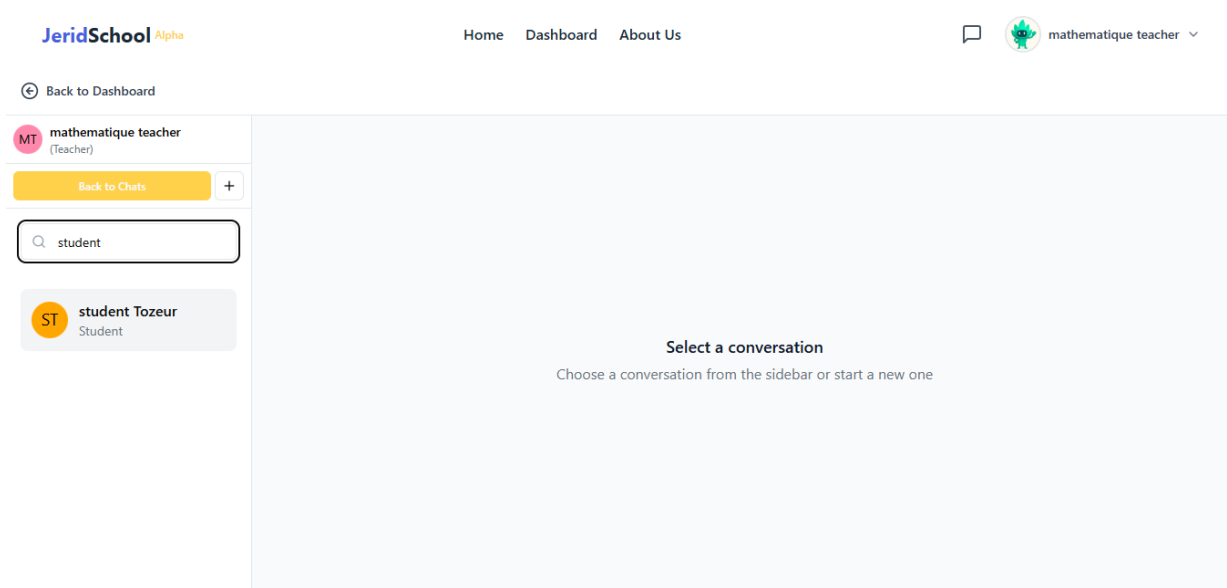


Figure 4. 60: Chat interface

1.7. Student profile

Students are the primary end-users of the application's learning and scheduling features.

Students who are assigned to a school by the headmaster have full access to all platform features related to their classes and academic performance. This includes authenticating into the system, managing their profile, viewing personal timetables and schedules, accessing attendance records, participating in quizzes, and using learning tools such as flashcards and the collaborative whiteboard. They can also communicate with teachers via chat, manage their notes, use the focus room for study, view detailed academic performance reports including exam grades and semester averages, and engage with the AI learning interface to upload files or YouTube links, generate study cards, and interact with the AI chatbot.

In contrast, students who register themselves without being assigned to a school have limited access. They can authenticate, manage their profiles, view basic schedules, and use general communication and learning tools, but they do not have access to features that depend on school-specific data such as class-related activities, detailed academic performance, and AI-powered personalized learning based on their school records.

This section highlights the key elements of the student experience.

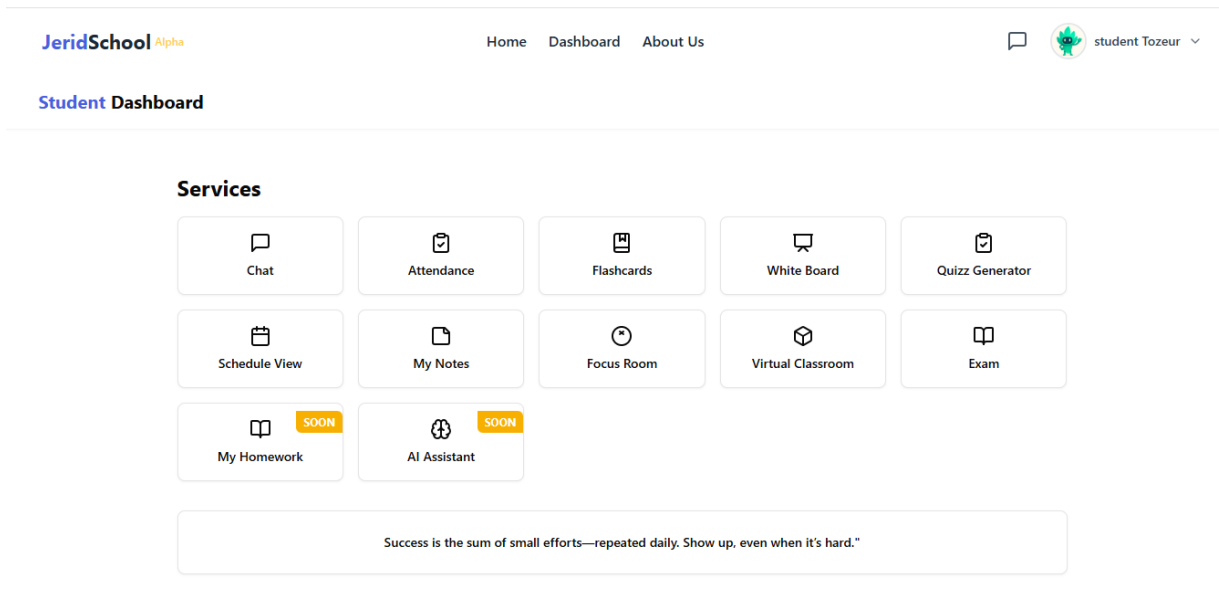


Figure 4. 61: Student Dashboard

The student dashboard provides learners with intuitive access to the tools and resources they need to stay organized and engaged. The menu is designed to support both academic progress and real-time communication. It includes:

- **Chat:** Communicate with teachers and classmates through a secure, real-time messaging system.
- **Attendance:** View personal attendance records and stay informed about presence history.
- **Flashcards:** Study and memorize key concepts through interactive flashcards created by teachers or students.
- **Whiteboard:** Access collaborative whiteboard sessions used during lessons for explanations and illustrations.
- **Quiz Generator:** Take practice quizzes to reinforce learning and prepare for assessments.
- **Schedule View:** Easily view the daily and weekly timetable with class periods, rooms, and subjects.
- **My Notes:** Write, organize, and store personal study notes for review at any time.
- **Focus Room:** A dedicated distraction-free zone for concentrated study sessions or guided activities.
- **Academic Performance:** Track grades for each subject's exams, view semester averages, analyze performance trends, and compare results across semesters to monitor progress.
- **All-in-One AI Learning:** Upload YouTube links, PDFs, or images to automatically extract key concepts into flashcards or engage with an AI chatbot for personalized learning assistance and content interaction.

All in One AI Learning AI Ready

AI Processing Jeridooo AI

AI Content Processing

YouTube File Upload

Upload File

Click to upload or drag and drop
PDF, DOC, TXT, CSV, JPG, PNG (Max 10MB)

Custom Prompt (Optional)

e.g., Create flashcards from this document...

Process File

All in One AI Learning AI Ready

AI Processing Jeridooo AI

AI Content Processing

YouTube File Upload

YouTube URL

<https://www.youtube.com/watch?v=...>

Custom Prompt (Optional)

e.g., Create a quiz focusing on the main concepts...

Process YouTube Video

Figure 4. 62: All in One AI Learning interface

All in One AI Learning AI Ready

AI Processing Jeridooo AI

Jeridooo AI Assistant Clear Chat

Hello! I'm Jeridooo, your AI learning assistant. I can help you with questions about your study materials, explain complex concepts, create study plans, or assist with your learning journey. How can I help you today?
10:59 PM

Hello! How can I assist you today?
11:00 PM

Ask Jeridooo anything about your studies...

Session: c5c73b0... 2 messages

Figure 4. 63: AI chatbot

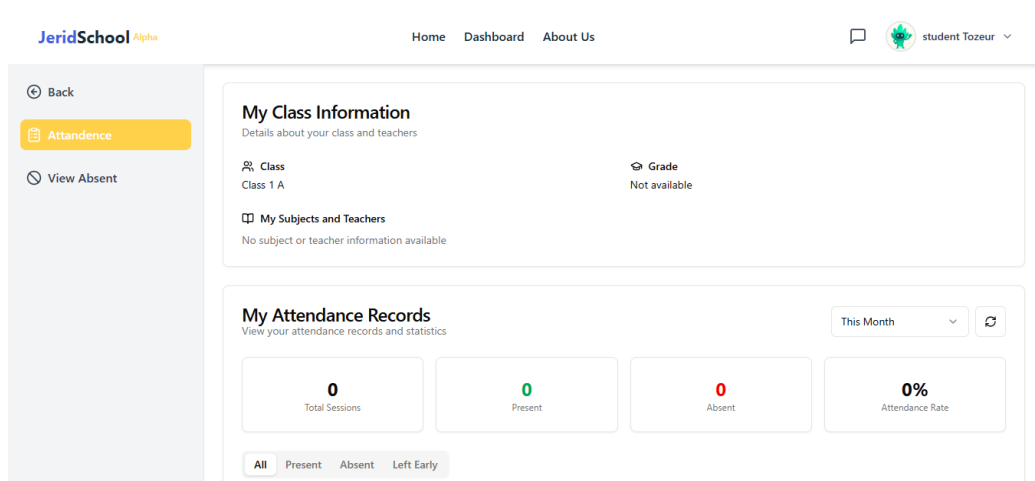


Figure 4. 64: Attendance interface

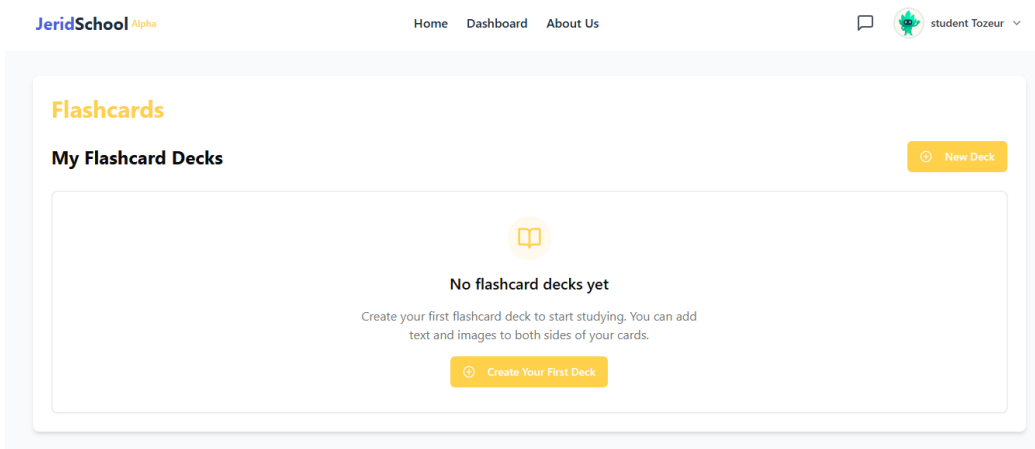


Figure 4. 65: Flashcards interface

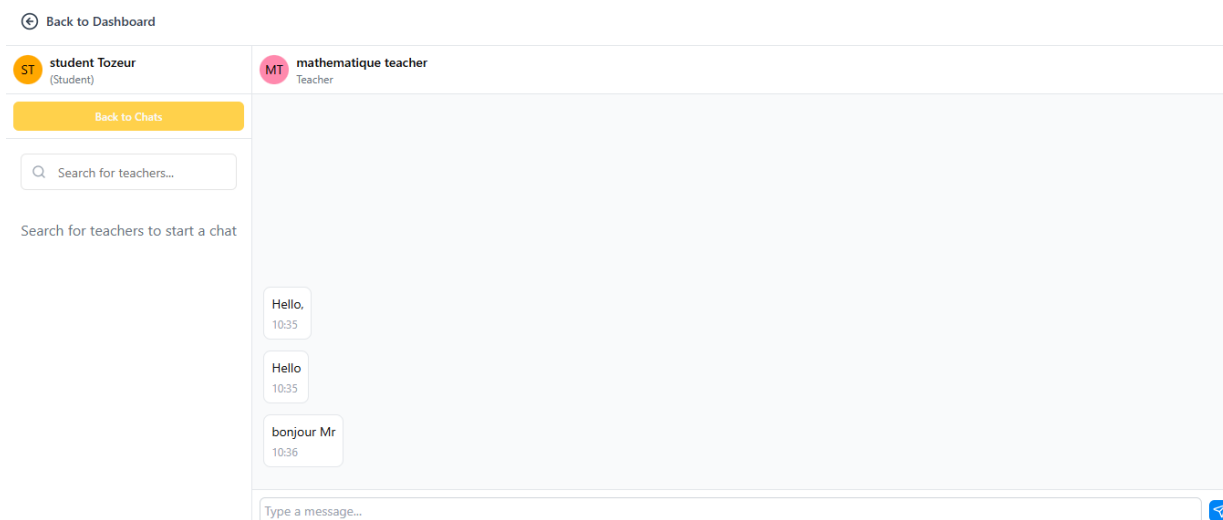


Figure 4. 66: Chat interface

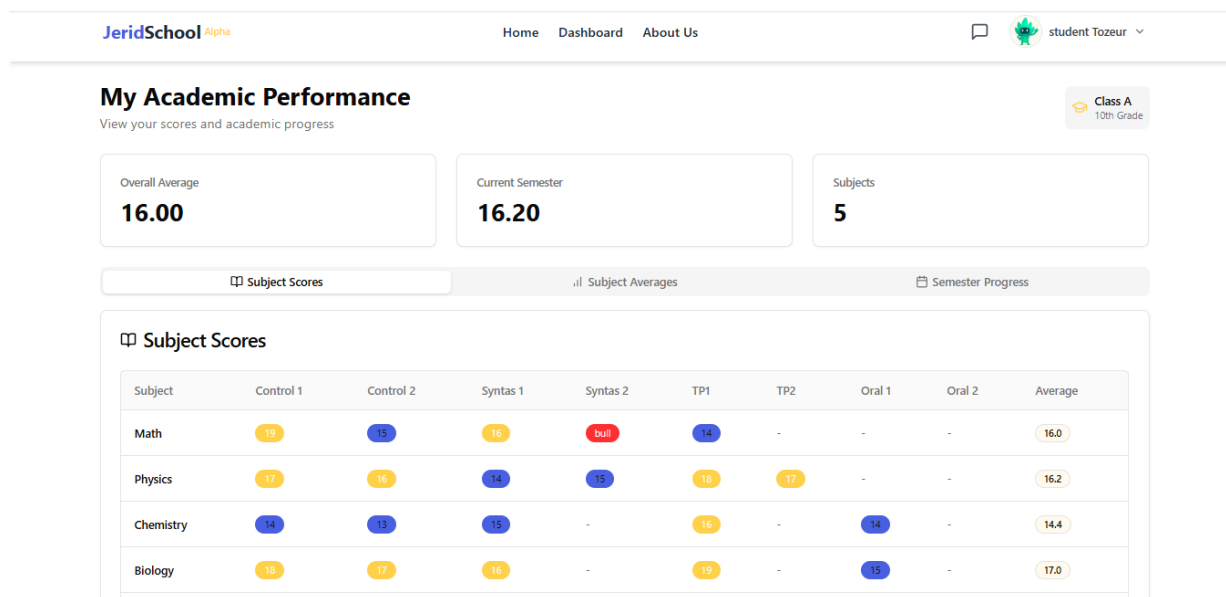


Figure 4. 67: Academic performance interface « Subject scores »

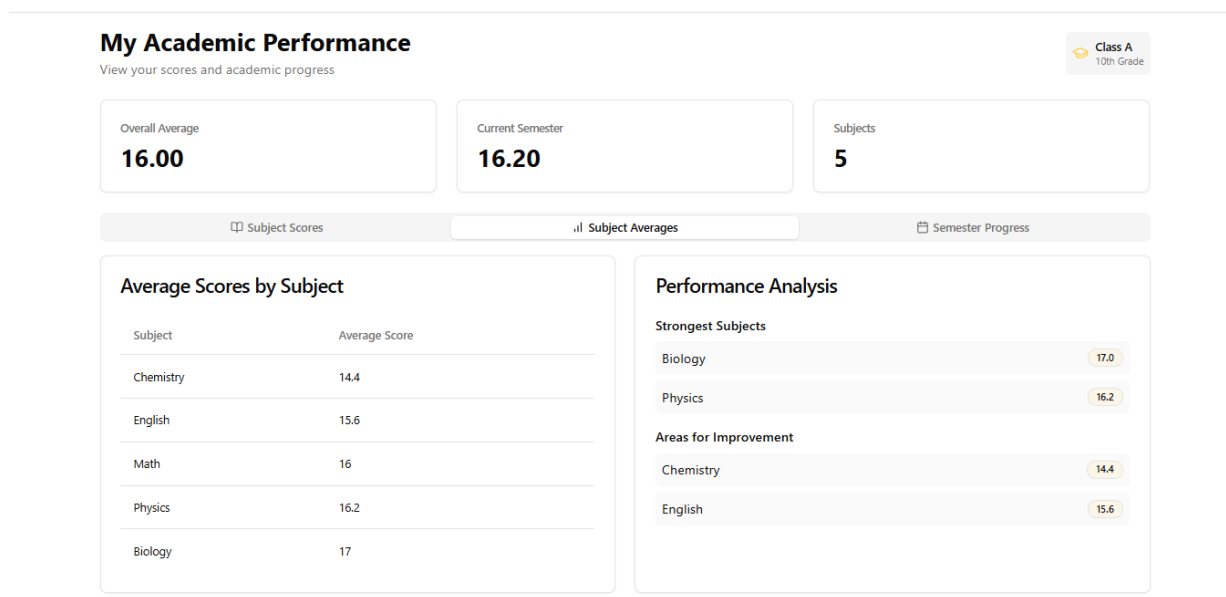


Figure 4. 68: Performance analysis

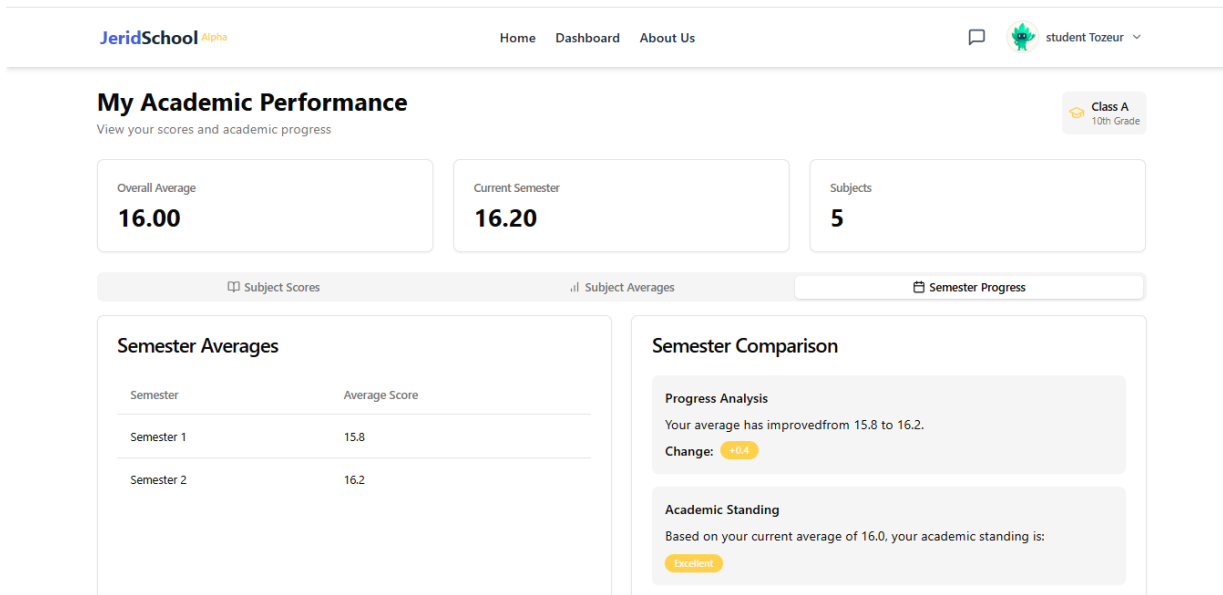


Figure 4. 69: Academic performance interface « Semester averages and comparison »

1.8. Mobile Experience and Responsiveness

Our platform has been carefully designed as a fully responsive web application. This means the same website automatically adapts to various screen sizes whether accessed on a desktop, tablet, or smartphone offering a seamless and optimized experience for all users. The layout, navigation, and user interface elements dynamically adjust to ensure readability, usability, and accessibility on smaller screens. This approach simplifies maintenance and ensures the benefit from the full functionality of the platform without needing to download an additional app.

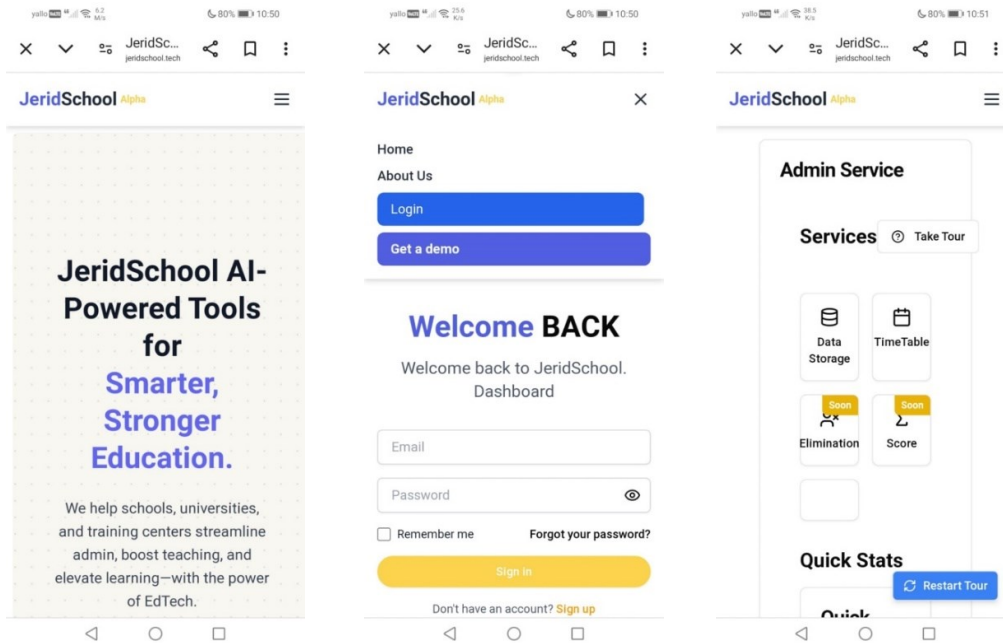


Figure 4. 70: Mobile Experience and Responsiveness

As for Mobile app, we used React native. The chat functionality and the flashcard feature are available on both mobile and web applications. These mobile interface demonstrate the responsive design and user experience across different platforms.

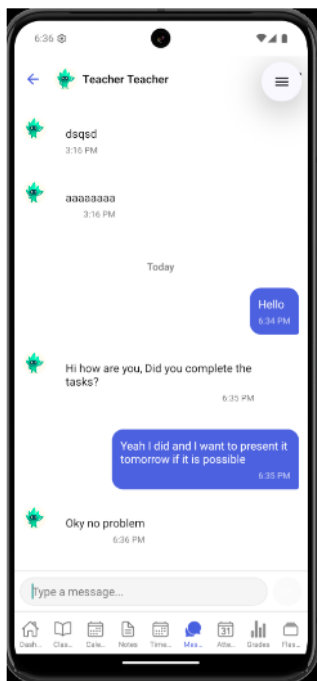


Figure 4. 71: Chat Mobile Interface

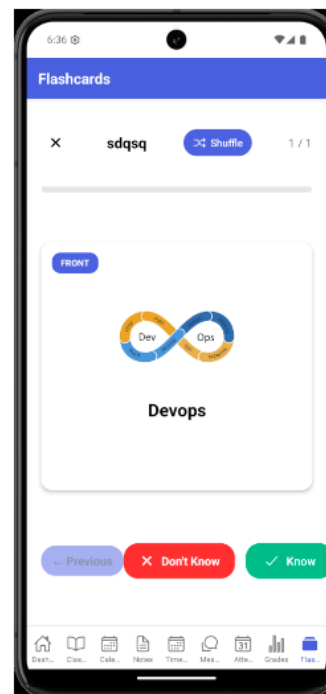


Figure 4. 72: Flashcard Mobile Interface

2. Tests and user Insights

To assess the reliability, performance, and user satisfaction of the developed platform, we conducted a comprehensive evaluation process that combined technical testing with real-world feedback.

The goal was to ensure the system not only met its functional requirements but also delivered a smooth and intuitive experience for all types of users including administrators, teachers, and students. This section presents our testing strategy, key test results, feedback from pilot users, and insights into user acceptance, all of which were crucial for refining and validating the solution before wider deployment.

2.1. Tests and evaluation

The testing and evaluation phase plays a crucial role in ensuring the reliability, performance, and usability of the system. This section presents the overall testing strategy adopted, the types of tests conducted, and a detailed analysis of the test cases, their outcomes, and the issues encountered. It aims to validate that the system meets its specifications and behaves correctly under various scenarios.

A set of key test cases was designed to verify both functional and non-functional requirements. Below are some representative test cases and their outcomes:

Table 4.1: Test cases and their outcomes

Test Case ID	Description	Expected Result	Actual Result	Status
TC01	User login with valid credentials	Redirect to dashboard	Redirect successful	Passed
TC02	Timetable generation for a full academic week	Correct schedule without conflicts	Successfully generated	Passed
TC03	Chat functionality between teacher and student	Messages sent and received in real-time	Functional	Passed
TC04	Invalid input in registration form	Display error message	Error shown correctly	Passed
TC05	Quiz submission with empty answers	Block submission and notify user	Properly handled	Passed

During the testing process, several bugs and inconsistencies were identified and resolved.

Overall, over 90% of test cases passed in the first execution. The remaining cases were fixed and retested successfully before the final deployment.

2.2. Customer Feedback and User Acceptance

After deploying the platform in real educational environments, we invited pilot users to test and evaluate the system. These users included school administrators, teachers, and students from various institutions. The feedback gathered provided valuable insights into both the strengths and areas of improvement for the platform.

Among the pilot users (names are anonymized for privacy):

Mr. Karim H., a school administrator from a private high school in Tunis, used the platform to manage class schedules and teacher allocations. He praised its clarity and efficiency, stating, *"It saves me at least 3 hours a week during scheduling season."*

Ms. Salma R., a mathematics teacher, relied on the mobile app to monitor her timetable and interact with students. She remarked, *"The mobile app is smooth and helps me track changes on the go."*

Youssef B., a 10th-grade student, used the app daily to check his schedule, access flashcards, and prepare for quizzes. He shared, *"It feels like the school finally went digital."*

Overall, the user response was highly positive. Users appreciated the platform's intuitive interface, the automation of complex scheduling tasks, and the enhanced communication between different stakeholders. Students valued the mobile experience, which helped them stay informed in real-time.

Some constructive suggestions were also made, such as the implementation of a dark mode, improved notification settings, and the ability to export schedules as PDF files. These insights were instrumental in shaping our future development roadmap. The high engagement levels and the willingness of institutions to continue using the platform beyond the testing phase reflect strong user acceptance and the real-world impact of the solution.

3. Challenges and encountered difficulties

Throughout the development process, I encountered a wide range of challenges that tested my technical expertise, problem-solving skills, and capacity to independently manage a full-scale software project. These obstacles not only pushed me to adopt creative solutions but also played a pivotal role in shaping the robustness, scalability, and real-world readiness of the final product.

One of the most technically demanding tasks was the design and implementation of a dynamic scheduling algorithm capable of handling complex constraints such as subject hours, teacher availability, classroom limitations, and public holidays without sacrificing performance or accuracy. Multiple algorithmic strategies were explored, including brute-force, greedy, and hybrid methods, before achieving a reliable and efficient solution.

While I worked independently, I maintained close collaboration with my academic and professional supervisors for guidance and validation. To manage the project efficiently, I adopted agile development principles and utilized tools like GitHub for version control. These tools helped ensure a structured, transparent, and iterative workflow throughout the project lifecycle.

Deploying a full-stack application with real-time features also posed significant technical challenges. I conducted deep research and practical experimentation to integrate services such as authentication service, cloud hosting, and third-party APIs for features like analytics and real-time messaging. Each integration had to be carefully configured to ensure security, compatibility, and performance.

Other technical challenges included:

- Designing a user role management system that accommodates the specific permissions and responsibilities of students, teachers, administrators, and school owners.
- Building a flexible timetable generator that adapts to real-world dynamics such as holidays, last-minute changes, and subject-specific room requirements.
- Implementing socket-based real-time communication across both web and mobile views.
- Ensuring secure backend deployment while managing database scalability and cloud infrastructure.

Despite the complexity of the project, each challenge became an opportunity for growth. The process enhanced my mastery of full-stack architecture, improved my adaptability, and sharpened my decision-making when facing real-world constraints. These lessons represent a strong foundation for future projects and my professional career in software engineering.

4. Lessons Learned

Working on this project provided an invaluable opportunity to deepen both my technical and professional skills. One of the key lessons I learned was the importance of iterative development and

flexibility. Building a complex system like a school management and timetable platform required constant refinement, continuous testing, and the ability to pivot quickly when early assumptions proved incorrect.

I also gained practical experience in solving real-world problems using advanced algorithmic strategies, especially in handling constraint-based scheduling. The process taught me how to balance performance with correctness and how to move beyond theoretical approaches to find pragmatic, working solutions.

On the organizational level, I learned how to independently manage a large-scale project from planning and architecture to deployment while leveraging tools such as GitHub, Trello, and CI/CD pipelines. This experience reinforced the value of version control, documentation, and clean development practices in maintaining code quality and team (or supervisor) communication.

Finally, integrating AI-powered features, cloud infrastructure, and real-time services helped me understand the complexity of modern full-stack systems and the need for thoughtful, scalable architecture. These lessons will guide my approach to future projects and strengthen my ability to contribute to innovative software solutions in real professional settings.

5. Impact and Real-World Deployment

This project was not only developed as a university capstone, but also envisioned as the foundation of a scalable, real-world startup. What began as a prototype evolved into a fully functional and deployable school timetable platform, currently used in live educational settings.

A major milestone in this journey was being selected for the Microsoft Azure for Startups program, which has an acceptance rate of less than 10%. This rare recognition provided us with valuable resources such as free cloud infrastructure, substantial Azure credits, and access to enterprise-grade development and deployment tools. These assets significantly empowered our DevOps strategy, enabling secure deployment, scalable backend services, and advanced monitoring capabilities.

Receiving Azure's backing was not only a technical advantage but also a strong validation of the project's innovation, scalability, and market relevance. It allowed us to reduce operational costs while delivering high-performance, production-ready software under real-world conditions.

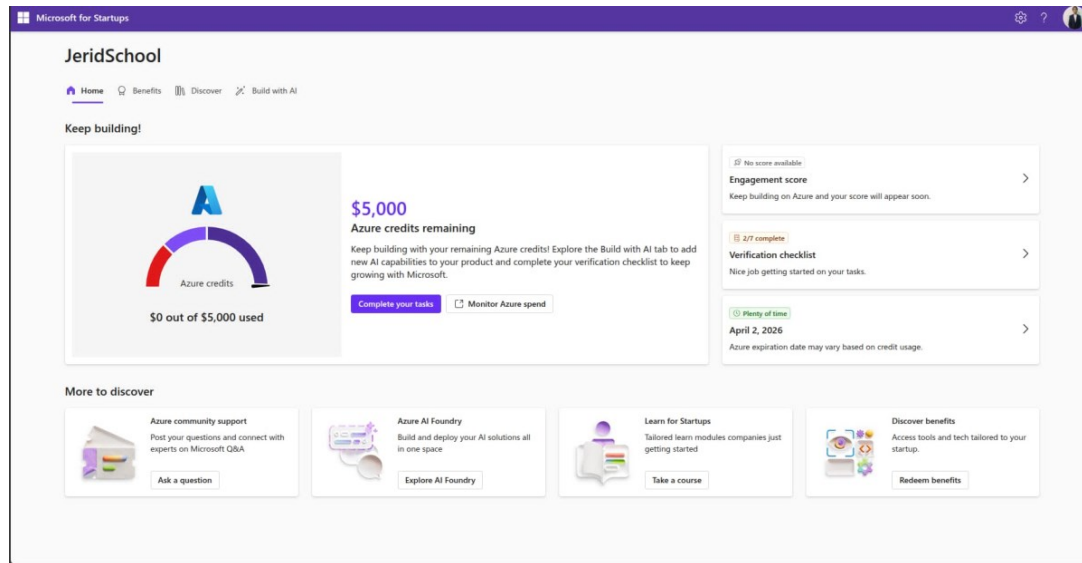


Figure 4. 73: Azure found

6. Future Perspective

To ensure the long-term sustainability, scalability, and relevance of the platform, we have established a clear roadmap for future development. This vision includes both functional enhancements and technical improvements aimed at refining user experience and system performance.

One major focus is the enhancement of the timetable generation algorithm through AI-based optimization techniques. Additionally, we plan to incorporate machine learning models to analyze user behavior and predict student performance trends. These features will allow the platform to offer more personalized and data-driven insights to educators and students alike. Another key goal is the integration of real-time messaging functionalities to further streamline communication between administrators, teachers, and students.

To support a growing international user base, we also aim to implement multilingual capabilities, making the platform more accessible to non-English-speaking users and expanding its global reach.

From a technical maintenance perspective, we are adopting continuous integration and continuous deployment (CI/CD) pipelines to automate testing and delivery of new features. This approach will ensure that future updates are deployed quickly and reliably. Automated testing frameworks will be established to maintain system integrity and minimize downtime. As the platform

scales, regular performance monitoring and comprehensive security audits will be essential to uphold user trust and maintain enterprise-grade reliability.

Beyond this academic project, development of the platform will continue with several high-impact features in the pipeline, including:

- AI-powered generation of personalized study plans.
- A notification system for real-time alerts regarding schedule changes.
- Enhanced whiteboard tools, including support for drawing and annotations.
- Expanded analytics and reporting capabilities for school administrators.

The platform remains operational and actively used. Ongoing monitoring processes are in place to promptly identify and resolve critical issues, ensuring that users continue to benefit from a stable and evolving educational solution.

7. Final Thoughts

While the platform is already functional and deployed in real-world environments, several enhancements are planned to increase its impact and scalability. This project is just the beginning. The application is already helping users, and with continued development, it has the potential to grow into a fully scalable SaaS product. The journey from an academic prototype to a startup-backed platform has been challenging but incredibly rewarding.

The system will continue to evolve, adapt, and serve schools that seek to digitize their academic management in an efficient and user-friendly way.

Conclusion

In this chapter, we present the implementation details of the project, including the working environment, the technological stack chosen, and the deployment setup. We then describe the validation tests conducted to ensure the system's correctness, performance, and usability. The chapter concludes with feedback from users and considerations for future improvements.

General Conclusion

The development of JeridSchool marks a significant step toward digital transformation in the education sector, merging advanced technologies with real academic needs. From dynamic timetable generation and role-based dashboards to AI-enhanced learning interfaces and scalable multi-tenant deployment, the platform offers a well-rounded, intuitive, and future-ready environment for all users.

Through this project, we explored a wide range of software engineering practices, from conceptual design and sprint-based development to validation and user feedback collection, while addressing practical constraints of usability, responsiveness, and performance. Moreover, the integration of artificial intelligence in document processing and personalized study support has shown the potential of AI to revolutionize educational tools.

The integration of cutting-edge tools like FastAPI, Docker, and GitHub Actions enabled us to implement CI/CD pipelines and microservices that meet modern software delivery standards. The inclusion of AI-powered features, such as semantic document search and quiz generation, further demonstrates the platform's potential to innovate learning environments.

Beyond the technical achievements, this project has reinforced the importance of user-centric design, continuous iteration, and the value of scalable architectures in real-world applications. The experience gained during this journey has not only enriched our technical expertise but also highlighted our ability to design impactful, socially beneficial digital systems.

Bibliography

- [1]: www.codecooperation.com
- [2]: <https://www.techtarget.com/searchsoftwarequality/definition/agile-software-development>
- [3]: <https://kanbanboard.co.uk/kanban-board>
- [4]: <https://www.oracle.com/ch-fr/application-development/what-is-devops/>
- [5]: <https://www.paloaltonetworks.com/cyberpedia/what-is-a-container/>
- [6]: <https://www.blackduck.com/glossary/what-is-cicd.html>
- [7]: <https://www.docker.com/>
- [8]: <https://nginx.org/en/>
- [9]: <https://www.digitalocean.com/>
- [10]: <https://www.infoworld.com/article/2252875/arch-linux-a-simpler-kind-of-linux.html>
- [11]: <https://staruml.io/>
- [12]: <https://code.visualstudio.com/>
- [13]: <https://docs.github.com/en/get-started/start-your-journey/about-github-and-git>
- [14]: <https://docs.github.com/en/get-started/using-git/about-git>
- [15]: <https://www.figma.com/>
- [16]: <https://www.postman.com/product/what-is-postman/>
- [17]: <https://fr.overleaf.com/about/features-overview>
- [18]: <https://legacy.reactjs.org/>
- [19]: <https://tanstack.com/query/latest>
- [20]: <https://discourse.threejs.org/t/what-exactly-is-three-js-for/36588>
- [21]: <https://nodejs.org/en/learn/getting-started/the-v8-javascript-engine>
- [22]: <https://nestjs.com/>
- [23]: <https://fastapi.tiangolo.com/>
- [24]: <https://www.capicua.com/blog/about-python>
- [25]: <https://www.postgresql.org/>
- [26]: <https://research.ibm.com/publications/docling-an-efficient-open-source-toolkit-for-ai-driven-document-conversion>
- [27]: <https://openai.com/index/whisper/>

Abstract

This project consists of designing and developing an intelligent school management platform with a smart timetable generator. It includes role-based access (guest, student, student assigned to a school, teacher, headmaster, and super administrator), real-time communication tools, and AI-powered learning features. The system is multi-tenant, responsive, and already deployed on the web. It aims to optimize scheduling, improve collaboration, and digitize the academic experience for educational institutions.

Keywords: School Management System, Timetable Generation, AI, Web Socket, Multi-Tenancy, CI/CD, React, Node.js, PostgreSQL.

Résumé

Ce projet consiste à concevoir et développer une plateforme intelligente de gestion scolaire avec un générateur automatique d'emploi du temps. Elle intègre des accès selon les rôles (visiteur, élève, élève assigné à une école, enseignant, directeur d'école, et super administrateur), des outils de communication en temps réel et des fonctionnalités d'apprentissage assisté par l'IA. Le système est multi-tenant, responsive, et déjà déployé. Il vise à optimiser la planification, améliorer la collaboration et digitaliser l'expérience éducative.

Mots-clés : Système de gestion scolaire, Génération d'emploi du temps, Intelligence artificielle (IA), WebSocket, Multi-locataire (Multi-Tenancy), Intégration et déploiement continu (CI/CD), React, Node.js, PostgreSQL.

الملخص

يهدف هذا المشروع إلى تطوير منصة ذكية لإدارة المدارس تتضمن نظامًا آليًا لإنشاء جداول الحصص. توفر المنصة صلاحيات مختلفة (زائر، طالب، طالب مُعين في مدرسة، مُعَلِّم، مدير مدرسة، مُدير مُشْرِف)، وأدوات تواصل فوري، وميزات تعليمية مدعومة بالذكاء الاصطناعي. النظام متعدد المؤسسات، متجاوب، وتم نشره فعليًا. هدفه تحسين تنظيم الجداول، وتعزيز التفاعل، ورقمنة التجربة التعليمية.

الكلمات المفتاحية : نظام إدارة المدارس، إنشاء الجداول الزمنية، الذكاء الاصطناعي (AI)، مقبس ويب (WebSocket)، تعدد المستأجرين (Multi-Tenancy)، التكامل والنشر المستمر (CI/CD)، ريكيت، نود جي إس، بوستجريسل.